

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ПРОЕКТИРАНЕ И РАЗРАБОТВАНЕ НА
БИБЛИОТЕКА НА C++ ЗА
ОБЕКТНО-РЕЛАЦИОННО СЪПОСТАВЯНЕ
ПО ВРЕМЕ НА КОМПИЛАЦИЯ**

ДИПЛОМНА РАБОТА

Изготвил:

Алекс Цветанов, фак. № 121222225

Специалност: Компютърно и софтуерно инженерство

Научен ръководител:

доц. д-р инж. Галя Павлова

София, 2026

СЪДЪРЖАНИЕ

I.	Въведение	3
1.1.	Постановка на проблема	3
1.2.	Цел и задачи на разработката	4
1.3.	Обхват и ограничения	5
1.4.	Структура на изложението	5
II.	Преглед на съществуващите решения	7
2.1.	Класификация на подходите	7
2.2.	Критерии за сравнение	9
2.3.	Платформено-специфични клиентски библиотеки	10
2.4.	Асинхронност в съществуващите клиентски стекове	11
2.5.	Конструктори на заявки по време на изпълнение	11
2.6.	Типизирани SQL решения	12
2.7.	Класически рамки за обектно-реляционно съпоставяне и кодогенерация	12
2.8.	Граници на реляционно центрираните абстракции	13
2.9.	Позициониране на настоящата разработка	14
2.10.	Изводи от сравнителния анализ	15
III.	Теоретични основи на библиотеката и моделите за съхранение	16
3.1.	Таксономия на проблемната област	16
3.2.	Обектно-реляционното съпоставяне като архитектурен подход	17
3.3.	Статичен модел на данните в C++	18
3.4.	Типизирано междинно представяне на заявките	20
3.5.	Теория на връзките и стратегии за съхранение	22
3.6.	Реляционен модел	23
3.7.	Документен модел	24
3.8.	Ключ-стойност модел	24
3.9.	Графов модел	24
3.10.	Широк колонен модел	25
3.11.	Тестова опора на теоретичните твърдения	26
3.12.	Граници на унифицираната абстракция	27

IV. Теория на корутините и асинхронното изпълнение в C++	28
4.1. Процеси, нишки, задачи и корутини	28
4.2. Синтаксис, протокол за изчакване и езикови свързванеing-и	30
4.3. Корутината като автомат с крайни състояния	30
4.4. Корутинна рамка, обект-обещание, продължение и безопасност	31
4.5. Корутини, набор от нишки и инфраструктура за цикъл за събития	32
4.6. Значение за библиотеката за обектно-релационно съпоставяне по време на компиляция	33
V. По време на компиляция ORM архитектура	35
5.1. Архитектурни цели и ограничения	35
5.2. Слоеста организация и разделяне на отговорностите	36
5.3. Слой на същностите като архитектурна основа	37
5.4. Междинното представяне на заявките като централен архитектурен компонент	38
5.5. Конекторен слой и формалният договор	39
5.6. Потребителският фасаден слой db<DB>	41
5.7. Миграции като архитектурно разширение	42
5.8. Реализация на основните подсистеми	43
5.8.1. Мини казус с типовете User и Post	43
5.8.2. Скаларни свойства и именуване на контейнерите	44
5.8.3. Обвивки на полета, правила и заместители	44
5.8.4. Изграждане на обекти на заявки	45
5.8.5. Изпълнение, резултати и достъп до полета	46
5.8.6. Подготвени заявки (prepared заявки) като артефакт за многократно изпълнение	47
5.8.7. Миграционен слой и DDL генериране	48
5.8.8. Нишкова безопасност и transaction помощници	49
5.8.9. Извод за реализацията	51
5.9. Архитектурни инварианти	52
5.10. Архитектурен извод	52
VI. Асинхронна архитектура на изпълнението	53

6.1.	Архитектурна позиция на асинхронния слой	53
6.2.	Task<T> като носител на жизнения цикъл на корутината	54
6.3.	thread_pool и run_on_pool(): универсалният път на изпълнение	54
6.4.	async_db<DB> и диспечирание, ограничено от възможностите	55
6.5.	IoContext и платформено-специфична неблокираща интеграция	58
6.6.	Поддържане на пул от връзки, съобразено с корутини, и помощници за транзакции	58
6.7.	Архитектурен извод за асинхронния слой	60
VII.	Изпълнителни модели на свързващите компоненти и платформено-специфични клиентски библиотеки	62
7.1.	Методика на анализа	62
7.2.	Релационен базов сценарий: SQLite	64
7.3.	Релационна преносимост: PostgreSQL и MySQL	64
7.4.	Документен сценарий: MongoDB	65
7.5.	Сценарий ключ-стойност: Redis	65
7.6.	Графов сценарий: Neo4j	66
7.7.	Сценарий за широки колонни бази: Cassandra	67
7.8.	Синтезирана матрица на възможностите	68
7.9.	Съпоставка между тестови доказателства и договор на компонента	72
7.10.	Сравнителен синтез	73
VIII.	Верификация и емпирична оценка	75
8.1.	Стратегия на верификацията	75
8.2.	Каталог на модулните и интеграционните доказателства	77
8.3.	Карта на зрелостта според доказателствената опора	81
8.4.	Методология на сравнителен тест оценката	83
8.5.	Основни резултати за пропускателна способност	85
8.6.	Нулева латентност на изчакване (0ns) и режимни разходи за планиране на корутините	86
8.7.	Интерпретация на микротестовите (microсравнителен tests) за режимни разходи	88
8.8.	Ограничения на емпиричната оценка	88

IX. Надграждаемост чрез нови конектори	90
9.1. Формален договор на конекторния слой	90
9.2. Задължителни елементи на минимален конектор	91
9.3. Типизиране, сигнатури и именуване	92
9.4. Механизъм на възможностите и ограничения по време на компилация	93
9.5. Минимален и експлоатационен скелет на конектора	94
9.6. Транзакции, нишкова безопасност и подготвени заявки	95
9.7. Миграции и интеграция, съобразена със схемата	96
9.8. Методика за добавяне на нов свързващ компонент	96
9.9. Критерии за „напълно имплементиран и функциониращ“ конектор	98
9.10. Верификация чрез тестове	98
9.11. Оценка на наличните конектори	99
X. Ограничения и бъдещо развитие	100
10.1. Стратегически направления за развитие	100
10.2. Разширяване на покритието на целеви системи на живо	101
10.3. Пълна интеграция, съобразена със схемата, и интроспекция по време на изпълнение	102
10.4. Нишкова безопасност и жизнен цикъл на връзките	103
10.5. Ниско ниво на изпълнение и оптимизации на мрежовия протокол	103
10.6. Интеграция с отражение (отражение) в C++26 и допълнителна автоматизация	104
10.7. Разширяване на договора на конектора и таксономията на възможностите .	105
10.8. Необходимост от по-строга емпирична оценка	106
10.9. Обобщение	107
XI. Заключение	108
11.1. Обобщение на решената задача	108
11.2. Обобщение на постигнатите резултати	109
11.3. Научни и приложни приноси	110
11.4. Основни ограничения и граници на обобщението	111
11.5. Значение за практиката и за бъдещите изследвания	112
11.6. Заключителни бележки	112

РЕЗЮМЕ

Настоящата дипломна работа разглежда проектирането, реализацията и оценяването на библиотека за обектно-реляционно съпоставяне по време на компилация за C++23 [1]. Работата е организирана около две допълващи се оси. Първата е моделирането на данните и заявките по време на компилация, при което логическата структура на достъпа до данни не се описва чрез низове, интерпретирани по време на изпълнение, а чрез статично типизирани `constexpr` конструкции. Втората е базираната на корутини асинхронна архитектура, чрез която същият типове ограничен модел се разширява към неблокиращо или псевдо-неблокиращо изпълнение според възможностите на конкретния драйвер на свързващия компонент.

Разработената библиотека използва статичен модел на данните, изразен в C++ чрез конструкции като `property<T,Name>`, `relationship<...>` и `table_name_trait<T>`, както и типизирано междинно представяне на заявките, реализирано чрез `select_query`, `insert_query`, `update_query` и `delete_query`. Този логически слой е независим от конкретния тип база данни. Материализацията към реляционни и нереляционни системи се осъществява чрез специализации на `connector_trait<DB>`, които декларираат поддържаните възможности и превеждат една и съща логическа заявка към Structured Заявка Language (SQL), Binary JSON (BSON)-подобни филтри, Redis команди, Cypher или Cassandra Заявка Language (CQL) според избора свързващ компонент.

Съществено разширение на архитектурата е асинхронният слой, изграден върху корутини, `Task<T>`, `thread_pool`, `async_db<DB>` и управление на връзки, съобразено с корутини. Показано е, че единна асинхронна абстракция може да бъде реализирана по два начина: чрез изнасяне на блокиращи операции към набор от нишки и чрез платформено-специфични неблокиращи интерфейси за драйвери, които ги предоставят. Така работата не само описва логическа OPC абстракция, но и изследва изпълнителния модел, чрез който тази абстракция остава практически използвана при конкурентни натоварвания.

В дипломната работа се анализират както реляционните сценарии за SQLite, PostgreSQL и MySQL, така и нереляционните сценарии за MongoDB, Redis, Neo4j и Cassandra. Показано е как един и същ C++ модел диктува логическата схема, но се материализира различно като таблица, документ, ключ-стойност структура, графов възел

или широк колонен ред. Отделно се разглеждат клиентските библиотеки на съответните платформи и техните семантики на изпълнение, за да се определи къде библиотеката може честно да предложи платформено-специфично асинхронно поведение и къде правилният избор остава синхронен драйвер с контролирано изнесено изпълнение.

Верификацията на разработката се основава на модулни и интеграционни тестове от хранилището, а емпиричната оценка се допълва от сценарии за измерване на производителността за сравнение между синхронния и асинхронния режим. Те покриват изграждането на заявките, ограничаването на операции въз основа на възможностите, подготвените заявки, миграциите, нишката безопасност, инфраструктурата за корутини и поведението на конкретните конектори при работа с реални или имитиращи драйвери. Това позволява формулиране не само на архитектурен модел, но и на конкретни критерии за „напълно имплементиран и функциониращ“ конектор и за практически оправдан асинхронен слой.

Основният принос на дипломната работа е в четири направления. Първо, предложен е практически използваем модел за библиотека за обектно-релационно съпоставяне по време на компилация, който съчетава статична типова безопасност и архитектура, независима от свързващия компонент. Второ, показано е как C++ описанието на данните може да служи като първичен източник за логическа схема при различни типове бази данни. Трето, изградена е базирана на корутини асинхронна архитектура, която разширява модела на конектора без компромис с ограниченията по време на компилация. Четвърто, формализиран е договор на конектора, чрез който библиотеката може да се надгражда с нови реализации на свързващи компоненти и асинхронни възможности без промяна в потребителския програмен интерфейс за заявки.

Ключови думи: C++23, обектно-релационно съпоставяне по време на компилация, статична типова безопасност, корутини, асинхронно изпълнение, `connector_trait`, релационни и нерелационни бази данни, емпирична оценка

I. ВЪВЕДЕНИЕ

1.1. Постановка на проблема

Достъпът до данни остава едно от местата, в които разминаването между езика на приложението и модела на съхранение създава най-много структурни грешки. В традиционните слоеве за достъп до бази данни заявките се описват като низове, които се валидират едва при изпълнение. Това означава, че неправилни имена на колони, несъвместими типове, невалидни JOIN конструкции или неправилно подадени параметри се откриват късно — често върху реална база данни и след допълнителни ръчни тестове. Същевременно смяната на свързващия компонент от релационен към документен, графов или ключ-стойност модел обикновено изисква пълно пренаписване на слоя за достъп до данни.

Обектно-релационното картографиране е класическият подход за свързване на обектния модел на приложението с персистентното съхранение [2]. Повечето реализации на обектно-релационно съпоставяне (ORC) обаче остават силно зависими от механизми по време на изпълнение: SQL низове, кодогенерация, анотации, макроси или динамични описания на схема. В контекста на C++ това създава допълнителен проблем. Езикът предлага мощна система от шаблони, constexpr изчисления и concepts, но тези механизми често не се използват докрай в библиотеките за достъп до данни. Настоящата работа изследва доколко е възможно компилаторът да поеме ролята на първи валидатор на структурата на заявката и на съвместимостта със свързващия компонент.

Съществена особеност на разработваната система е, че въпреки името библиотека за обектно-релационно съпоставяне по време на компилация, архитектурата ѝ не е ограничена само до релационния модел. Напротив, тя използва еднакъв C++ модел на данните и еднакво междинно представяне на заявките като логически слой над релационни и нерелационни системи. Това поставя втори изследователски въпрос: кои части на ORC абстракцията са универсални и кои неизбежно трябва да се ограничават чрез базиран на възможности договор на свързващия компонент.

Третият проблемен пласт е свързан с начина на изпълнение. Дори когато логическият модел и междинното представяне на заявките са типове коректни, практическата стойност на една съвременна библиотека за достъп до данни зависи и от това дали тя може да интегрира асинхронно изпълнение без разпадане на типовете

гаранции и без принудително преминаване към стил с функции за обратно извикване (функция за обратно извикване). Така дипломната работа се организира около две допълващи се оси: моделиране по време на компилация и асинхронна архитектура на изпълнението.

1.2. Цел и задачи на разработката

Основната цел на дипломната работа е да представи проектирането и реализацията на библиотека за обектно-релационно съпоставяне по време на компилация за C++, която:

1. моделира данните и заявките чрез статично типизирани C++ конструкции;
2. позволява изграждане на SELECT, INSERT, UPDATE и DELETE заявки като `constexpr` обекти;
3. отделя логическата структура на заявката от специфичното ѝ материализиране за дадения свързващ компонент;
4. поддържа релационни и нерелационни конектори чрез `connector_trait<DB>`;
5. използва механизъм за ограничаване на неподдържани операции по време на компилация въз основа на възможностите на компонента;
6. демонстрира как C++ моделът определя логическата схема и физическата структура на данните при различни свързващи компоненти;
7. реализира базиран на корутини асинхронен слой, който надгражда синхронния модел на конектора по формален и типово безопасен начин;
8. формализира договор за добавяне на нови конектори, включително когато те поддържат платформено-специфично асинхронно изпълнение;
9. валидира архитектурата чрез систематични модулни и интеграционни тестове и чрез емпирична сравнителна (сравнителен тест) оценка.

За постигане на тази цел работата решава следните задачи: анализ на съществуващи решения; дефиниране на статичен модел на същностите; изграждане на междинно представяне на заявките; проектиране на свързващ слой и `labels` за възможности;

изграждане на базиран на корутини асинхронен слой; имплементиране на конектори за различни типове бази данни и различни модели на изпълнение; изследване на миграции, подготвени заявки, нишкова безопасност и асинхронно управление на връзки; формулиране на критерии за разширяемост; изграждане на методология за сравнителен анализ за оценка на синхронния и асинхронния режим.

1.3. Обхват и ограничения

Работата разглежда библиотеката като слой за моделиране, изграждане и изпълнение на заявки. В обхвата ѝ попадат моделът на същностите, междинното представяне на заявките, механизмът за заместители (заместители), подготвените заявки, миграционният модел, архитектурата на свързващия компонент, ограничаването въз основа на възможности, базираният на корутини асинхронен слой и конкретните реализации на свързващи компоненти в хранилището. Съществена част от оценяването е сценарийният анализ на SQLite, PostgreSQL, MySQL, MongoDB, Redis, Neo4j и Cassandra, както и сравнителната оценка на синхронния и асинхронния режим на изпълнение.

Извън непосредствения обхват остават оптимизации на производствено ниво като разпределено управление на връзки, автоматизирани стратегии за миграция за всички свързващи компоненти, измерване на производителността в реално време върху пълна матрица от производствени среди и пълна експлоатация на бъдещата функционалност за отражение (отражение) на C++26. Тези направления са разгледани като перспектива в Глава X.

1.4. Структура на изложението

Дипломната работа е организирана по следния начин.

Глава II разглежда съществуващи C++ решения за достъп до бази данни и подходи за обектно-релационно съпоставяне и позиционира разработката спрямо тях.

Глава III представя теоретичните основи на обектно-релационното съпоставяне по време на компилация и моделите за съхранение, необходими за по-късния анализ на релационни и нерелационни сценарии.

Глава IV въвежда теоретичния апарат на корутините и асинхронното изпълнение в C++, който е необходим за разбирането на втория основен принос на библиотеката.

Глава V описва архитектурата на библиотеката за обектно-релационно

съпоставяне по време на компилация: описанията на същностите, междинното представяне на заявките, `connector_trait`, механизмите за възможности и мястото на миграциите в общия модел.

Глава VI представя асинхронната архитектура на изпълнението и показва как базираният на корутини слой се свързва със синхронния програмен интерфейс на конектора, с модела на набор от нишки и с управлението на връзките.

Глава VII анализира как една и съща логическа абстракция и различни стратегии за изпълнение се материализират в релационни, документни, ключ-стойност, графови и широки колонни свързващи компоненти.

Глава VIII обединява верификационните доказателства и емпиричната оценка чрез резултатите от сравнителните тестове.

Глава IX формализира договора за надграждане чрез нови конектори, включително изискванията за възможности и асинхронност.

Глава X очертава ограниченията на текущата версия и възможните направления за бъдещо развитие, а **Глава XI** обобщава резултатите и приносите.

II. ПРЕГЛЕД НА СЪЩЕСТВУВАЩИТЕ РЕШЕНИЯ

Анализът на съществуващите решения е необходим, за да се оцени дали предложената архитектура действително допринася с нова стойност, или само преформулира вече познати практики. При C++ библиотеките за достъп до данни могат да се разграничат две основни оси и една допълнителна изпълнителна перспектива: нивото на абстракция, моментът, в който се изгражда и валидира заявката, и моделът на изпълнение. Нивото на абстракция варира от почти директно използване на клиентски C програмен интерфейс (API) до пълно поведение за обектно-релационно съпоставяне (ORC). Моментът на валидиране може да бъде по време на изпълнение, хибриден или по време на компилация. Изпълнителният модел варира от директни блокиращи извиквания на програмния интерфейс до асинхронни автомати с крайни състояния, специфични за свързващия компонент, и по-високи слоеве, които се опитват да ги уеднаквят.

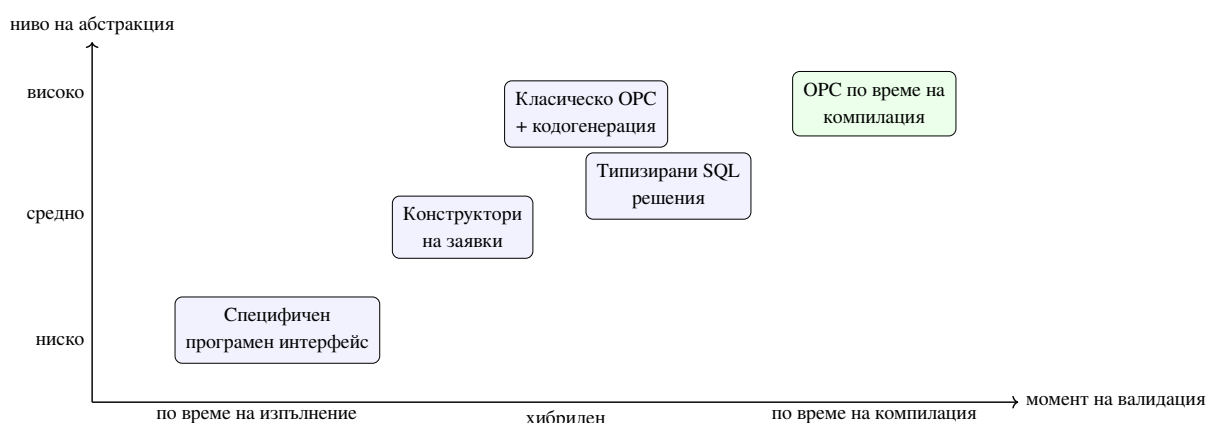
Тези оси са особено важни в контекста на настоящата дипломна работа. Ако се наблюдава само “удобството на програмния интерфейс”, лесно може да се пропусне съществената разлика между библиотека, която просто прави SQL низовете по-четими, и библиотека, която действително прехвърля структурната коректност към компилатора. Ако, от друга страна, се гледа само кога възниква грешката, може да се подцени цената на различните нива на абстракция, сложността на инструментариума (инструментариум) за изграждане и пригодността за работа с повече от един модел на съхранение. А ако се игнорира изпълнителният модел, лесно се стига до погрешното впечатление, че “асинхронност” е еднотипна способност, независимо дали става дума за платформено-специфичен неблокиращ програмен интерфейс, слой с функции за обратно извикване върху цикъл за събития (цикъл за събития) или просто за изнасяне на работата към набор от нишки.

2.1. Класификация на подходите

В най-ниския слой стоят платформено-специфичните клиентски библиотеки като SQLiteC API [3], libpq [4] и MySQLConnector/C [5]. Те предоставят максимален контрол върху заявките, параметрите и резултатите, но не предлагат логическа абстракция. Приложението е принудено да работи директно със SQL низове, индекси на колони, позиции на параметри и типове, специфични за свързващия компонент.

Следващата група са библиотеките за конструиране на заявки по време на изпълнение. Те обикновено подобряват четимостта на заявките и намаляват част от шаблонния код, но продължават да разчитат на изграждане на текстово представяне по време на изпълнение. Типовата безопасност е частична и обикновено спира до границата на извикването на програмния интерфейс. Структурната валидност на заявката остава проблем на времето за изпълнение.

Най-високото ниво на абстракция е подходът за обектно-релационно съпоставяне (ОРС). Там C++ класовете се асоциират с таблици, колони и връзки, а библиотеката генерира необходимите заявки. Предимството е намаляване на шаблонния код. Недостатъкът е, че често се въвеждат външни генератори, макроси, анотации и непрозрачни метаданни по време на изпълнение, които затрудняват проследимостта между C++ кода и реалната заявка.



Фигура 1. Позициониране на основните семейства решения според абстракция и момент на валидация

Фигура 1 показва, че подходът за обектно-релационно съпоставяне по време на компилация не е просто “още една ОРС библиотека”. Той стои в горния десен квадрант, където високото ниво на абстракция се комбинира с ранна структурна проверка. Тази комбинация е рядка, защото изисква повече архитектурна дисциплина в самата библиотека.

Таблица 1. Съпоставка на основните подходи за достъп до данни в C++

Подход	Ниво на абстракция	Валидация	Предимство	Основен недостатък
Специфичен програмен интерфейс	ниско	по време на изпълнение	пълен контрол и близост до драйвера	силна обвързаност с конкретен свързващ компонент
Конструктор на заявки	средно	основно по време на изпълнение	по-четим интерфейс и по-малко шаблонен код	заявката пак се материализира като низ
Типизирано SQL решение	средно към високо	частично по време на компилация	по-добра статична структура за SQL света	обикновено остава ограничено до SQL
Класическо OPC	високо	по време на изпълнение или чрез кодогенерация	удобно съпоставяне на обекти	сложна интеграция в инструментариума и непрозрачност
OPC по време на компилация	високо	по време на компилация за структурата	ранно откриване на структурни грешки и формален договор на конектора	по-висока сложност на шаблонния слой

2.2. Критерии за сравнение

За да бъде сравнението академично пълно, не е достатъчно да се каже, че една библиотека е “по-удобна”, а друга е “по-ниско ниво”. В настоящата работа се използват шест сравнителни критерия:

1. **Ниво на логическа абстракция** — доколко библиотеката позволява моделът на данните да се описва независимо от непосредствения синтаксис на свързващия компонент.

2. **Степен на статична валидация** — кои аспекти на заявката и на модела могат да бъдат проверени преди изпълнение.
3. **Преносимост между свързващи компоненти** — възможно ли е логическият модел да бъде адаптиран към повече от един тип система за съхранение.
4. **Сложност на инструментариума** — изисква ли библиотеката външна кодогенерация, допълнителни стъпки за предварителна обработка (предварителна обработка) или генератори на метаданни.
5. **Прозрачност на грешките и на изпълнението** — доколко разработчикът може да проследи връзката между C++ кода, генерираната заявка и поведението по време на изпълнение.
6. **Изпълнителен модел и асинхронна пригодност** — дали библиотеката предоставя единен и проверим модел за конкурентно или неблокиращо изпълнение, или оставя тази задача изцяло на конкретния драйвер и приложния код.

Тези критерии са пряко свързани с целите на дипломната работа. Ако предложената библиотека трябва да бъде едновременно изразителна и формално проверяема, тогава тя трябва да превъзхожда алтернативите не само по удобство, а по баланса между тези шест измерения.

2.3. Платформено-специфични клиентски библиотеки

SQLite [6], libpq и MySQLConnector/C са представителни за подхода, при който приложението конструира и подава заявките директно към драйвера. Това е най-универсалният и често най-бързият вариант, но поставя тежестта на коректността изцяло върху приложния код. Разработчикът трябва ръчно да поддържа синхрон между имената на полета, поредността на параметрите и реалната схема на базата данни. При промяна на свързващия компонент се променя не само диалектът на заявката, но и целият програмен интерфейс за връзка, подготвяне, свързване (свързване) на параметри и четене на резултати.

Този подход е особено проблематичен, когато един и същ домейн модел трябва да бъде свързан последователно с повече от един тип база данни. В такъв случай приложението започва да натрупва условни пътища, специфични за дадения свързващ

компонент SQL низове и дублирана логика за попълване на резултати. След определен размер на системата това намалява не само удобството, но и проверимостта на коректността.

Силната страна на този подход е неговата честност. Той не обещава повече абстракция, отколкото реално предоставя. Именно затова той често служи като базова линия, спрямо която могат да бъдат оценявани по-високите слоеве на абстракция.

2.4. Асинхронност в съществуващите клиентски стекове

Съществена част от съвременния контекст е, че платформено-специфичните клиентски библиотеки невинаги са само синхронни. `libpq` предоставя автомат с крайни състояния за асинхронно изпращане и получаване на заявки [7], MySQL Конектор/C предлага двойки от тип `_start/_cont` за неблокиращи операции [8], `hiredis` има слой за функции за обратно извикване [9], а драйверът за Cassandra използва механизъм с бъдеща стойност и функция за обратно извикване [10]. Тези възможности обаче са силно специфични за съответния компонент както по интерфейс, така и по семантика на изпълнение.

Оттук следва важен извод: наличието на платформено-специфичен асинхронен програмен интерфейс само по себе си не решава проблема за обща C++ абстракция. Във всеки отделен случай приложението трябва да знае дали работи с цикъл за готовност на мрежово гнездо (готовност на мрежово гнездо), с регистрация на функция за обратно извикване, с периодично допитване до бъдеща стойност (периодично допитване до бъдеща стойност) или с вътрешно управлявани драйверни нишки. При MongoDB акцентът остава върху поддържането на пул и безопасна многонишкова употреба на `libmongoc` [11], а при `libneo4j-client` преобладава синхронният модел заявка/отговор [12]. Именно тази хетерогенност прави необходим междинен асинхронен слой, който да бъде достатъчно честен към различията в свързващите компоненти и едновременно достатъчно единен за потребителския програмен интерфейс.

2.5. Конструктори на заявки по време на изпълнение

Библиотеки като SOCI [13] и част от по-лекия инструментариум за изграждане на SQL търсят баланс между четимост и контрол. Те предлагат по-удобен C++ интерфейс за изграждане на заявки, но в много случаи продължават да разчитат на описване по

време на изпълнение и последващо рендиране към SQL. В практиката това означава, че известна част от структурата на заявката е по-добре капсулирана, но смяната на модел за съхранение, независима от свързващия компонент, остава ограничена.

За настоящата разработка тези библиотеки са важни като отправна точка. Те показват, че изразителните програмни интерфейси (изразителен API) и типизирани елементи са полезни, но не са достатъчни сами по себе си. Същинският въпрос е дали компилаторът може да валидира цялата логическа структура на заявката и дали същата абстракция може да бъде пренесена извън SQL света.

2.6. Типизирани SQL решения

sqlpp11 [14] е особено интересен случай, защото стои между традиционния конструктор на заявки по време на изпълнение и по-строго статичния дизайн. Подобни библиотеки използват шаблони и типове, за да направят част от SQL структурата видима за компилатора. Това е съществена стъпка напред спрямо чистите конструктори по време на изпълнение, защото част от структурните грешки могат да бъдат хванати по-рано.

Същевременно обаче типизираното SQL решение обикновено остава дълбоко свързано с релационния модел. Дори когато е много силно като C++ програмен интерфейс, то най-често предполага свят, в който таблици, колони, съединявания (JOIN) и агрегати остават централната метафора. Това прави такива решения изключително ценни за SQL домейна, но по-трудно преносими към документни, графови или ключ-стойност системи.

Именно тук се очертава една от основните разлики с настоящата разработка. Тя не цели просто структуриране на SQL по време на компилация. Тя цели логически слой по време на компилация, който може да бъде материализиран различно според семейството на свързващия компонент.

2.7. Класически рамки за обектно-релационно съпоставяне и кодогенерация

ODB [15] е характерен пример за рамка за обектно-релационно съпоставяне, която предлага богато картографиране между класове и релационни таблици, но използва генерация на код и външен инструментариум. Това улеснява потребителя на ниво употреба, но измества сложността към процеса на изграждане (процеса на изграждане) и към

автоматично генерираните артефакти. Получава се силна зависимост между модела на данните, допълнителната стъпка в инструментариума и конкретните релационни компоненти.

В класическите ОРС рамки често се наблюдава и друго напрежение: колкото по-високо е удобството на ниво приложен код, толкова по-непрозрачен става пътят до реалната заявка и до експлоатационните характеристики на системата. Това не е задължително фатален проблем, но в академичния контекст на настоящата работа е важно ограничение, защото затруднява проследимостта между абстракцията и материализацията.

2.8. Граници на релационно центрираните абстракции

За приложения, които трябва да работят с документни, графови или ключ-стойност системи, класическият ОРС модел показва естествени ограничения. Част от концепциите — като JOIN, външни ключове и строго таблична схема — нямат директен еквивалент във всички модели за съхранение. Поради това обобщението на обектно-релационното съпоставяне отвъд релационния свят не може да бъде постигнато само чрез генериране на SQL.

Това наблюдение е важно и за специфичния програмен интерфейс, и за типизираните SQL решения. Дори когато те са много силни технически, обикновено предполагат, че SQL е централният език на света на данните. Настоящата разработка приема по-различна отправна точка: логическият модел трябва да бъде първичен, а релационната материализация — само една от възможните интерпретации.

Таблица 2. Ограничения на различните семейства решения спрямо целта за множество свързващи компоненти

Семейство решения	Какво прави добре	Къде се появява ограничението
Специфичен интерфейс	максимален контрол и близост до драйвера	липса на общ логически модел и голямо дублиране при повече от един свързващ компонент
Конструктори по време на изпълнение	по-добра четимост и капсулиране на заявката	валидацията остава предимно по време на изпълнение и често е центрирана около SQL
Типизирани SQL решения	по-силна статична структура за SQL заявки	логиката обикновено остава ограничена в рамките на релационната метафора
Класическо OPC	удобно съпоставяне на обекти за релационни системи	зависимост от кодогенерация и трудна преносимост към нерелационни модели

2.9. Позициониране на настоящата разработка

Предложената библиотека се позиционира като решение за обектно-релационно съпоставяне по време на компилация, което комбинира силна статична типизация, липса на външна кодогенерация, модел с конектори за пренасяне на логическата абстракция към различни целеви системи и базиран на корутини асинхронен слой за изпълнение. В сравнение с подхода със специфични програмни интерфейси тя намалява дублирането на логиката и риска от късно открити грешки. В сравнение с решенията с конструктори на заявки по време на изпълнение премества структурната валидация към компилатора. В сравнение с класическите OPC рамки тя избягва външни генератори и поставя ясен акцент върху договора на конектора. В сравнение със специфичните за компонента асинхронни

интерфейси тя предлага единен модел с корутини, без да твърди, че всички драйвери имат еднакви неблокиращи възможности.

Ключовата разлика обаче е по-дълбока. Настоящата разработка не разглежда техниките по време на компилация като средство за “по-умен SQL конструктор”, а като начин за формализиране на логическия модел на достъпа до данни. Аналогично, тя не разглежда асинхронността като декоративна обвивка, а като отделен архитектурен проблем със собствени ограничения, правила за жизнения цикъл и специфични за компонента пътища на изпълнение. Точно това позволява следващите глави да преминат от теорията за корутините към архитектурата по време на компилация, асинхронния слой, договора на конектора и реалните сценарии с множество целеви системи.

2.10. Изводи от сравнителния анализ

Сравнението със съществуващите решения води до четири основни извода. Първо, подходът с платформено-специфични интерфейси остава незаменим като базова линия за контрол и производителност, но не предлага удовлетворително ниво на логическа абстракция. Второ, конструкторите на заявки по време на изпълнение и типизираните SQL решения подобряват работата със Structured Заявка Language (SQL), но не решават напълно проблема за логически слой, независим от компонента. Трето, класическите OPC рамки предлагат висока абстракция, но често за сметка на сложност на инструментариума и ограничена пригодност извън релационния свят. Четвърто, платформено-специфичните асинхронни клиентски интерфейси са ценни, но показват толкова различни модели на изпълнение, че сами по себе си не осигуряват единна потребителска абстракция.

Точно това позициониране оправдава по-детайлното разглеждане в следващата глава на отделен, но ключов въпрос: кои езикови и теоретични механизми правят възможна базираната на корутини асинхронна архитектура, върху която по-късно стъпват OPC моделът по време на компилация и изпълнителният слой за множество свързващи компоненти.

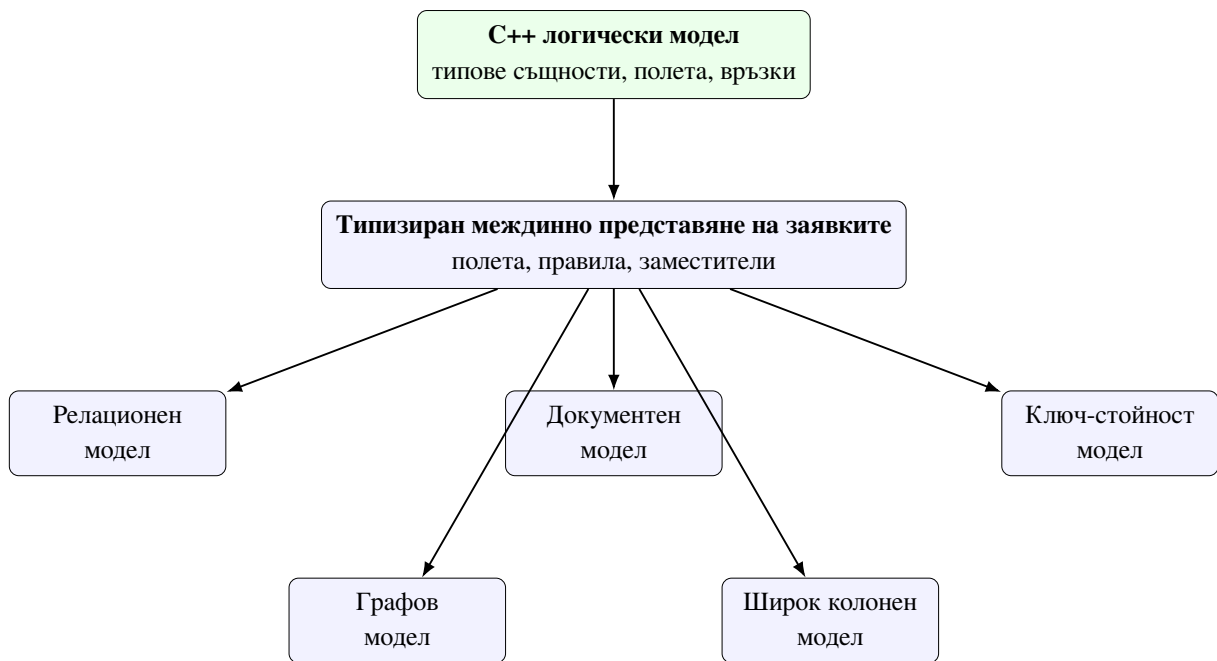
III. ТЕОРЕТИЧНИ ОСНОВИ НА БИБЛИОТЕКАТА И МОДЕЛИТЕ ЗА СЪХРАНЕНИЕ

Теоретичната основа на настоящата разработка не се свежда до една езикова възможност или до един тип база данни. Библиотеката комбинира идеи от теорията за обектно-релационното съпоставяне (ОРС), статичното типизиране в C++, изграждането на междинни представяния по време на компилация и сравнителния анализ на различни модели за съхранение. За да бъде разбираема по-късната архитектура, е необходимо първо да се отдели логическият слой на описване на данните от физическия слой на тяхното специфично материализиране за дадения свързващ компонент.

В контекста на тази дипломна работа терминът обектно-релационно съпоставяне по време на компилация не трябва да се разбира стеснено само като SQL техника. По-точно той обозначава архитектурен подход, при който моделът на данните, формата на заявките и част от ограниченията върху операциите се описват чрез типовата система на езика, а не чрез конфигурация по време на изпълнение. Това позволява една и съща логическа структура да бъде пренесена към релационни, документни, ключ-стойност, графови и широки колонни системи, без специфичните за свързващия компонент различия да се скриват или подменят.

3.1. Таксономия на проблемната област

Преди разглеждане на отделните компоненти е полезно да се фиксира общата картина на проблемната област. От една страна стои C++ моделът, в който приложението описва типове същности, полета, връзки и ограничения. От друга страна стоят различни семейства системи за съхранение, всяко със собствен модел на физическа организация, достъп и оптимизация. Между тях е разположен типизираният слой за междинно представяне на заявките, който служи като посредник между логиката на приложението и конкретния слой на конектора.



Фигура 2. Таксономия на логическия модел, междинното представяне на заявките и семействата свързващи компоненти

Фигура 2 подчертава централното теоретично допускане на библиотеката: общото между свързващите компоненти не е техният синтаксис, а логическият модел на данните и операциите. Следователно валидната абстракция трябва да бъде разположена по-високо от SQL, BSON, Redis команди, Cypher или CQL. Тя трябва да работи на равнището на полета, предикати, проекции, връзки и допустими операции.

3.2. Обектно-релационното съпоставяне като архитектурен подход

Класическото OPC свързва обектния модел на приложението с неговата персистентна схема [2]. Във варианта по време на компилация това свързване се конструира не чрез отражение (отражение) и конфигурации по време на изпълнение, а чрез типове, шаблони и constexpr стойности. Практически това означава, че изборът на полета, допустимите операции и част от съвместимостта със свързващия компонент се проверяват от компилатора. Грешки като обръщение към несъществуващо поле или използване на съединяване (JOIN) върху конектор без supports_joins престават да бъдат изненади по време на изпълнение.

Съществената особеност е, че OPC по време на компилация не означава липса на данни по време на изпълнение. Стойностите на параметрите се подават в момента на изпълнение, но структурата на заявката е вече фиксирана и валидирана. Така се получава

ясно разграничение между *логическа форма* и *стойности по време на изпълнение*. Това разграничение е ключово за тезата, защото позволява едновременно статични гаранции и реална работа с външни бази данни.

От теоретична гледна точка този подход доближава ОРС слоя до компилаторен конвейер. Приложният код описва семантична структура, компонентите по време на компилация я превръщат в междинно представяне, а едва най-долният слой материализира тази структура в специфичен език или протокол на свързващия компонент. Затова по-късните архитектурни решения в библиотеката могат да се анализират през понятия като междинно представяне, статична проверка и материализация в целевата система.

3.3. Статичен модел на данните в C++

В разглежданата библиотека домейн моделът се описва чрез C++ агрегати, чиито полета са от тип `property<T,Name>`. По този начин всяко поле носи едновременно стойностен тип и символно име на колоната или атрибута. Допълнително `table_name_trait<T>` позволява типът да бъде свързан с логически контейнер за съхранение — таблица, колекция, префикс на ключ, `label` или друго специфично за компонента понятие.

Тази организация има важно теоретично следствие: C++ типът става първичен носител на логическата схема. Отделните свързващи компоненти не налагат нов потребителски модел, а само интерпретират един и същ модел по различен начин. Именно затова по-късно може да се говори за еднакъв логически обект, който в релационен компонент се превръща в таблица, а в документен — в документ с вложени или референтни полета.

Таблица 3. Съпоставка между C++ конструкциите и теоретичните OPC понятия

Конструкция	Теоретична роля	Практически ефект в библиотеката
<code>property<T,Name></code>	атрибут от логическата схема	носи тип на стойността и символно име за материализация
<code>relationship<...></code>	логическа връзка между обекти	отделя връзката от физическия механизъм за съхранение
<code>table_name_trait<T></code>	логически контейнер	служи като име на таблица, колекция, префикс на ключ или label
<code>field<&T::m></code>	адресиране на атрибут в пространството на заявката	свързва слоя на същностите с междинното представяне на заявките
<code>connector_trait<DB></code>	материализация в целевата система	определя как логическият модел се изпълнява физически

Следователно OPC слойът не започва от драйвер или от рендиране на SQL. Той започва от статично описание на смисъла на данните. Това е и причината слойът на същностите да бъде фундаментален за цялата архитектура: ако този слой е неясен или непълен, няма как по-долу да се получи коректна и надграждаема материализация.

```

1  template <typename T, detail::string_literal Name>
2  struct property
3  {
4      using value_type = T;
5
6      [[nodiscard]] static constexpr std::string_view column_name() noexcept
7      {
8          return Name.view();
9      }
10
11     T value{};
12 };
13
14 enum class store_as { reference, embed };
15
16 template <store_as Strategy, typename T, detail::string_literal Name>
17 struct relationship
18 {
19     static constexpr store_as strategy = Strategy;
20     using related_type = T;

```



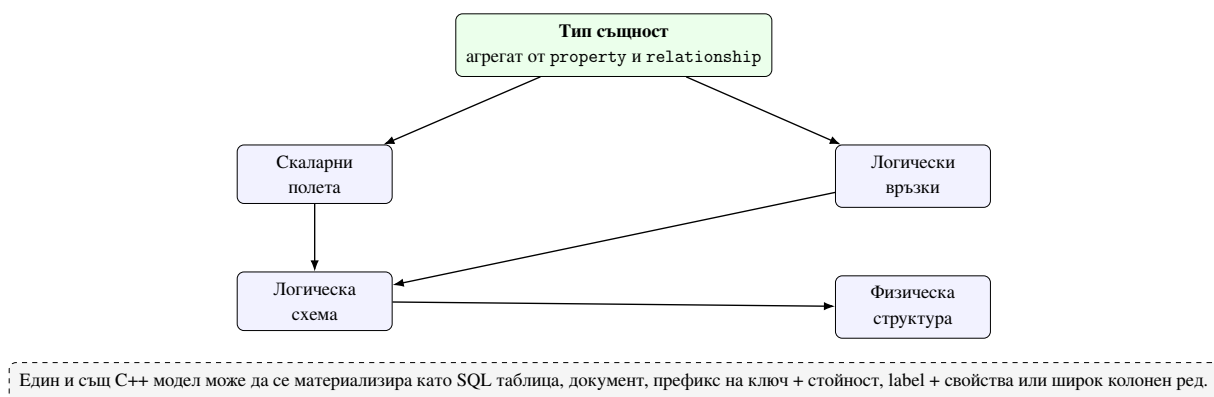
```

21
22     [[nodiscard]] static constexpr std::string_view field_name() noexcept
23     {
24         return Name.view();
25     }
26 };

```

Listing 1: Извадка от property и relationship в слоя на същностите

Listing 1 показва, че слойът на същностите е минималистичен по форма, но достатъчно богат по смисъл. property не е просто контейнер за стойност; той носи и име, а relationship не е просто член-данна; тя фиксира стратегията за материализация като част от типа.



Фигура 3. Премаване от C++ описание на същност към логическа схема и физическа материализация

3.4. Типизирано междинно представяне на заявките

Вместо заявката да се описва директно като SQL или друго текстово представяне, библиотеката използва типизирано междинно представяне на заявките. Неговите елементи са field, Rule, placeholder и обекти на заявките като select_query, insert_query, update_query и delete_query. Този модел дава две предимства. Първо, структурата на заявката може да бъде анализирана по време на компилация и да участва в static_assert проверки. Второ, едно и също междинно представяне може да бъде рендирано към различни езици и протоколи на свързващите компоненти.

Теоретично междинното представяне играе ролята на междинен език между домейн модела и конкретния слой за изпълнение. Това е аналогично на междинните представяния в компилаторите: потребителят работи с високо ниво на абстракция, а свързващият компонент получава специализирана форма, оптимизирана за собствения си модел.

Именно поради това междинното представяне е логически по-близо до семантиката на операциите, отколкото до синтаксиса на конкретна заявка.

```
1 template <typename Response, typename Joins, typename Wheres,  
2         typename Limits, typename Groups, typename Orders>  
3 struct select_заявка : select_заявка_tag  
4 {  
5     using response_type = Response;  
6     using row_type = projected_type<Response>;  
7  
8     template <typename RuleType>  
9         requires is_rule<RuleType>  
10    [[nodiscard]] constexpr auto where(RuleType r) const  
11    {  
12        using NewWheres = detail::append_type_t<Wheres, RuleType>;  
13        return select_заявка<Response, Joins, NewWheres, Limits, Groups, Orders>(  
14            response_, joins_,  
15            detail::копнеж_cat(wheres_, detail::orm_копнеж<RuleType>(r)),  
16            limits_, groups_, orders_);  
17    }  
18 };
```

Listing 2: Извадка от `select_query` и композицията на `where` клаузи

Listing 2 илюстрира важно теоретично свойство: конструкторът на заявки не мутира текст, а изгражда нови типизирани обекти. Това означава, че последователността от операции `select(...).where(...).group_by(...)` може да бъде разглеждана като конструиране на нов тип, а не като поредица от странични ефекти върху низ.

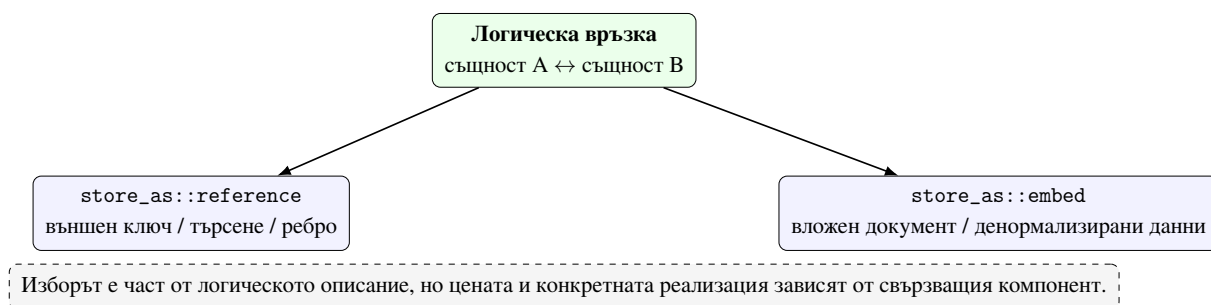


Фигура 4. Конвейер на типизираното междинно представяне от приложния интерфейс до изпълнението в целевата система

3.5. Теория на връзките и стратегии за съхранение

Връзките между обекти са един от най-трудните аспекти при обобщаване на обектно-релационното съпоставяне извън релационния свят. В релационен контекст една връзка често се материализира чрез външен ключ и съединяване (JOIN). В документен контекст обаче същата логическа връзка може да се реализира като вграден документ, а в ключ-стойност контекст — чрез вторично търсене или денормализиране.

Библиотеката абстрахира тези варианти чрез `relationship<Strategy,T,Name>` и `store_as::reference/store_as::embed`. `reference` обозначава логическа външна препратка, която свързващият компонент може да реализира чрез външен ключ, търсене, отделен ключ или графова връзка. `embed` обозначава физическо вграждане на свързаните данни в същата физическа структура. По този начин връзката се описва веднъж логически, а интерпретацията остава задача на конектора.



Фигура 5. Двете основни стратегии за материализация на връзки

Таблица 4. Сравнение между стратегиите reference и embed

Стратегия	Логическа идея	Естествени целиви системи	Основен компромис
reference	отделна идентичност на свързания обект	SQL, графови системи, търсене по ключ	необходим е допълнителен механизъм за навигация или свързване
embed	физическо включване в родителската структура	документни системи, денормализирани записи	по-трудно независимо обновяване и споделяне на данни

3.6. Релационен модел

Релационният модел организира данните в таблици, редове и колони. Основните му предимства са ясна схема, декларативен език за заявки и добре дефинирани ограничения върху целостта. За системите за обектно-релационно съпоставяне това е естествената среда: свойствата на C++ обекта лесно се съпоставят с колони, а връзките — с външни ключове. Операции като съединяване (JOIN), групиране (GROUPBY) и транзакции са част от нормалния езиков модел.

Именно релационният свързващ компонент е естественият базов сценарий, спрямо който може да се оцени до каква степен една библиотека за съпоставяне запазва удобството си и извън SQL света. В настоящата разработка този базов сценарий по-късно се реализира основно чрез SQLiteDB, а преносимостта на междинното представяне се анализира допълнително чрез PostgreSQLDB и MySQLDB.

3.7. Документен модел

Документният модел организира данните в колекции от документи, чиито полета могат да бъдат вложени и не е задължително всички документи да имат еднаква физическа форма. Това го прави естествено съвместим със стратегията за вграждане (embed) и по-гъвкав при денормализирани структури. От друга страна, твърде директното пренасяне на релационни абстракции върху документен свързващ компонент може да бъде подвеждащо. Не всяка релационна операция има еквивалент със същата цена и семантика.

В контекста на настоящата работа документният модел е важен като тест за това дали междинното представяне описва логика, а не синтаксис на конкретен език. Ако едно и също междинно представяне може да се интерпретира като BSON-подобен филтър и проекция, без потребителят да пише платформено-специфични документни заявки, тогава действително е постигнат слой на абстракция, независим от съхранението.

3.8. Ключ-стойност модел

Ключ-стойност системите организират достъпа около ключ и стойност. Те са особено ефективни, когато моделът на достъп е ясен и концентриран около търсене по идентификатор. Недостатъкът е, че общите заявки, базирани на предикати, и релационните операции са силно ограничени. Затова една абстракция за съпоставяне върху ключ-стойност компонент не може да обещава същата изразителност като върху SQL система.

Този модел е полезен за разграничаване между *логически обект* и *допустим модел на достъп*. Обектът може да бъде един и същ, но не всяка заявка е смислена върху всяка физическа организация. В хранилището това по-късно се вижда ясно при RedisDB, където логиката за търсене по ключ е естествена, а стилът, базиран на съединявания, е архитектурно неуместен.

3.9. Графов модел

Графовият модел представя данните като възли, ребра и свойства. Силата му е в изразяването на връзки като първокласни обекти, а не като производни от външни ключове. Това го прави подходящ за сценарии със сложни навигации и операции за обхождане. В същото време графовият модел налага собствен език на заявки и собствена интуиция за

селекция и филтриране.

За библиотеката това е показателен сценарий, защото демонстрира, че `connector_trait` може да пренесе общо междинно представяне към MATCH/RETURN/WHERE структура, без потребителят да пише Cypher директно. Ако това е възможно, значи общият слой действително е разположен над конкретния синтактичен слой.

3.10. Широк колонен модел

Широките колонни системи, каквато е Cassandra, се основават на разпределени таблици с ключове за разделяне (разделяне) и групиране (групиране). Макар синтактично да напомнят релационни таблици, те имат различна теория на достъпа: правилната заявка е тази, която следва физическия модел на ключовото разпределение. Следователно наличието на поле в C++ модела не означава автоматично, че може да бъде използвано свободно във WHERE клаузи.

Точно този тип ограничения оправдава подхода с възможности и ограничения на библиотеката. Не е достатъчно междинното представяне да е типowo коректно; то трябва да е и допустимо спрямо модела на съхранение. По тази причина широкият колонен модел е особено ценен за теоретичната защита на ограниченията по време на компилация.

Таблица 5. Съпоставка между основните модели за съхранение

Модел	Единица за съхранение	Типични връзки	Основно ограничение
Релационен	таблица/ред	външен ключ, съединяване	необходимост от съгласувана схема и силна типова дисциплина
Документен	документ	вграждане или препратка	не всяка релационна операция е естествена или евтина
Ключ-стойност	ключ и стойност	търсене по ключ, хеш полета	силно ограничени заявки с предикати извън ключовия достъп
Графов	възел и ребро	връзки за обхождане	специфичен език за заявки и графова семантика
Широк колонен	разделен (разделяне) ред	ключово-ориентирано разделяне зависимости	заявките трябва да следват логиката на разделянето

3.11. Тестова опора на теоретичните твърдения

Теоретичните твърдения в тази глава не са откъснати от кода. Слой на същностите се валидира пряко в `tests/РѢР«РҮСҢРѢР,,Рҫ/test_property.cpp` и `tests/РѢР«РҮСҢРѢР,,Рҫ/test_relationship.cpp`, където се проверяват имената на колоните, разпознаването на типовете същности, различаването между `store_as::reference` и `store_as::embed`, както и логиката за извеждане (извеждане) на отношения едно-към-много. Типизираното междинно представяне е покрито в `tests/РѢР«РҮСҢРѢР,,Рҫ/test_РҫРѢСРҮРѢРѢ.cpp`, където се демонстрират предикати, базирани на полета, форми на съединяване, композиция на обновявания и семантика на изтриване. Допълнително `tests/РѢР«РҮСҢРѢР,,Рҫ/test_cpp26_Р«СҢСҢРѢРҫРҫР,,РҫРҫ.cpp` показва, че описанието на полетата по време на компилация може да бъде надградено и чрез подход с отражение, когато инструментариумът го позволява.

Тази връзка между теория и тестова опора е съществена. Тя показва, че представените понятия не са свободни академични метафори, а реални архитектурни единици, които могат да бъдат проверявани в изходния код и в автоматизирани тестове.

3.12. Граници на унифицираната абстракция

След изложеното може да се направи важен извод. Унифицираната абстракция не означава, че всички свързващи компоненти стават еднакви. Унифицирането е възможно на равнище логически модел, поле, връзка, междинно представяне и базови операции. Отвъд тази граница започват различията: SQL JOIN няма универсален смисъл в Redis; логиката за вграждане няма идентична цена в класическа SQL таблица; обхождането на граф не се свежда до обикновен SELECT; ограниченията, продиктувани от разделянето в Cassandra, не могат да бъдат маскирани като обикновени предикатни правила.

Поради това архитектурно правилният подход не е да се скрият всички различия, а да се направят явни и формално контролирани. Именно тази роля изпълняват `connect` or `trait`, `label`ите за възможности и ограниченията по време на компилация, разгледани в следващите глави. Точно там ще стане видимо как теоретичната рамка от настоящата глава се превръща в работеща архитектура на библиотеката.

IV. ТЕОРИЯ НА КОРУТИНИТЕ И АСИНХРОННОТО ИЗПЪЛНЕНИЕ

В C++

Корутините са езиков механизъм за описване на прекъсваеми изчисления, при които изпълнението може да бъде спряно и по-късно възобновено без загуба на локалното състояние. В стандарта C++20 корутините влизат в основния език, а не се реализират като външна библиотечна симулация [16, 17, 18]. Това е важно за настоящата работа, защото позволява асинхронният слой на библиотеката да се изгражда върху дефинирани по време на компилация правила за типове, жизнен цикъл и поток на управление (поток на управление), вместо върху неструктурирана архитектура от функции за обратно извикване (функция за обратно извикване).

В тази глава корутините се разглеждат не като удобен синтаксис, а като изпълнителен модел, който стои между чисто синхронното блокиращо изпълнение и тежките нишки на операционната система. Целта не е да се покаже само как се използват `co_await` и `co_return`, а да се изведе защо преобразуването от компилатора към автомат с крайни състояния прави този подход особено подходящ за библиотека, която и без това се основава на статични типове, специализации на характеристики и валидиране по време на компилация.

Този теоретичен пласт е необходим и поради още една причина. В контекста на библиотеката за обектно-релационно съпоставяне по време на компилация корутините не са изолиран езиков експеримент, а основа за единен асинхронен интерфейс над свързващи компоненти с много различни експлоатационни характеристики. За да бъде такава архитектура научно защитима, трябва ясно да се разграничат езиковият механизъм, инфраструктурата по време на изпълнение и конкретната семантика на изпълнение на драйверите.

4.1. Процеси, нишки, задачи и корутини

Процесът е защитен адресен контекст на операционната система, нишката е планирана от ядрото последователност от инструкции, а задачата е по-общо програмно понятие за единица работа. Корутината не е нито процес, нито нишка. Тя е кооперативна единица за изпълнение, чието спиране и подновяване се управлява от програмния модел и генерираната структура по време на изпълнение, а не директно от планировчика на

операционната система.

Това разграничение е съществено, защото библиотеката комбинира корутини и набор от нишки (`thread_pool`), без да приравнява двете абстракции. Нишката е скъпа ресурсна единица на ядрото, свързана с отделен стек, планиране и примитиви за синхронизация. Корутината е значително по-лека логическа единица, която пази само състоянието, необходимо за спиране и продължаване на конкретно изчисление. Следователно преминаването към модел, базиран на корутини, не означава „повече нишки“, а по-структуриран начин за описване на това къде едно изчисление може да изчака външно събитие.

Таблица 6. Съпоставка между основните изпълнителни единици

Единица	Същност	Значение за библиотеката
Процес	Изолиран адресен контекст с ресурси на операционната система	Не е пряка единица на архитектурата, но определя границата на средата за изпълнение
Нишка	Планиран от ядрото поток на изпълнение със собствен стек	Използва се от <code>thread_pool</code> за изнасяне (изнасяне) на блокиращи операции
Корутина	Кооперативно прекъсваемо изчисление със запазено локално състояние	Осигурява формалния модел за <code>Task<T></code> и за асинхронната композиция

Таблица 6 подчертава, че корутината е логическа, а не единица на ядрото. Именно това позволява в една и съща нишка да се редуват множество спиране/възобновяване последователности, без всяка от тях да изисква отделен стек и отделна нишкова идентичност.

4.2. Синтаксис, протокол за изчакване и езикови свързване `await`-и

Ключовите езикови оператори `co_await`, `co_return` и `co_yield` маркират, че дадена функция следва семантиката на корутините. От гледна точка на архитектурата най-важен е `co_await`, защото именно той описва точката, в която текущото изчисление може да отстъпи изпълнението, докато външна операция — например база данни или планиране към набор от нишки — не стане готова. Така синтаксисът на потребителя остава близък до линейния код, докато изпълнителният модел става асинхронен.

За текущата дипломна работа е важно, че тази езикова форма не е динамично „откривана“ по време на изпълнение, а е компилаторно свързана с типа на връщаната стойност на функцията и с `promise_type`, който `std::coroutine_traits` извежда за конкретния подпис [16, 17]. Това означава, че начинът, по който една корутина създава резултат, предава грешка или съхранява продължение (продължение), може да бъде формализиран още на ниво типова система.

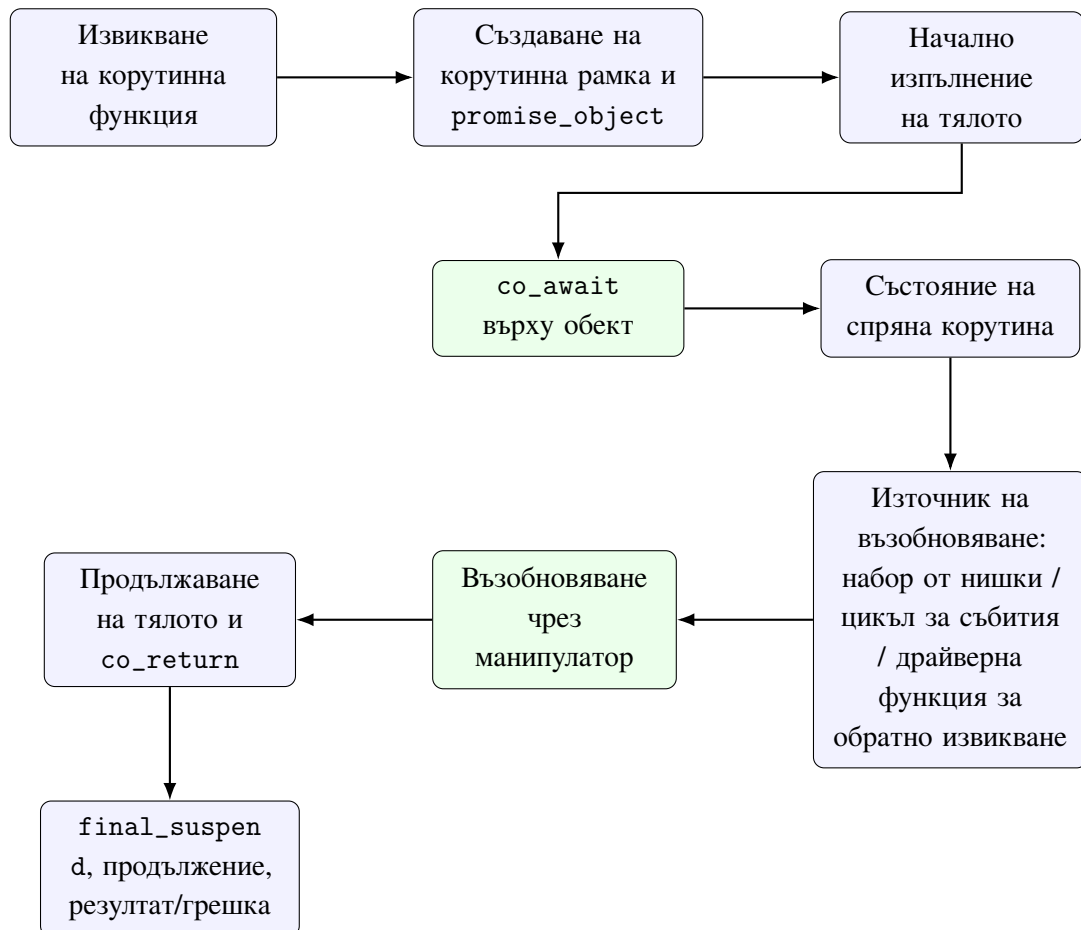
На практика `co_await` работи чрез протокол за изчакване (протокол за изчакване). Обектът, който се изчаква, участва в тристъпкова схема: `await_ready()` определя дали спиране изобщо е необходимо, `await_suspend()` получава манипулатор на корутината (манипулатор на корутината) и решава как да организира възобновяването, а `await_resume()` връща резултата или повдига грешка. Точно тази тройка прави възможно различни източници на асинхронност — задачи в набор от нишки, готовност на цикъл за събития, функции за обратно извикване от драйвер — да бъдат уеднаквени под общ `co_await` синтаксис.

4.3. Корутина като автомат с крайни състояния

Компилаторът не „изпълнява магия“, а преобразува корутинната функция в автомат с крайни състояния с ясно определени точки на спиране, преходи на възобновяване и съхранение на рамки [17, 18]. Това преобразуване е от особено значение за настоящата теза, защото показва, че подходът по време на компилация на библиотеката не се изчерпва само с моделиране на заявки. Същият езиков инструментариум позволява и асинхронният поток на управление да бъде формализиран чрез типове и статични правила, вместо да се описва чрез неструктурирани вериги от функции за обратно извикване.

Концептуално една корутина функция преминава през няколко ясно разграничени

фази: създаване на рамка и `promise_object`, начално изпълнение, достигане до точка на спиране, външно организирано възобновяване и финално завършване. Това е съществено, защото коректност вече не зависи само от „какво прави кодът“, а и от това кога е допустимо той да бъде продължен и кой носи отговорност за продължаването.



Фигура 6. Концептуален жизнен цикъл на корутината като рамка, точки на спиране и външно управлявано възобновяване

Фигура 6 показва защо корутинният модел е естествен за библиотека с ясно отделени слоеве. Логическият код на потребителя остава линеен, но реалното управление на изпълнението преминава през формални преходи, които могат да бъдат свързани с конкретни механизми по време на изпълнение.

4.4. Корутинна рамка, обект-обещание, продължение и безопасност

При всяка корутина компилаторът създава корутинна рамка, в която се съхраняват локалните променливи, текущото състояние на изпълнението и обектът-обещание (обект-обещание), чрез който корутината комуникира резултат, грешка или завършване.

Това има директно отражение върху безопасността: жизненият цикъл на рамката, собствеността върху резултата и мястото на продължението не са второстепенни детайли, а част от договора за коректност на асинхронния слой.

Трябва да се подчертае, че корутинната рамка не е обикновена стекова рамка. Тя може да преживее излизането от текущия стеков контекст и да бъде възобновена по-късно от друга нишка или от инфраструктура за цикъл за събития. Поради това проблеми като преждевременно разрушаване на рамката, двойно възобновяване, неправилно предаване на състояние за изключение или загуба на продължението не са „детайли на реализацията на ниско ниво“, а фундаментални рискове за коректността.

Именно обектът-обещание организира връщането на стойност, разпространението на грешка и поведението при `initial_suspend` и `final_suspend`. От гледна точка на архитектурата това е важен пример за свързване по време на компилация: видът на резултата и семантиката на завършването не се определят от динамична схема за регистрация, а са закодирани в типа на връщаната стойност на корутината. Това позволява `Task<T>` да бъде проектиран като формална абстракция, а не като неформален обект, подобен на бъдеще (бъдеще).

4.5. Корутини, набор от нишки и инфраструктура за цикъл за събития

Наборът от нишки (`thread_pool`) и корутините решават различни проблеми. Наборът от нишки предоставя ограничен брой работни нишки и политика за планиране на работа, докато корутините описват как една логическа задача може да бъде спряна и по-късно възобновена. Комбинацията между двете позволява блокиращи свързващи компоненти да бъдат интегрирани по универсален начин чрез изнасяне (изнасяне), без потребителят да се връща към програмиране, ориентирано към функции за обратно извикване.

От друга страна, при истински неблокиращи драйвери корутината може да бъде възобновявана не от работна нишка, а от цикъл за събития или мост за функции за обратно извикване. Средоподобни на реактори наблюдават готовността на файлови дескриптори чрез системни механизми като `kqueue` на Mac OS и BSD системите [19]. Следователно корутината сама по себе си не „изпълнява входно-изходни операции“, а предоставя формален интерфейс, през който готовността за вход-изход или функцията за обратно

извикване на драйвера може да се превърне в действие за възобновяване.

Точно тук става ясно защо корутините не трябва да бъдат смесвани с нишките. Нишката е носител на физическо изпълнение; корутината е носител на логическа последователност. Един и същ асинхронен модел може да използва набор от нишки в един сценарий на свързващ компонент и цикъл за събития в друг, без да променя потребителския синтаксис на `co_await`.

4.6. Значение за библиотеката за обектно-релационно съпоставяне по време на компилация

За разработваната библиотека този теоретичен анализ има пряко архитектурно следствие. Моделираното по време на компилация междинно представяне на заявките и договорът на конектора определят какво е позволено да се изпълни; базираният на корутини слой определя как това позволено изчисление може да бъде изпълнено асинхронно. С други думи, типово коректната заявка и коректният път на изпълнение са различни, но взаимно зависими пластове.

Това разграничение е особено важно при библиотека с множество свързващи компоненти. PostgreSQL и MySQL предоставят платформено-специфични неблокиращи клиентски интерфейси, но с различни експлоатационни форми [7, 8]. hiredis използва асинхронен слой, базиран на функции за обратно извикване [9], а драйверът за Cassandra работи чрез механизъм с бъдеща стойност и функция за обратно извикване [10]. При MongoDB фокусът остава върху безопасен пул и многонишкова употреба на `libmongoc` [11], докато `libneo4j-client` следва преобладаващо синхронен модел заявка/отговор [12]. Именно тази нееднородност оправдава асинхронна архитектура с две стратегии: универсален модел за изнасяне (изнасяне) и платформено-специфична асинхронна интеграция за драйвери, които я позволяват.

Следователно корутините не правят автоматично всяка операция неблокираща и не отменят ограниченията на даден драйвер. Те дават формален модел за асинхронно описание на изчислението; реалният ефект върху производителността зависи от това дали клиентската библиотека има платформено-специфичен неблокиращ интерфейс, дали работата се изнася към набор от нишки или дали задачата остава синхронна. Именно затова следващите глави разглеждат отделно OPC архитектурата по време на компилация, асинхронния слой на рамката и семантиката на изпълнение на конкретните свързващи

КОМПОНЕНТИ.

V. ПО ВРЕМЕ НА КОМПИЛАЦИЯ ORM АРХИТЕКТУРА

Архитектурата за обектно-реляционно съпоставяне по време на компиляция на библиотеката е организирана около ясно разграничение между логическия модел на данните, междинното представяне на заявките и специфичния за свързващия компонент слой за материализация. Именно този пласт носи първата основна научна ос на разработката: как компилаторът може да участва в моделирането, ограничаването и валидирането на достъпа до данни още преди фазата на изпълнение. За да бъде постигнато това, архитектурата е изградена от няколко слоя, всеки със собствена отговорност и със строго определени интерфейси.

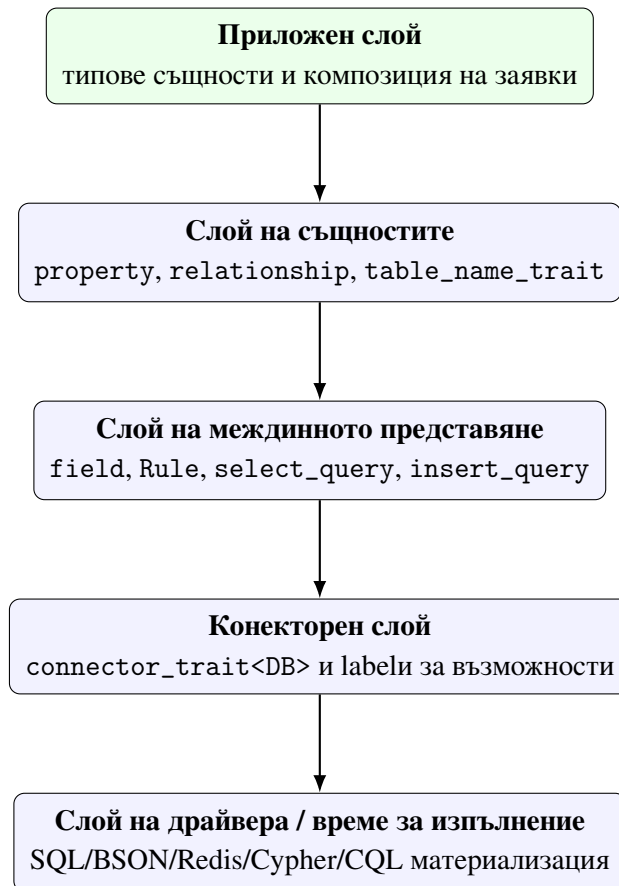
В теоретичната глава беше показано, че унифицирането е възможно на равнището на логическия модел и междинното представяне на заявките, но не и чрез игнориране на различията между свързващите компоненти. След теорията за корутините от предходната глава тук фокусът съзнателно се връща към ядрото по време на компиляция на системата: описанията на същностите, междинното представяне на заявките, `connector_trait<DB>` и механизма за възможности (възможност). По този начин библиотеката се разглежда не просто като идея, а като организирана система от типове, договори и формални граници на допустимите операции.

5.1. Архитектурни цели и ограничения

Архитектурата е проектирана при едновременно действие на няколко ограничения. Първо, потребителският програмен интерфейс трябва да остане в рамките на идиоматичен C++, без въвеждане на отделен специфичен за домейна език (DSL) и без механизъм за отражение по време на изпълнение. Второ, структурата на заявката трябва да бъде представима като `constexpr` обект, така че значима част от валидирането да се извършва от компилатора. Трето, слойът на свързващия компонент трябва да бъде разширяем без наследяване от общ виртуален интерфейс, тъй като подобен интерфейс би скриел различията между компонентите и би изместил проверките към времето на изпълнение.

Четвъртото ограничение е свързано със самия характер на системата, поддържаща множество свързващи компоненти. Библиотеката трябва да може да поддържа SQL и не-SQL конектори без фалшиво уеднаквяване. Това означава, че архитектурата трябва едновременно да допуска общ програмен интерфейс и да изразява ограниченията на

конкретен свързващ компонент по формален и проверим начин. Оттук естествено следва изборът на конекторен слой, базиран на характеристики, и механизъм за възможности.



Фигура 7. Слоеста архитектура на библиотеката

Фигура 7 показва, че библиотеката следва последователна вертикална организация. Приложният код не комуникира директно с драйвера; той работи с C++ типове и конструктори на заявки. Драйверът от своя страна не познава семантиката на домейна, а само материализираната форма, генерирана от конекторния слой. Именно поради това архитектурата може да остане едновременно строга и надграждаема.

5.2. Слоеста организация и разделяне на отговорностите

Най-горният слой е приложният код, който дефинира типове същности, изгражда заявки чрез select, insert, update и deleteq, и ги подава на db<DB>. Следващият слой е логическият модел на библиотеката — описанията на същностите и междинното представяне на заявките. Третият слой е абстракцията на конектора, където connector_trait<DB> материализира същата логика към специфичен за целевата система синтаксис и

поведение. Най-долу стои конкретният драйвер, мрежов протокол (мрежов протокол) или имитираща реализация.

Тази архитектура има важен практически ефект. Ако потребителят промени SQLite с MongoDB, междинното представяне на заявките остава логически същото, но конекторът го тълкува като изпълнение на филтър/проекция вместо рендиране на SQL. Това не означава, че всяко междинно представяне е допустимо върху всеки свързващ компонент; означава, че *формата на абстракцията* е обща, а допустимостта се определя формално чрез механизма за възможности.

Таблица 7. Основни архитектурни слоеве и техните отговорности

Слой	Основни артефакти	Отговорност
Приложен	типове същности, извиквания на конструктора на заявки	описва логиката на домейна и формулира операции в идиоматичен C++
Логически	property, relationship, field, Rule	моделира данните и заявките независимо от конкретен свързващ компонент
Конекторен	connector_trait<DB>, labels за възможности	проверява допустимостта и материализира междинното представяне към поведение на целевата система
Време за изпълнение / драйвер	конкретни манипулатори на връзки и драйверни програмни интерфейси	изпълнява материализираната заявка и връща резултати

Таблица 7 показва, че библиотеката не използва “голям обект”, който да прави всичко. Вместо това различните решения са локализирани в отделни слоеве. Това е важно както за надграждаемостта, така и за академичния анализ, защото позволява отделните твърдения да бъдат проследени до конкретни типове и модули.

5.3. Слой на същностите като архитектурна основа

Слой на същностите е изграден върху `property<T, Name>`, `relationship<...>` и `table_name_trait<T>`. Той има двойна роля. От една страна, представя C++ домейн

модела. От друга страна, служи като логическо описание на схемата или структурата на персистентните данни. Това означава, че архитектурата не прави отделен регистър по време на изпълнение от метаданни. Вместо това типовата система съдържа достатъчно информация, за да бъдат извлечени имена на полета, типове, връзки и целеви контейнер за съхранение.

Особено важен е фактът, че слойът на същностите е независим от вида база данни. Нито `property`, нито `relationship` са SQL-специфични. Те описват логически атрибути и отношения. Така се създава основата за сравнимо поведение на релационни и нерелационни свързващи компоненти. На архитектурно ниво именно това прави възможно библиотеката да твърди, че C++ моделът е първичният носител на логическата схема.

5.4. Междинното представяне на заявките като централен архитектурен компонент

Междинното представяне на заявките (междинно представяне на заявките) е централният архитектурен компонент. Той обединява референции към полета, предикати, заместители и допълнителни клаузи като `order_by`, `group_by` и `limit`. Вместо да се работи с текст на заявка, библиотеката изгражда типизиран обект, чийто шаблонни параметри кодират логическата структура на операцията.

От архитектурна гледна точка това решение има няколко последствия. Първо, валидирането на структурата се извършва рано. Второ, междинното представяне може да бъде анализирано от конекторния слой без загуба на семантична информация. Трето, подготвените заявки могат да запазят не SQL низ, а самото междинно представяне, което е по-естествено за библиотека, ориентирана към C++ типовата система.

Четвърто, междинното представяне на заявките е единствената зона, в която се срещат описанието на същностите и по-късната материализация в целевата система. В този смисъл то е архитектурният “шарнир” на цялата библиотека: достатъчно високо, за да остане независимо от съхранението, и достатъчно конкретно, за да може да бъде изпълнено.

5.5. Конекторен слой и формалният договор

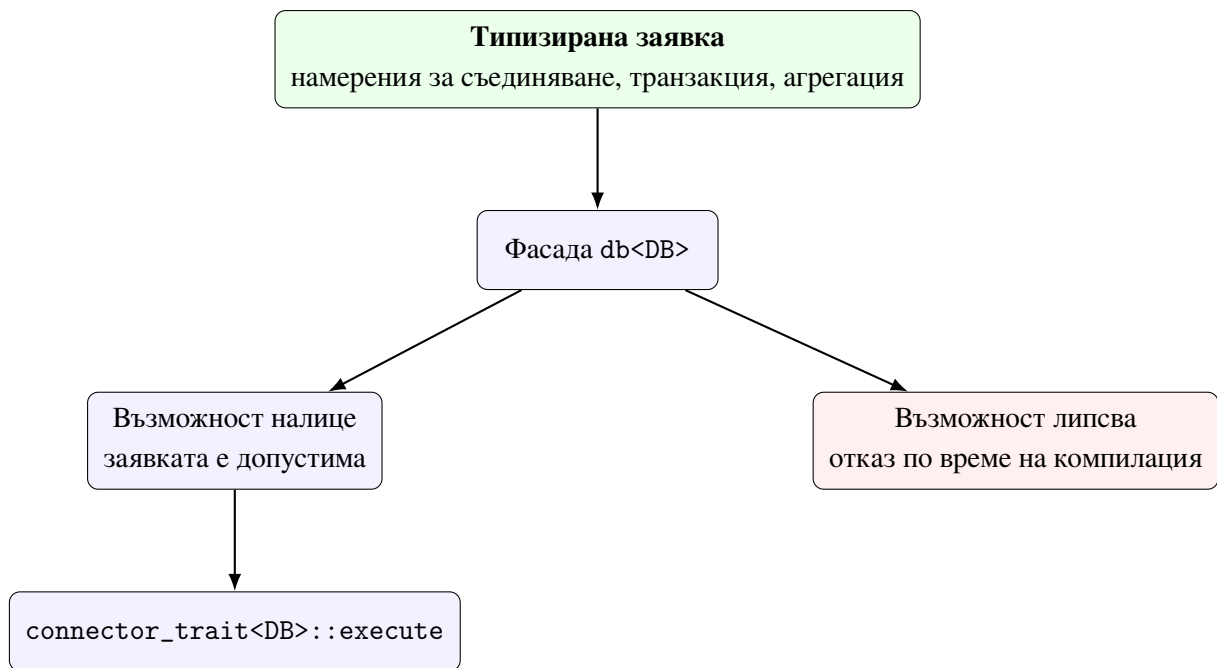
Формално конекторният слой е реализиран чрез специализация на `connector_trait<DB>`. Този подход е аналогичен на стандартни шаблони за характеристики като `std::iterator_traits`. Типът `DB` може да бъде проста обвивка (обвивка) над манипулатор на връзка, а цялата ОРС логика за него да остане в специализацията на характеристиката. По този начин библиотеката не изисква потребителят или авторът на конектора да наследява общ базов клас.

`connector_trait<DB>` определя поне три ключови аспекта: `wire_type`, `cursor_type` и `execute(...)`. По желание специализацията декларира и `labels` за възможности като `supports_joins`, `supports_transactions`, `supports_aggregation` или `supports_concurrent_execute`. Наличието или отсъствието на тези псевдо-маркери е достатъчно, за да може по-горният слой да наложи ограничения по време на компилация.

```
1  template <typename DB>
2  концепт is_конектор = requires {
3      typename конектор_trait<DB>::template wire_type<int>::type;
4      typename конектор_trait<DB>::cursor_type;
5  };
6
7  template <typename DB, typename Cap>
8  inline constexpr bool has_възможност =
9      detail::възможност_check<DB, Cap>::value;
```

Listing 3: Извадка от формалния договор на конектора

Listing 3 показва, че договорът е структурен, а не наследствен. Това е решаващ архитектурен избор. Ако съществуваше виртуален интерфейс с диспечирание по време на изпълнение (диспечирание по време на изпълнение), част от типово проверимата информация щеше да бъде изгубена. Вместо това библиотеката предпочита структура по време на компилация, при която изискванията са видими и за компилатора, и за автора на конектора.



Фигура 8. Проверка за възможности между междинното представяне на заявката и конекторния слой

Фигура 8 е важна, защото показва къде архитектурата взема решение за допустимост. Това не е драйверът и не е логът по време на изпълнение. Решението се взема в публичния приложен програмен интерфейс (API слой), който вече има достъп до междинното представяне и до информацията за характеристиките на конектора.

Таблица 8. Роля на основните labels за възможности в архитектурата

Възможност	Архитектурен смисъл
<code>supports_joins</code>	разрешава базираните на съединяване форми на заявки за съответния свързващ компонент
<code>supports_transactions</code>	позволява използване на <code>transaction_guard</code> и куки (куки) за <code>begin/commit/rollback</code>
<code>supports_aggregation</code>	обозначава поддръжка на операции за агрегация в логическия слой на заявките
<code>supports_embedding</code>	прави експлицитна платформено-специфичната поддръжка на вградени структури
<code>supports_upsert</code>	маркира компоненти, при които обновяването с вмъкване (<code>upsert</code>) е част от естествения договор за изпълнение
<code>supports_bulk_insert</code>	обозначава наличието на по-ефективен път за масово записване

5.6. Потребителският фасаден слой `db<DB>`

Класът `db<DB>` е публичният фасаден слой. Той капсулира препратка към конкретния обект на целевата система и предоставя уеднаквен програмен интерфейс за изпълнение. На практика този слой има две роли. Първата е удобство за потребителя: библиотеката се използва чрез един и същ интерфейс независимо от свързващия компонент. Втората е архитектурен филтър: точно тук се извършват проверки по време на компилация за възможности и съвместимост.

Следователно `db<DB>` не е просто тънка обвивка над `execute`. Той е мястото, където библиотеката формализира границата между разрешено и неразрешено поведение за избрания компонент. Точно този слой позволява съобщения за грешка от вида “конекторът не поддържа JOIN операции”, вместо грешката да се появи по-късно като неясно поведение по време на изпълнение.

```

1  template <typename DB>
2      requires is_конектор<DB>
3  class db
4  {

```

```

5 public:
6     template <typename Заявка>
7     auto operator<<(Заявка q)
8     {
9         if constexpr (is_select_заявка<Заявка>)
10        {
11            if constexpr (decltype(q.join_clauses())::size > 0)
12            {
13                static_assert(has_възможност<DB, cap::supports_joins>,
14                              "orm::db: този конектор не поддържа JOIN операции.");
15            }
16        }
17        return конектор_trait<DB>::execute(*conn_, std::move(q));
18    }
19 };

```

Listing 4: Извадка от фасадния слой db<DB>

Listing 4 показва защо db<DB> е повече от удобна фасада. Той е формалното място, където структурата на заявката и информацията за възможностите на конектора се срещат. Именно тук архитектурата по време на компилация се превръща в реално ограничение върху употребата на програмния интерфейс.

5.7. Миграции като архитектурно разширение

Архитектурата включва и прототипен миграционен слой чрез migrate<DB>, live_schema, entity_meta и операции за дефиниране на данни (DDL) като create_table_op, add_column_op, drop_column_op и alter_column_type_op. Този слой е особено важен от гледна точка на тезата, защото прави явна връзката между C++ модела и реалната схема на съхранение.

Миграционният слой не е универсално завършен за всеки свързващ компонент, но архитектурно показва, че библиотеката не се ограничава само до изпълнение на заявки. При достатъчно богата специализация на connector_trait<DB> описанието на същността може да участва и в еволюцията на схемата. Това променя и самия смисъл на ОРС слоя: той вече не е само интерфейс към данните, а потенциален източник на инструментариум, съобразен със схемата (инструментариум, съобразен със схемата).

5.8. Реализация на основните подсистеми

След като архитектурният модел е очертан, следващият въпрос е как той се материализира в конкретни C++ конструкции и как отделните подсистеми си взаимодействат в хранилището. Настоящият раздел възстановява липсващата инженерна перспектива: не само какви слоеве съществуват, а как точно се натрупват `where` клаузите, как обектът на заявката се запазва за повторна употреба, как `migrate<DB>` превръща разликата между описанието на същността и `live_schema` в DDL операции и как помощният (помощен) слой налага правила за нишкова безопасност и транзакции.

Тук се фиксира и работната нотация, без да се връща отделна речникова глава. В настоящия раздел `DB` означава общ тип свързващ компонент, `db<DB>` обозначава публичната фасада за изпълнение, `connector_trait<DB>` е формалният конекторен договор, `field<...>` е представяне на поле по време на компилация, `placeholder<T>` е анонимен параметър по време на изпълнение, а `orm::ph<T, _N>` е индексирани заместител. Контекстът на тази механика в хранилището е концентриран главно в `lib/include/ORM/entity/`, `lib/include/ORM/PчPеCSPÿPеPе/`, `lib/include/ORM/PеP«P,,PхPеCЪP«CГ/`, `lib/include/ORM/db/migration/` и във верификационния слой под `tests/PёP«PÿCЪPъP,,Pç/` и `tests/PçP,,CЪPхPÿCГPеCЗPçP«P,,P,,Pç/`.

5.8.1. Мини казус с типовете User и Post

За да остане изложението непосредствено обвързано с реалния код, този раздел използва като опорен мини казус (мини казус) типовете `User` и `Post` от `tests/PçP,,CЪPхPÿCГPеCЗPçP«P,,P,,Pç/test_mockdb.cpp`. Те са особено полезни, защото съчетават скаларни полета, явно именуване на логическите контейнери и сценарий със съединяване (`JOIN`). Същият пример по-късно се използва и за свързване на заместители, поведение на подготвени заявки и утвърждавания (утвърждавания) за рендиране на SQL.

```
1 struct Post
2 {
3     orm::property<int, "id">          id;
4     orm::property<int, "author_id">    author_id;
5     orm::property<std::u8string, "body"> body;
6 };
7
8 struct User
9 {
10     orm::property<int, "id">          id;
```



```

11     orm::property<std::u8string, "name"> name;
12     orm::property<double, "score"> score;
13 };

```

Listing 5: Типове същности от `tests/РџР,,СЪРхРҮГРРѐСЗРџР«Р,,Р,,Рџ/``test_mockdb.cpp`

Listing 5 показва не само как се описва моделът на същностите, но и как той непосредствено участва в тестовете. `table_name_trait<User>` и `table_name_trait<Post>` фиксират логическите контейнери "users" и "posts", а указателите към членове (указатели към членове) като `&User::id` и `&Post::author_id` по-късно участват в конструктора на заявки. Това е пряка илюстрация на един от централните мотиви на дипломната работа: моделът, формата на заявката и тестовете за изпълнение използват едни и същи артефакти по време на компилация.

5.8.2. Скалярни свойства и именуване на контейнерите

Реализацията на `property<T, Name>` свързва стойностен тип с име по време на компилация. Функцията `column_name()` дава еднозначен достъп до символното име на полето и се използва от конекторите при рендиране към SQL колони, полета в документи, сегменти в ключ-стойност пространства или други специфични за целевата система понятия. Конструкцията е минималистична, но достатъчна, защото не изисква външна регистрация на метаданни по време на изпълнение.

Именуването на логическия контейнер за съхранение се реализира чрез `table_name_trait<T>`. Макар името да е наследено от OPC традицията, архитектурно то играе по-обща роля: в SQLite, PostgreSQL и MySQL означава таблица, в MongoDB — колекция, в Redis — префикс на ключ, а в Neo4j — label. Така библиотеката използва единен логически интерфейс за адресиране на физически различни структури.

Практическата последица е важна: авторът на конектора не извлича схема от външен регистър, а от типовата система. Това намалява шаблонния код и прави грешките в именуването по-лесни за локализиране както в компилационната фаза, така и в модулните (модулни) тестове.

5.8.3. Обвивки на полета, правила и заместители

Полето в израз на заявка се представя чрез `field<&T::m>`. Това е `constexpr` обвивка над указател към член, която носи достатъчно информация за типа на контейнера, типа на стойността и името на колоната. Операторите върху `field` изграждат типизирани

Rule обекти. Така изрази от вида `field<&User::id>==placeholder<int>\{\}` не са низове, а описания на предикати по време на компилация.

Механизмът на заместителите (заместител) има две форми. `placeholder<T>` моделира анонимни параметри, които се консумират позиционно от аргументите на `execute().orm::ph<T,std::placeholders::_N>` моделира индексирани параметри, които позволяват една стойност по време на изпълнение да се използва на повече от една позиция. Това е особено важно за свързващи компоненти като SQLite, PostgreSQL и MySQL, при които слотът на заместителя е отделна част от логиката на материализация.

Тестовите `IndexedPlaceholderRendersQuestionMarkN`, `IndexedPlaceholderReusedSameArgInSQL` и `TwoDistinctIndexedPlaceholdersRenderCorrectSlots` в `tests/РџР,,СЪРхРѸГРѢСЗРџР«Р,,Рџ/test_mockdb.cpp` показват, че тази подсистема не е теоретична добавка, а реално използван механизъм за по-точни сценарии за свързване на параметри.

5.8.4. Изграждане на обекти на заявки

Фабричните функции `select`, `insert`, `update` и `deleteq` създават обекти на заявки, върху които последователно се наслагват `where`, `join`, `order_by`, `group_by` и `limit`. Същественото е, че всяка от тези операции не променя низ, а връща нов типизиран обект с разширено вътрешно състояние. В резултат конструкторът на заявки остава едновременно изразителен (изразителен) на вид и статичен по природа.

Това позволява в тестовите да се проверява не само крайният резултат от рендирането, а и самата форма на междинното представяне — например броят на `where` клаузите, типът на отговора или коректността на композицията на `GROUPBY` и `ORDERBY`. В `tests/РџР,,СЪРхРѸГРѢСЗРџР«Р,,Рџ/test_mockdb.cpp` това се вижда в `SelectWithInnerJoin`, `SelectWithOrderByDesc`, `GroupByMultipleFields`, `SelectWithLimit` и множество други сценарии, които валидират различни фази на композицията на заявката.

Ключов имплементационен детайл е, че веригата на конструктора е съвместима с `constexpr`. Вътрешният помощник `orm::detail::tuple_cat` използва разгъване на пакет чрез индексни последователности вместо зависимост от конкатенация по време на изпълнение. Практическата последица е, че значима част от формата на заявката може да бъде изградена и проверена още по време на компилация.

```
1 using namespace orm::literals;
```

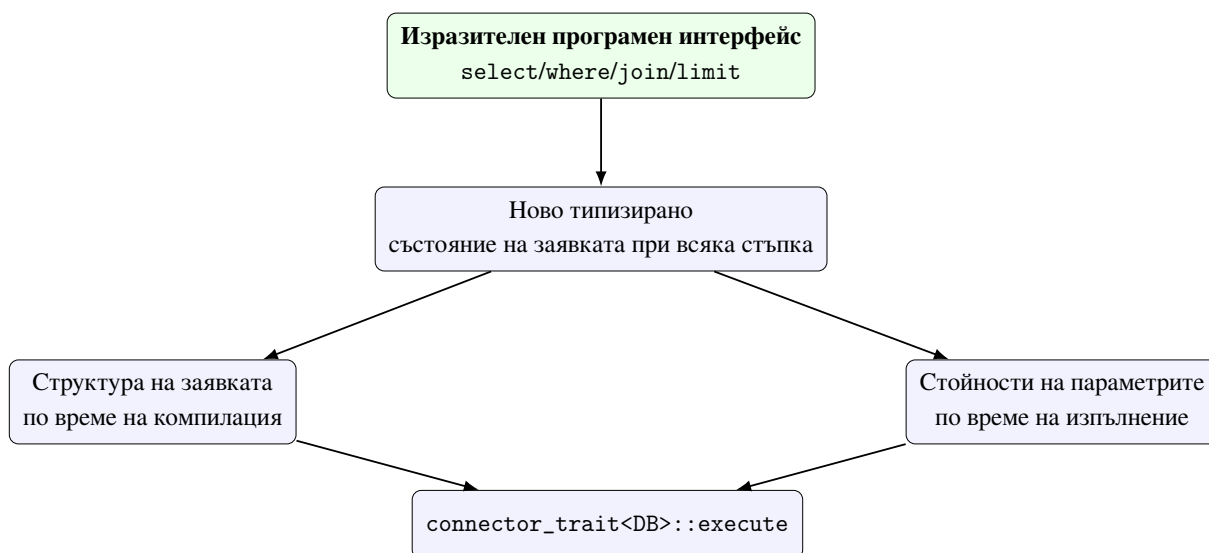
```

2 static constexpr auto get_active =
3     orm::select(orm::field<&User::id>, orm::field<&User::name>)
4         .where(orm::field<&User::score> > 0.0)
5         .where(orm::field<&User::id> == orm::Заместител<int>{})
6         .limit(10_per_page & 2_page);

```

Listing 6: Обект на заявка, деклариран като константа по време на компилация

Операторът & комбинира помощниците `_per_page` и `_page` в обект `Pagification`, чиято структура е известна по време на компилация. Хедърът `lib/include/ORM/РѢРѢCSPЎРѢРѢ/limits.hpp` поддържа и симетрична форма чрез `*`, но използването на & в Listing 6 е по-четливо за заявъчния стил на библиотеката. Така заявката може да бъде декларирана като `staticconstexpr` и да се използва многократно без реконструиране на формата на конструктора при всяко извикване.



Фигура 9. Взаимодействие между композиция на заявката, структура по време на компилация и параметри по време на изпълнение

Фигура 9 показва, че реализацията работи в два режима едновременно: структурната форма на заявката е фиксирана в типа, докато конкретните стойности се подават отделно към `execute(...)`. Именно това прави възможно едновременното наличие на проверки по време на компилация и параметризация по време на изпълнение.

5.8.5. Изпълнение, резултати и достъп до полета

На етап изпълнение `db<DB>` предава междинното представяне на заявката към `connector_trait<DB>::execute`. Резултатът се връща като `orm::result<Row, FieldTuple>`. В тази конструкция `Row` е материализираният тип кортеж (кортеж), а `FieldTuple` запазва

метаданните на проекцията за избраните полета. Това решение позволява както итериране върху материализираните редове, така и достъп чрез `get_field<&T::m>` или позиционен достъп чрез `get<I>`. Реализацията така остава типова безопасна и след изпълнението на заявката.

Тестовите `SelectResultRowTypeIsTuple`, `SelectMultiFieldRowType`, `SelectGetByMemberPointer` и `SelectGetByIndex` показват, че слой за резултати не е просто контейнер от анонимни редове, а структурирана абстракция с типова проследима проекция. Това е важно за тезата, защото демонстрира, че типовата дисциплина не приключва с конструктора на заявки, а продължава и в обработката на резултатите (обработка на резултатите).

5.8.6. Подготвени заявки (prepared заявки) като артефакт за многократно изпълнение

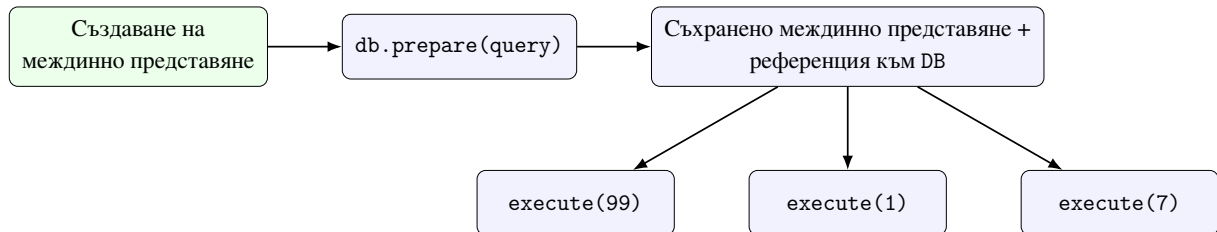
`prepared_query<DB, Query>` разширява изпълнителния модел. Вместо да кешира SQL низ, той съхранява връзка към обекта на свързващия компонент и самото междинно представяне. Това е естествено продължение на подхода по време на компилация: библиотеката третира обекта на заявката като първичен артефакт, а не като временна стъпка по пътя към текстов резултат.

```
1 template <typename DB, typename Заявка>
2 class prepared_заявка
3 {
4 public:
5     prepared_заявка(DB& conn, Заявка q) : conn_(conn), заявка_(std::move(q)) {}
6
7     template <typename... Params>
8     auto execute(Params&&... params) const
9     {
10         return конектор_trait<DB>::execute(
11             conn_, заявка_, std::forward<Params>(params)...);
12     }
13
14     [[nodiscard]] const Заявка& заявка() const noexcept { return заявка_; }
15 };
```

Listing 7: Извадка от `prepared_query<DB, Query>`

Listing 7 показва важен инженерен избор: повторната употреба е организирана около междинното представяне, а не около текстовата материализация. Това позволява една и съща логическа заявка да бъде изпълнявана многократно с различни параметри,

без да се реконструира формата на конструктора. Точно това поведение се валидира от `PrepareReturnsExecutableObject`, `PrepareWithParamExecutesCorrectly`, `PrepareCanBeCalledMultipleTimesWithDifferentParams` и `PrepareExposesQueryIR`.



Фигура 10. Жизнен цикъл на prepared_query: едно междинно представяне, множество изпълнения

5.8.7. Миграционен слой и DDL генериране

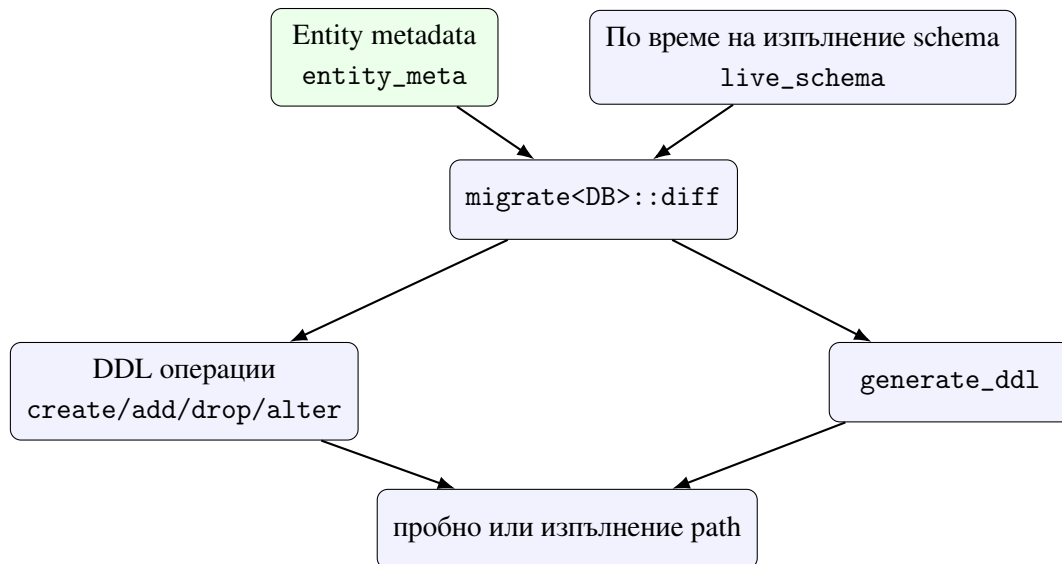
Реализацията на `migrate<DB>` сравнява описаната от типовете същности схема с `live_schema`. Когато има разлика, се създават DDL операции и се подават към `connector_trait<DB>::ddl_for(...)`. Това е добър пример за чисто разделяне на отговорностите: логиката за разлики е обща, а конкретният DDL синтаксис остава специфичен за свързващия компонент.

```

1 [[nodiscard]] std::vector<ddl_op> diff(
2     const live_schema& live,
3     std::span<const entity_meta> entities) const
4 {
5     std::vector<ddl_op> ops;
6
7     for (const auto& entity : entities)
8     {
9         auto live_it = live.find(entity.table_name);
10        if (live_it == live.end())
11        {
12            ops.push_back(create_table_op{entity.table_name, entity.columns});
13            continue;
14        }
15
16        for (const auto& [col, type] : entity.columns)
17        {
18            if (live_it->second.find(col) == live_it->second.end())
19                ops.push_back(add_column_op{entity.table_name, col, type});
20        }
21    }
22    return ops;
23 }
  
```

Listing 8: Ядро на migration diff конвейер-а

Тук ясно се вижда разликата между *минимален работещ конектор* и *пълен конектор договор*. Един свързващ компонент може да изпълнява заявки, без да поддържа DDL генериране. Но ако трябва да участва в инструментариум, съобразен със схемата, неговият договор трябва да бъде разширен с `ddl_for(...)` куки и последователна миграционна верификация.



Фигура 11. Поток на schema migration подсистемата

Тестовите в `tests/РёР«РүСЃРѢР,Рҷ/test_schema_migration.cpp` покриват създаване на таблица, добавяне и отпадане на колони, пробно кодове на завършване, генериране на DDL и поведението на автомат с крайни състояния на `run()`. Това е силно доказателство, че миграционният слой не е само концепция, а реално проследим конвейер.

5.8.8. Нишкова безопасност и transaction помощници

В библиотеката съществува отделен слой за `connection_pool<DB,N>`, `thread_local_db<DB>` и `transaction_guard<DB>`. Тези компоненти показват, че реализацията не спира до еднократно изпълнение на заявка, а разглежда и експлоатационни сценарии с повторна употреба на връзки, изолиране по нишки и отмяна дисциплина. Важното архитектурно решение е, че достъпът до тези механизми също е възможност-базиран. Например `connection_pool` изисква `supports_concurrent_execute`, а `transaction_guard` — `supports_transactions`.

```

1 template <typename DB, std::size_t N>
2 class connection_pool
3 {

```

```

4      static_assert(
5          requires { typename конектор_trait<DB>::supports_concurrent_execute; },
6          "connection_pool requires "
7          "конектор_trait<DB>::supports_concurrent_execute. "
8          "Add 'using supports_concurrent_execute = void;' "
9          "to конектор_trait<DB>.");
10
11 public:
12     [[nodiscard]] connection_предпазител<DB> acquire();
13 };
14
15 template <typename DB>
16 class transaction_предпазител
17 {
18     static_assert(
19         requires { typename конектор_trait<DB>::supports_transactions; },
20         "transaction_предпазител requires "
21         "конектор_trait<DB>::supports_transactions. "
22         "Add 'using supports_transactions = void;' "
23         "to конектор_trait<DB>.");
24 };

```

Listing 9: Извадка от нишкова безопасност помощен слой

Така по време на компилация договорът не се ограничава до формата на заявката, а достига и до жизнен цикъл политиките на конектора. Това е важно за аргумента на тезата, че библиотеката не е просто конструктор за заявки, а инфраструктура за систематична и безопасна работа с свързващ компонент ресурси.

Таблица 9. Основни подсистеми и тяхната тестова опора

Подсистема	Основни файлове	Тестова опора
Моделиране на същности	ORM/entity/property.hpp, ORM/entity/relationship.hpp	tests/PëP«PŸCŦPŦP,,Pç/test_property.cpp p, tests/PëP«PŸCŦPŦP,,Pç/test_relationships.cpp
Композиция на заявки	ORM/PçPөCSPŸPөPө/se lect.hpp и свързаните заглавни файлове	tests/PëP«PŸCŦPŦP,,Pç/test_PçPөCSPŸPөPө.cpp, tests/PçP,,CŦPçPŸCŦPөCЗPçP«P,,P,,Pç/test_mockdb.cpp
Подготвени заявки	ORM / PөP«P,,PçPөCŦP«CŦ/pr epared_PçPөCSPŸPөPө.hpp	Сценарии Prepare* в tests / PçP,,CŦPçPŸCŦPөCЗPçP«P,,P,,Pç/test_mockdb.cpp
Миграция на схема	ORM/db/migration/migration.hpp	tests/PëP«PŸCŦPŦP,,Pç/test_schema_migration.cpp
Нишкова безопасност	ORM / db /PөP«P,,PçPөCŦP«CŦ's /PŦPçCŦPөPөSafety/P,,PçCŦPөPө_safety.hpp	tests/PëP«PŸCŦPŦP,,Pç/test_P,,PçCŦPөPө_safety.cpp

5.8.9. Извод за реализацията

Основните подсистеми имат пряка опора в кода и тестовете. tests/PëP«PŸCŦPŦP,,Pç/test_property.cpp и tests/PëP«PŸCŦPŦP,,Pç/test_relationships.cpp валидират описанията на същностите. tests/PëP«PŸCŦPŦP,,Pç/test_PçPөCSPŸPөPө.cpp доказва, че конструкторът на заявки действително конструира правилни структури по време на компилация. tests/PçP,,CŦPçPŸCŦPөCЗPçP«P,,P,,Pç/test_mockdb.cpp и tests/PçP,,CŦPçPŸCŦPөCЗPçP«P,,P,,Pç/test_sqlite.cpp валидират изпълнението, сценариите за подготвени заявки и обработката на резултатите (обработка на резултатите). tests/PëP«PŸCŦPŦP,,Pç/test_schema_migration.cpp покрива логиката за разлики при миграциите, а tests/PëP«PŸCŦPŦP,,Pç/test_P,,PçCŦPөPө_safety.cpp проверява слоя

на жизнения цикъл за конкурентен достъп.

Следователно реализацията на основните подсистеми не е само проектно намерение, а архитектурно решение, подкрепено от систематична верификация. Това е и най-важният извод от настоящия раздел: архитектурата за обектно-релационно съпоставяне по време на компилация в хранилището не остава на равнище абстракция, а се материализира в работещи подсистеми с ясно разграничени отговорности и проследима опора в хранилището.

5.9. Архитектурни инварианти

На базата на разгледаните слоеве могат да се формулират няколко инварианта. Първо, приложният код никога не бива да зависи пряко от драйверен програмен интерфейс. Второ, логиката на заявката се формулира в типизирано междинно представяне, а не в низове, специфични за целевата система. Трето, разширяването към нов свързващ компонент се извършва чрез специализация на характеристика и декларации на възможности, а не чрез промяна на самия програмен интерфейс за заявки. Четвърто, ако една операция е недопустима за даден свързващ компонент, това трябва да стане видимо възможно най-рано — за предпочитане при компилация.

Тези инварианти правят архитектурата едновременно инженерно подредена и академично защитима. Те позволяват всяко следващо разширение да бъде оценявано не само по това дали “работи”, а и по това дали запазва вече формулираните слоеве и граници.

5.10. Архитектурен извод

Обобщено, архитектурата на библиотеката разделя проблема на три равнища: логически модел, типизирано междинно представяне на заявките и материализация в целевата система. Това разделение позволява едновременно строга типова дисциплина, формализирана разширяемост и контролирано поведение върху различни модели за съхранение. В следващата глава фокусът се премества към втората архитектурна ос — асинхронното изпълнение — където става ясно как същото ядро по време на компилация се свързва с жизнения цикъл на корутините, изнасянето (изнасяне) към набор от нишки и интеграцията с платформено-специфични неблокиращи интерфейси.

VI. АСИНХРОННА АРХИТЕКТУРА НА ИЗПЪЛНЕНИЕТО

След като архитектурният модел по време на компилация е дефиниран, следващият въпрос е как библиотеката реализира асинхронното изпълнение като втора основна архитектурна ос. Настоящата глава разглежда основните градивни елементи на асинхронния слой и ги проследява от `Task<T>` и композицията, базирана на `co_await`, до `thread_pool`, `async_db<DB>`, `IoContext`, съобразеното с корутини поддържане на пул от връзки и интеграцията със специфични за целевата система пътища на изпълнение.

Тук фокусът вече не е върху междинното представяне на заявките (междинно представяне на заявките) като структура по време на компилация, а върху реалната механика на спиране/възобновяване (спиране/възобновяване) на изпълнението. Как синхронният програмен интерфейс на конектора се повдига до асинхронен интерфейс? Къде изнасянето (изнасяне) към набор от нишки е правилната стратегия и къде платформено-специфичният неблокиращ интерфейс е по-точният модел? Как асинхронният слой гарантира коректна употреба на връзките и транзакциите без връщане към интерфейс, управляван от функции за обратно извикване (управляван от функции за обратно извикване)? Отговорът на тези въпроси показва доколко библиотеката е завършена не само като типова, но и като изпълнителна система.

6.1. Архитектурна позиция на асинхронния слой

Асинхронният слой е разположен над вече съществуващия синхронен модел `db<DB>` и `connector_trait<DB>`. Това е важен архитектурен избор. Вместо да дублира конструктора на заявки или да въвежда втори набор от типове заявки, асинхронната подсистема приема конструираното по време на компилация междинно представяне като готов вход и се занимава единствено с това *как* да бъде изпълнено. По този начин първата архитектурна ос — типovo описание на данните и операциите — остава отделена от втората ос — управление на асинхронния жизнен цикъл.

От тази гледна точка асинхронната архитектура има три основни отговорности. Първо, да осигури корутинен носител на състояние на изпълнение и продължение. Второ, да даде универсален механизъм за повдигане на блокиращи конектори към интерфейс, базиран на `Task`. Трето, да позволи платформено-специфични неблокиращи свързващи компоненти да участват в същия потребителски синтаксис, без библиотеката да симулира

фалшива еднородност там, където драйверите реално са различни.

6.2. Task<T> като носител на жизнения цикъл на корутината

Основната потребителска абстракция е Task<T>. Нейната роля не се изчерпва с това да „върне бъдещ резултат“. Тя е контейнер за корутинната рамка, обекта-обещание, продължението и политиката за завършване. Реализацията използва `initial_suspend()` и `final_suspend()` така, че корутината да бъде отложена (отложена) по начало и да предава управлението обратно към продължението при завършване. Следователно Task<T> е едновременно манипулатор на изпълнението и формален договор за това как резултат, изключение и завършване се предават между корутинни контексти.

От инженерна гледна точка са важни три свойства. Първо, `operator co_await()` свързва текущото продължение към изчакваната (изчаквана) задача и позволява естествено вложено композиране чрез `co_await`. Второ, `sync_wait()` осигурява мост към не-корутинен код, което е необходимо както за модулните (модулни) тестове, така и за гранични сценарии, в които приложението още не работи изцяло по корутинен начин (ориентиран към корутини). Трето, `detach()` и `start_detached()` позволяват контролирано поведение "изстреляй и забрави"(изстреляй и забрави), при което корутинната рамка се самоунищожава след `final_suspend()`.

Тази конструкция не е декоративна. Тя определя кога дадена асинхронна операция е отложена, кога започва да се изпълнява, кой носи отговорност за жизнения цикъл на рамката и как се разпространяват изключенията. Именно поради това Task<T> стои в центъра на всички останали асинхронни компоненти: `run_on_pool()`, `async_db<DB>`, `IOContext` изчакваемите обекти и помощниците за транзакции работят през един и същ корутинен договор.

6.3. thread_pool и run_on_pool(): универсалният път на изпълнение

Универсалната асинхронна стратегия е изнасяне към набор от нишки (изнасяне към набор от нишки). `thread_pool` поддържа ограничен набор от работни нишки и опашка от задачи. Върху него стъпва `run_on_pool()`, който приема извикваем обект (извикваем обект), публикува го към опашката, спира текущата корутина и я възобновява,

когато извикваният обект завърши. По този начин синхронният програмен интерфейс на конектора може да бъде използван през `co_await`, без потребителският код да се връща към ръчно свързване на обещания/бъдеща стойност (ръчно свързване на обещания/бъдещи стойности).

Критичният детайл е, че `run_on_pool()` не е просто помощник за „пусни нещо на друга нишка“. Неговият изчакващ обект (изчакващ обект) пази продължението, евентуалното изключение и резултата от извиквания обект. След изпълнение върху работната нишка продължението се възобновява (възобновяване), а `await_resume()` връща резултат или повдига грешка. Така пътят с изнасяне остава подчинен на същите корутинни правила като платформено-специфичния асинхронен път.

Този универсален модел е особено важен за свързващи компоненти, които нямат естествен платформено-специфичен неблокиращ клиентски интерфейс или при които библиотеката съзнателно не претендира такъв. В тези случаи честният архитектурен избор е блокиращата операция да бъде изолирана в `thread_pool`, а потребителят да получи единен интерфейс, базиран на `Task`. Точно това поведение се валидира от `RunOnPoolTest.*` и от `AsyncDbTest.AsyncSelectRunsOnPoolThread`, където се доказва, че реалното изпълнение се премества извън главната нишка.

6.4. `async_db<DB>` и диспечирание, ограничено от възможностите

Най-важната външна асинхронна фасада е `async_db<DB>`. Тя обгръща `db<DB>` и запазва познатия синтаксис на заявките, но връща `Task<result<...>>`. Архитектурното значение на тази обвивка е, че тя не решава динамично какво да направи по време на изпълнение, а използва ограничаване на база възможности (ограничаване на възможностите) по време на компилация. При `operator<<` има `ifconstexpr` разклонение върху `has_capability<DB, cap::supports_async>`. Ако конекторът декларира платформено-специфична асинхронна възможност, се извиква `connector_trait<DB>::async_execute()`. В противен случай синхронният път на `execute()` се обгръща в `run_on_pool()`.

```
1 template <typename Заявка>
2 [[nodiscard]] auto operator<<(Заявка q)
3     -> Task<decltype(std::declval<db<DB>>()) << std::declval<Заявка>())>
4 {
5     using ResultType = decltype(std::declval<db<DB>>()) << std::declval<Заявка>());
```

```

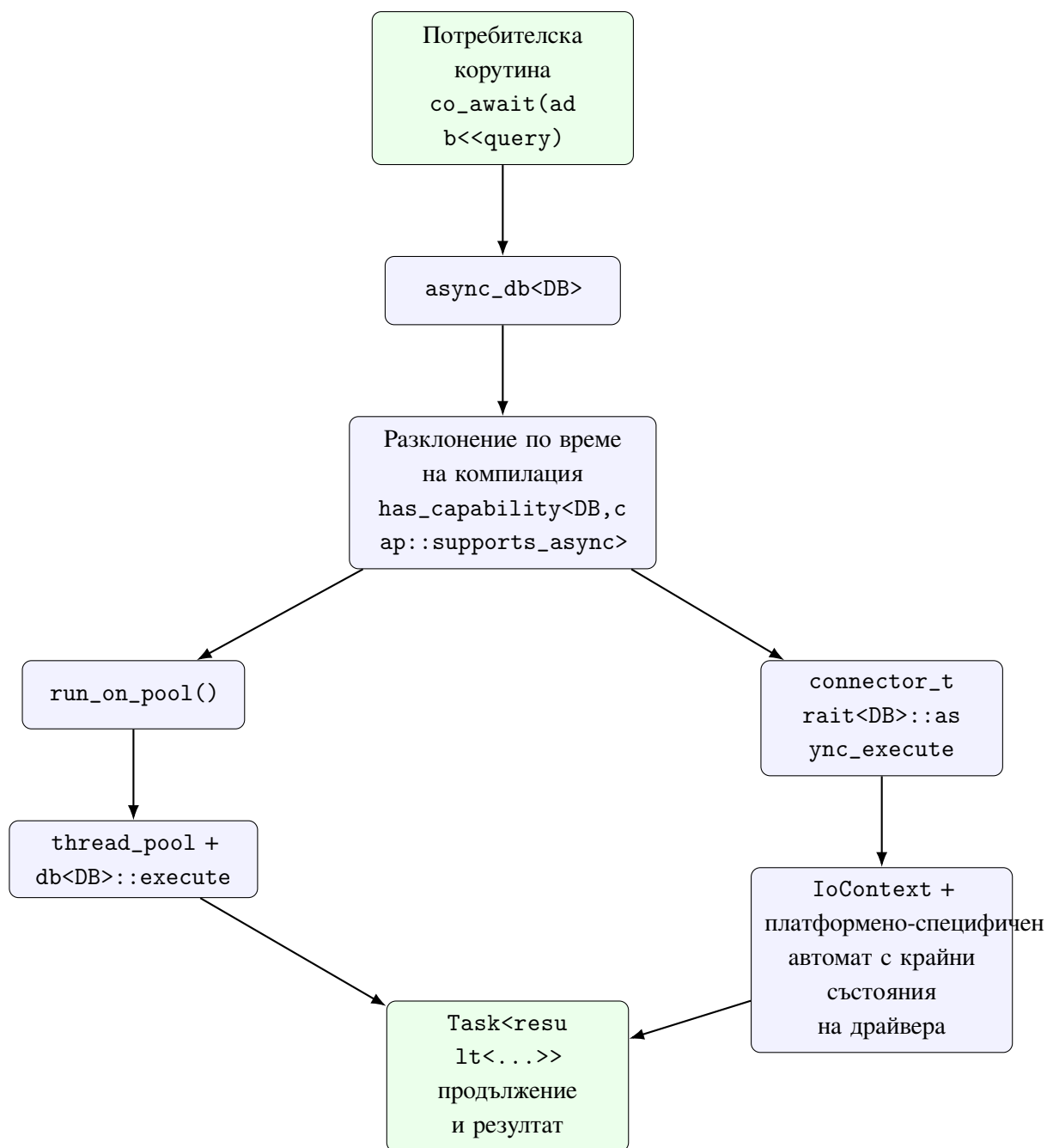
6
7     if constexpr (has_възможност<DB, cap::supports_асинхронен>)
8     {
9         co_return co_await конектор_trait<DB>::асинхронен_execute(
10             db_.connection(), std::move(q));
11     }
12     else
13     {
14         co_return co_await run_on_pool(
15             *pool_, [this, q = std::move(q)]() mutable -> ResultType {
16                 return db_ << std::move(q);
17             });
18     }
19 }

```

Listing 10: Централното разклонение, ограничено от възможностите, в `codeasync_db<DB>`

Listing 10 е централен за цялата асинхронна архитектура. Той показва, че библиотеката не използва регистър по време на изпълнение или базирано на низове диспечирание (базирано на низове диспечирание), а проверка по време на компилация върху възможностите (възможности) на конектора. По този начин потребителският програмен интерфейс остава еднороден, докато вътрешният път на изпълнение остава верен на реалните възможности на дадената целева система.

Помощният метод `async_execute(Query, Params...)` изпълнява сходна свързваща роля за императивни сценарии с явни параметри, а `sync_handle()` позволява контролиран изход към синхронния интерфейс, когато такава граница е необходима. Следователно `async_db<DB>` не е паралелна библиотека, а адаптационен слой между ядрото за OPC по време на компилация и корутинния модел на изпълнение.



Фигура 12. Двата пътя на изпълнение на `async_db<DB>`: универсално изнасяне (изнасяне) и платформено-специфична асинхронна интеграция

Фигура 12 показва най-важното архитектурно решение в асинхронния слой. Външният интерфейс остава единен, но вътрешният път на изпълнение е ограничен от възможностите (ограничен от възможностите). Това е причината библиотеката едновременно да предлага удобен корутинен програмен интерфейс и да избягва фалшивото твърдение, че всички драйвери са естествено неблокиращи.

6.5. IoContext и платформено-специфична неблокираща интеграция

За драйвери с истински платформено-специфичен асинхронен интерфейс библиотеката не се ограничава до изнасяне (изнасяне) към набор от нишки. Тук влиза в действие `IoContext` — абстракция за цикъл за събития, която предоставя `run()`, `run_one()`, `stop()`, `post()`, `schedule()` и най-важното `watch_readable(fd)` и `watch_writable(fd)`. Тези изчакваеми обекти не извършват входно-изходна операция сами по себе си. Те само спират корутината и я възобновяват, когато файловият дескриптор стане готов за четене или запис.

Точно това разделение между готовност и самото I/O действие е решаващо. `IoContext` не се опитва да „скрие“ драйвера, а предоставя съобразен с корутините реакторен (реакторен) интерфейс, върху който платформено-специфичната клиентска библиотека може да стъпи. В реализацията на `AsyncPostgreSQLDB` това се вижда особено ясно. Асинхронното свързване използва `PQconnectPoll()`, а корутината последователно изчаква (`await`) `watch_writable()` или `watch_readable()` според състоянието на автомата с крайни състояния на `libpq`. При изпълнение на заявка `PQsendQueryParams()`, `PQflush()`, `PQisBusy()` и `PQconsumeInput()` се комбинират със същия механизъм от `IoContext`.

Следователно платформено-специфичният асинхронен път не е „по-бърз набор от нишки“, а качествено различен изпълнителен модел. Работната нишка не блокира в очакване на мрежова готовност; корутината се спира (спиране), а възобновяването се управлява от наблюдението на цикъла за събития. Това е точно архитектурният модел, който следващата глава развива по свързващи компоненти: PostgreSQL, MySQL, Redis и Cassandra могат да предложат платформено-специфична асинхронна интеграция с различни експлоатационни форми, докато други драйвери остават честно описани като блокиращи и следователно се обслужват от универсалния път за изнасяне.

6.6. Поддържане на пул от връзки, съобразено с корутини, и помощници за транзакции

Асинхронната архитектура не приключва с единичното изпълнение на заявка. Реалното приложение трябва да управлява връзки и транзакции при конкурентен достъп.

Именно затова библиотеката предоставя `async_connection_pool<DB,N>`, `async_connection_guard<DB>`, `async_transaction_guard<DB>` и фабричната корутина `async_begin_transaction()`.

`async_connection_pool<DB,N>` е съобразено с корутини продължение на синхронния слой за поддържане на пул. Неговият метод `acquire()` връща `Task<async_connection_guard<DB>>`. Вътрешно той използва `run_on_pool()` и условна променлива (условна променлива), за да изчака слот за връзка със състояние `Idle`, без извикващата корутина да бъде изложена на блокиращ интерфейс. Полученият предпазител (предпазител) следва идиомата за придобиване на ресурси чрез инициализация (RAII): при разрушаване той връща състоянието на връзката към `Idle` и уведомява чакащите.

От своя страна `async_connection_guard<DB>::get()` не връща сурова връзка, а нов манипулатор `async_db<DB>`. Това е важен архитектурен детайл. Дори след придобиване на конкретен ресурс за връзка потребителят остава в същия корутинен модел на изпълнение и не пада обратно към програмен интерфейс на ниско ниво.

Сходна логика се вижда и при транзакциите. `async_begin_transaction()` първо изнася (изнасяне-ва) `BEGIN` към пула, а след това връща `async_transaction_guard<DB>`. Методите `commit()` и `rollback()` са `Task<void>` операции, а деструкторът на предпазителя изпълнява защитен `ROLLBACK`, ако транзакцията не е финализирана изрично. Този дизайн е едновременно удобен и консервативен: пътят на успеха е съобразен с корутини, а пътят на изчистването (път на изчистването) остава детерминистичен.

Точно тези свойства се валидират от `AsyncConnectionPoolTest.ConcurrentAcquires`, `AsyncTransactionTest.BeginAndCommit`, `AsyncTransactionTest.RollbackOnDestruction` и `AsyncTransactionTest.ExplicitRollback`. Следователно помощниците за пул и транзакции не са периферни удобства, а важна част от цялостната архитектура на изпълнение.

Таблица 10. Основни асинхронни подсистеми и тяхната кодова и тестова опора

Подсистема	Архитектурна роля	Основни файлове и тестове
Task<T>	жизнен цикъл на корутината, продължение, sync_wait(), семантика на отделяне (detach)	lib/include/ORM/PeCFPqP,,CЖCГP«P,,PxP,,/task.hpp, tests/PeP«PyCTPъP,,Pq/test_ask.cpp
thread_pool и run_on_pool()	универсален път за изнасяне (изнасяне) за блокиращи операции	lib/include/ORM/PeCFPqP,,CЖCГP«P,,PxP,,/P,,PqCГP«P« _ pool .hpp, tests / PeP«PyCTPъP,,Pq / test _P,,PqCГP«P« _ pool.cpp
async_db<DB>	ограничено от възможности диспечирание над db<DB>	lib/include/ORM/PeP«P,,PxP«CTP«CГ / PeCFPqP,,CЖCГP«P,,PxP,, _ db .hpp, tests / PeP«PyCTPъP,,Pq / test _PeCFPqP,,CЖCГP«P,,PxP,, _ PeP«P,,PxP«CTP«CГ .cpp
IoContext и платформено-специфични асинхронни конектори	готовност на файлови дескриптори в реакторен стил и платформено-специфична неблокираща интеграция	lib/include/ORM/PeCFPqP,,CЖCГP«P,,PxP,,/io_context.hpp, lib/include/ORM/db /PeP«P,,PxP«CTP«CГs/PostgreSQLDB/postgresql_PeCFPqP,,CЖCГP«P,,PxP,,.hpp, tests/PeP«PyCTPъP,,Pq/test_io_context.cpp
Пул и транзакции	съобразена с корутини собственост върху връзките и транзакционна безопасност	lib/include/ORM/db /PeP«P,,PxP«CTP«CГs /PъPqCГP«P« Safety / PeCFPqP,,CЖCГP«P,,PxP,, _ P,,PqCГP«P« _ safety .hpp, tests / PeP«PyCTPъP,,Pq / test _PeCFPqP,,CЖCГP«P,,PxP,, _ PeP«P,,PxP«CTP«CГ .cpp

6.7. Архитектурен извод за асинхронния слой

Асинхронната архитектура на библиотеката не е просто добавена корутинна обвивка около синхронното ОРС ядро. Тя е самостоятелен слой с ясно разграничени отговорности: Task<T> моделира жизнения цикъл на корутината, run_on_po

ol() осигурява универсален механизъм за изнасяне (изнасяне), async_db<DB> реализира диспечирание, ограничено от възможностите, IoContext предоставя платформено-специфична интеграция на готовността, а помощниците за пул и транзакции разширяват модела към практически приложими многозадачни сценарии.

Следователно втората архитектурна ос на дипломната работа не се свежда до това „да има асинхронен програмен интерфейс“. Тя показва как формализираният модел за достъп по време на компилация може да бъде изпълняван през два честно разграничени асинхронни пътя на изпълнение — универсален и платформено-специфичен — без библиотеката да губи типова дисциплина, безопасност на жизнения цикъл или прозрачност на конектора. Именно този слой прави възможен следващия анализ на изпълнителните модели на свързващите компоненти.

VII. ИЗПЪЛНИТЕЛНИ МОДЕЛИ НА СВЪРЗВАЩИТЕ КОМПОНЕНТИ И ПЛАТФОРМЕНО-СПЕЦИФИЧНИ КЛИЕНТСКИ БИБЛИОТЕКИ

Настоящата глава изследва не само как една и съща C++ абстракция се материализира в различни целеви системи (свързващи компоненти), но и как се изпълнява през конкретните платформено-специфични клиентски библиотеки. Анализът има две цели. Първо, да покаже, че библиотеката не е ограничена до чисто SQL интерпретация. Второ, да изясни къде асинхронната интеграция може да следва платформено-специфичен неблокиращ модел и къде честният архитектурен избор остава изнасяне към набор от нишки (изнасяне към набор от нишки) върху синхронен драйвер.

След теоретичната рамка от Глава III и архитектурното обяснение на асинхронния слой от Глава VI, тук вече не се пита дали различните модели за съхранение са еквивалентни. Вместо това се пита дали една и съща логическа OPC форма може да бъде адаптирана контролирано към тях, без библиотеката да обещава семантика на изпълнение, която даден компонент не може естествено да изпълнява. Тъкмо в това се проявява стойността на базирания на възможности договор на конектора и на разграничението между универсален асинхронен интерфейс и специфична за компонента реализация.

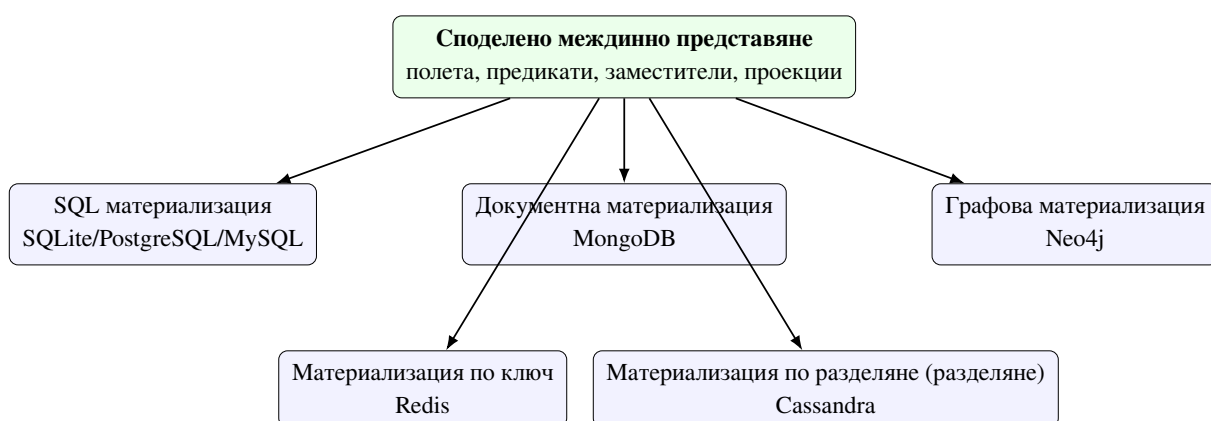
7.1. Методика на анализа

Всеки сценарий следва една и съща последователност: теоретична опора, логическо картографиране от C++ модела, реализация в `connector_trait<DB>`, ограничения и верификация чрез тестове. Това прави сравнението между компонентите възможно. Не се търси твърдение, че всички системи са еквивалентни, а че една и съща логическа OPC абстракция може да бъде адаптирана контролирано към различни модели за съхранение.

Таблица 11. Еднаква аналитична рамка за всеки сценарий на свързващ компонент

Критерий	Какво се оценява
Теоретична опора	към кой модел за съхранение от Глава III принадлежи свързващият компонент
Логическо картографиране	как <code>property</code> , <code>field</code> , <code>Rule</code> и <code>table_name_trait</code> се интерпретират в конкретната система
Профил на възможностите	кои OPC операции са изрично допустими и кои трябва да бъдат отхвърлени
Поведение при изпълнение	как се рендират заявки, параметри и резултатни проекции
Тестова опора	кои модулни (модулни) и интеграционни тестове доказват договора в хранилището

Тази аналитична рамка е важна, защото позволява сравнението да остане честно. SQL системите не се привилегирват само защото имат по-богат синтаксис, а не-SQL (NoSQL) системите не се оценяват като “непълни” само защото отказват операции, които са неподходящи за техния модел на съхранение.



Фигура 13. Едно споделено междинно представяне и множество специфични за компонентите пътища за материализация

Фигура 13 показва ключовото твърдение на главата: не съществуват множество OPC библиотеки в рамките на хранилището, а една логическа библиотека с различни пътища за материализация. Това е силният аргумент в полза на архитектурата на конектора — разнообразието от целеви системи е овладяно чрез общо представяне и изричен договор

(изричен договор), а не чрез копиране на програмни интерфейси.

7.2. Релационен базов сценарий: SQLite

Този сценарий стъпва върху релационния модел, разгледан в Глава III. SQLiteDB е най-естествената референтна целева система за библиотеката, защото съчетава реална SQL семантика, малък експлоатационен контекст и пряка интеграция чрез програмния интерфейс на sqlite3. В него `property` се материализира като колона, `table_name_trait` — като име на таблица, а дърветата `Rule` — като `WHERE` изрази.

Конекторът декларира `supports_joins`, `supports_transactions` и `supports_aggregation`, което го прави най-близък до класическото OPC очакване. Именно върху него се валидират подготвени заявки, логика за свързване на параметри (свързване), `find_one()` сценарии и попълване на резултати (попълване на резултати) към типизирани кортежи. Интеграционните тестове в `tests/РчР,,СЪРхРҮСГРөСЗРчР«Р,,Рч/test_sqlite.cpp` показват не само коректност на `SELECT`, `INSERT`, `UPDATE` и `DELETE`, но и поведение при конектор с възможност за транзакции и профил с възможност за съединяване (с възможност за съединяване).

От академична гледна точка SQLite играе ролята на релационна базова линия (базова линия). Ако дадена OPC форма е проблематична още тук, тя трудно би могла да бъде обобщена към по-екзотични системи. Затова настоящата работа използва SQLite като отправна точка, а не просто като един от многото примери.

7.3. Релационна преносимост: PostgreSQL и MySQL

Теоретичната основа и тук е релационният модел, но архитектурният интерес е различен: доколко едно и също междинно представяне остава валидно при различни SQL диалекти и различни правила за свързване на параметри. PostgreSQLDB и PostgreSQL LiveDB поддържат `supports_joins`, `supports_transactions` и `supports_aggregation`, а тестовете в `tests/РөР«РҮСГРөР»Р,,Рч/test_postgresql_РөР«Р,,РхРөСГР«СГ'.cpp` и `tests/РчР,,СЪРхРҮСГРөСЗРчР«Р,,Рч/test_postgresql_live.cpp` показват реално изпълнение срещу жива база данни.

MySQLDB и MySQLLiveDB също декларира `supports_joins` и `supports_transactions`, но тук е важно различието в модела за свързване. Докато PostgreSQL може естествено да адресира параметри чрез `$1`, `$2`, ..., MySQL разчита на позиционни ?

заместители (заместители). Това не нарушава общото междинно представяне, но показва, че логическата преносимост не означава идентична байт по байт материализация в целевата система.

Тъкмо тези сценарии от SQL семейството защитават по-силна теза от “работи със SQL”: библиотеката демонстрира, че междинното представяне е достатъчно общо за повече от един диалект, а в същото време е достатъчно честно, за да остави нисконивовите различия на конекторния слой.

7.4. Документен сценарий: MongoDB

Този сценарий стъпва върху документния модел от теоретичната глава. При MongoDB property се интерпретира като поле в документ, а дърветата от предикати на заявките се превеждат към BSON-подобни филтри. select изразите се материализират като проекция (проекция), а таблицата от гледна точка на обектно-релационното съпоставяне се превръща в колекция. Точно тук се вижда практическата стойност на независимото от съхранението междинно представяне: потребителят не описва `$eq`, `$and` и проекция директно, но конекторът може да ги изведе от същата логическа форма.

Важна подробност е, че `connector_trait<MongoDB>` не декларира `labels` за възможности в SQL стил. Това не е пропуск, а коректно описание на договора на конектора. Няма претенция, че Mongo целевата система е еквивалентна на съединяване (еквивалентна на съединяване) спрямо релационните конектори. Вместо това той предлага документно-естествено тълкуване на междинното представяне. Модулните и интеграционните тестове в `tests/PäP«PŸCŦPŦP,,Pç/test_mongodb_PəP«P,,PхPəCŦP«CŦ'.cpp` и `tests/PçP,,CŦPхPŸCŦPəCЗPçP«P,,Pç/test_mongodb_live.cpp` валидират тази адаптация срещу реален драйверен слой.

Архитектурната стойност на този сценарий е, че доказва едно важно ограничение: една библиотека може да работи с множество свързващи компоненти, без да твърди, че всички те поддържат едни и същи операции. Именно този тип “контролирана частичност” прави дизайна научно защитим.

7.5. Сценарий ключ-стойност: Redis

Този сценарий стъпва върху теорията за ключ-стойност базите данни. При Redis един и същ модел на същността може да бъде материализиран по два основни начина: като

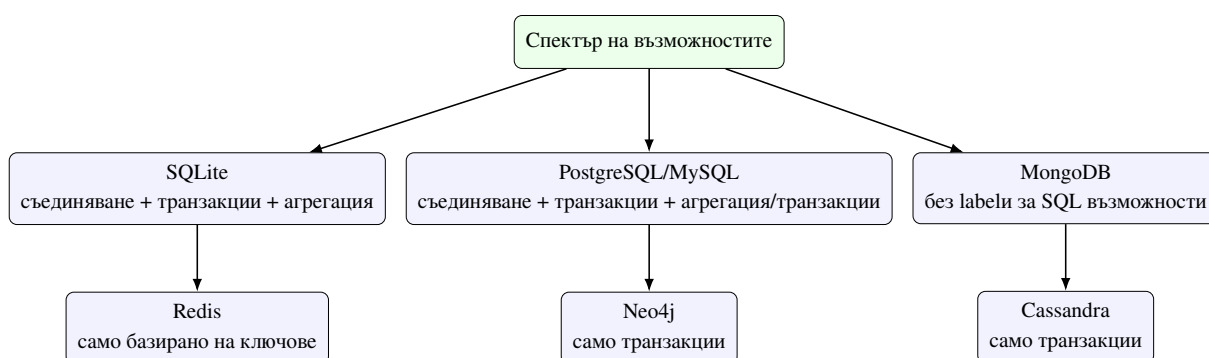
материализация, подкрепя централната архитектурна претенция.

7.7. Сценарий за широки колонни бази: Cassandra

Този сценарий стъпва върху теорията за широки колонни бази данни (широки колонни). Макар Cassandra да предлага CQL синтаксис, наподобяващ SQL, действителният модел на достъп е подчинен на ключовете за разделяне (ключове за разделяне) и на разпределената организация на данните. Затова библиотеката не може да третира Cassandra като обикновен SQL компонент. Наличието на WHERE клауза само по себе си не е достатъчно; тя трябва да бъде съвместима с логиката на разделянето.

CassandraDB и CassandraLiveDB декларират `supports_transactions`, но не и `supports_joins`. Това е много показателно. От една страна, свързващият компонент продължава да е част от същата OPC библиотека. От друга страна, договорът категорично отказва операции, които биха били подвеждащи или скъпи в среда на широки колони. Модулните тестове и тестовите на живо в `tests/РѢР«РҮСЃРЇР»Р,,Рҫ/test_cassandra_РѢР«Р,,РҫРѢСЃР«СЃ`.cpp и `tests/РҫР,,СЃРҫРҮСЃРЇР«СЃРҫР«Р,,Р,,Рҫ/test_cassandra_live.cpp` дават конкретна верификация на този по-строг експлоатационен профил (експлоатационен профил).

Именно поради това Cassandra е най-добрият пример, че разширяемостта на библиотеката не е базирана на фалшиво уеднаквяване, а на формално описани граници на допустимите операции.



Фигура 14. Сравнителен профил на възможностите на свързващите компоненти в хранилището

Фигура 14 синтезира едно от най-важните наблюдения в главата: библиотеката не налага еднакъв профил на възможности на всички свързващи компоненти. Точно обратното — различията се използват като първокласна (първокласна) архитектурна

информация.

7.8. Синтезирана матрица на възможностите

За да остане сравнението не само качествено, но и таблично проследимо, Фигура 14 може да бъде сведена до компактна матрица на възможностите. Тя не бива да се чете като универсална истина за самите системи за бази данни, а като описание на конкретната архитектурна позиция на библиотеката спрямо тях в разглежданата ревизия на хранилището.

Таблица 12. Профили на възможностите на основните свързващи компоненти според наличните тестове и специализации на конектори

Компонент	Транзак- Съединявания		Агрега- ция	Масово записване Обновяване (Bulk insert) с вмъкване (upsert)		Кратък коментар
SQLite	налична	налична	налична	ограни- чена	ограни- чена	най-близкият пълен релационен референтен път в хранилището
PostgreSQL	налична	налична	налична	специ- фична	въз- можна	силен релационен договор с доказателства за изпълнение на живо
MySQL	налична	налична	налична	специ- фична	въз- можна	експлоатационният път е наличен, но детайлите за свързване и диалекта остават специфични за конектора
MongoDB	липсва	липсва	липсва	аналог	не е централна	силен документен модел без профил на възможностите в SQL стил
Redis	липсва	липсва	липсва	аналог	ограни- чена	семантика ключ-стойност с минималистичен, но честен договорен профил
Neo4j	аналог	налична	липсва	не е централна	не е централна	графова целева система с фокус върху транзакциите, а не върху слой за SQL69 съединяване/агрегация
Cassandra	липсва	налична	липсва		въз- можна	широка колона

Таблица 12 позволява по-прецизно да се види къде библиотеката е най-близо до класическо ОРС поведение и къде съзнателно работи с частичен договор. Релационните целеви системи концентрират по-богат профил на възможностите, докато документните, графовите и ключ-стойност вариантите се оценяват през други критерии. Именно тази контролирана нееднородност прави дизайна за множество свързващи компоненти инженерно честен.

7.9. Съпоставка между тестови доказателства и договор на компонента

Таблица 13. Свързващи компоненти, профил на възможностите и налична тестова опора

Компонент	Профил на възможностите	Модулен тест/тест на договора	Интеграционен/жизнен тест
SQLite	съединявания, транзакции, агрегация	утвърждавания на възможностите в tests/ PчP,,CЪPхPүCГPөCЗPчP«P,,P,,Pч/ test_sqlite.cp P	tests/PчP,,CЪPхPүCГPөCЗPчP«P,,P,,Pч/te st_sqlite.cpp
PostgreSQL	съединявания, транзакции, агрегация	tests/ PөP«PүCГPөP,,Pч/ test_p ostgresql_ PөP«P,,PхPөCГP«CГ. cpp	tests/PчP,,CЪPхPүCГPөCЗPчP«P,,P,,Pч/te st_postgresql_live.cpp
MySQL	съединявания, транзакции, агрегация	tests/ PөP«PүCГPөP,,Pч/ test_mysql _PөP«P,,PхPөCГP«CГ. cpp	tests/PчP,,CЪPхPүCГPөCЗPчP«P,,P,,Pч/te st_mysql_live.cpp
MongoDB	без labels за SQL възможности	tests/ PөP«PүCГPөP,,Pч/ test_mongodb _PөP«P,,PхPөCГP«CГ. cpp	tests/PчP,,CЪPхPүCГPөCЗPчP«P,,P,,Pч/te st_mongodb_live.cpp
Redis	без съединявания/ транзакции/ агрегация	tests/ PөP«PүCГPөP,,Pч/ test_redis _PөP«P,,PхPөCГP«CГ. cpp	tests/PчP,,CЪPхPүCГPөCЗPчP«P,,P,,Pч/te st_redis_live.cpp

Таблица 13 е важна, защото превежда сравнението от равнището на описателен анализ (описателен анализ) към равнището на проверими доказателства от хранилището (проверими доказателства от хранилището). За всеки компонент има поне един ясен корелат на ниво договор или на ниво изпълнение на живо. Това намалява риска главата да се превърне в декларативен преглед без техническа опора.

7.10. Сравнителен синтез

Съпоставката между целевите системи показва, че библиотеката успява да запази единен логически модел на равнището на типове същности, полета, заместители и междинно представяне. Различията се появяват на равнището на възможностите и на физическата организация на данните. Това е и най-важният извод на настоящата глава: успешната ОРС архитектура за множество компоненти не премахва различията между моделите за съхранение, а ги прави контролируеми чрез формален договор на конектора.

Таблица 14. Съпоставка на материализацията в целевите системи

Компонент	Основна структура	Логическа интерпретация	Показателно ограничение
SQLite	таблица	пълна SQL OPC интерпретация	стандартна релационна дисциплина и богата изразителност на заявките
PostgreSQL/MySQL	таблица	преносимо SQL представяне	различен модел на свързване и диалектни особености
MongoDB	документ	интерпретация като филтър/проекция	частична еквивалентност спрямо SQL операции
Redis	ключ/стойност, хеш (hash)	базиран на ключове достъп до същности	търсене предимно по първичен ключ и липса на семантика за съединяване
Neo4j	възел/свойства	интерпретация като графов модел (graph-pattern)	специфична за графове семантика на заявките вместо слой за SQL съединяване
Cassandra	широк колонен ред	CQL с ключови ограничения	достъп, управляван от разделянето, и ограничен профил на предикатите

В резултат дизайнът на библиотеката за множество свързващи компоненти може да бъде защитен с по-прецизна формулировка. Системата не обещава “универсална база данни през един програмен интерфейс”, а *унифициран логически слой с експлицитни граници на целевата система*. Именно тази формулировка е едновременно инженерно честна и научно обоснована.

VIII. ВЕРИФИКАЦИЯ И ЕМПИРИЧНА ОЦЕНКА

Тази глава обединява двата вида доказателства, върху които се опира настоящата работа. От една страна стоят ограниченията по време на компилация, модулните (модулни) тестове, интеграционните тестове и проверките на целевите системи на живо, чрез които се валидира коректността на архитектурата. От друга страна стои емпиричната оценка на асинхронния слой, чиято цел е да покаже кога базираното на корутини изпълнение носи реална практическа полза и кога неговата цена остава сравнима или по-висока от синхронната алтернатива.

Важният методологичен принцип е, че тези два вида доказателства не се разглеждат като взаимозаменяеми. Тестовата верификация отговаря на въпроса дали библиотеката е коректна спрямо собствените си договори (договорс) и ограничения на целевата система. Оценката чрез сравнителни тестове (сравнителен тест) отговаря на различен въпрос: носи ли асинхронният модел на изпълнение измерима полза при натоварвания, в които фазата на изчакване (фаза на изчакване) доминира над локалната изчислителна работа. Само съвместното присъствие на двата пласта прави архитектурните претенции на разработката достатъчно убедителни.

8.1. Стратегия на верификацията

Верификационната стратегия следва самата архитектура на библиотеката. Сложат по време на компилация се проверява чрез концепти, ограничения на възможностите и типизирано изграждане на междинно представяне на заявките. Сложат по време на изпълнение се проверява чрез модулни тестове за рендиране, параметризиране, нишкова безопасност, миграции, корутинна инфраструктура и асинхронен достъп. Най-външният слой се валидира чрез интеграционни тестове с реални целеви системи, така че да не се смесват коректност на имитациите (коректност на имитациите) и реално поведение на драйверите.

В текущата ревизия на хранилището пълният автоматизиран тестов набор включва 271 теста при изпълнение чрез `cctest`. Това число само по себе си не е научен резултат, но е важно като индикатор, че изложените в предходните глави механизми не са оставени на ниво концептуална схема. Те са покрити от систематични проверки по слоеве и по семейства целеви системи.

Таблица 15. Основни източници на верификационни доказателства

Източник	Какво валидира	Представителни артефакти
Ограничения по време на компилация и междинно представяне	типова коректност на заявки, базирано на възможности ограничаване и договори на конекторите	<code>tests / PëP«PÿCñPñP,,Pç / test _PçPëCSPÿPëPë . c p p, tests / PëP«PÿCñPñP,,Pç / test _post g r e s q l _ PëP«P,,PçPëCñP«CΓ . c p p, tests / PëP«PÿCñPñP,,Pç / test _mon godb_PëP«P,,PçPëCñP«CΓ . c p p</code>
Асинхронна инфраструктура	<code>Task<T></code> , <code>thread_po ol</code> , <code>IoContext</code> , <code>async _db<DB></code> , съобразено с корутини поддържане на пул и транзакции	<code>tests / PëP«PÿCñPñP,,Pç / test _t a s k . c p p, tests / PëP«PÿCñPñP,,Pç / test _P,,PçCïPëPë _p o o l . c p p, tests / PëP«PÿCñPñP,,Pç / test _io _context . c p p, tests / PëP«PÿCñPñP,,Pç / t e s t _PëCñPçP,,CñCïP«P,,PçP,, _ PëP«P,,PçPëCñP«CΓ . c p p</code>
Миграции и нишкова безопасност	логика за разлики в схемата (schema diff), семантика на отмяната (отмяна semantics), предпазители на връзки и конкурентна безопасност	<code>tests / PëP«PÿCñPñP,,Pç / test _s c h e m a _m i g r a t i o n . c p p, tests / PëP«PÿCñPñP,,Pç / test _P,,PçCïPëPë _s a f e t y . c p p</code>
Интеграции с целеви системи на живо	поведение върху реални SQLite, PostgreSQL, MySQL, MongoDB, Redis, Neo4j и Cassandra среди	<code>tests / PçP,,CñPçPÿCïPëCçPçP«P,,Pç / test _s q l i t e . c p p, tests / PçP,,CñPçPÿCïPëCçPçP«P,,Pç / test _p o s t g r e s q l _l i v e . c p p, tests / PçP,,CñPçPÿCïPëCçPçP«P,,Pç / test _m o n g o d b _l i v e . c p p, tests / PçP,,CñPçPÿCïPëCçPçP«P,,Pç / test _r e d i s _l i v e . c p p</code>
Емпирична оценка	пропускателна способност при симулирана фаза на изчакване и изолирани измервания на режийните разходи на корутините	<code>CñCïPëPÿP,,PçCñPçPñPçP,,CñPçCñCñs / b e n c h _PëCñPçP,,CñCïP«P,,PçP,, . c p p</code>

Таблица 15 показва, че доказателствената база е хетерогенна по необходимост. Една библиотека за обектно-релационно съпоставяне по време на компилация не може да бъде защитена само с числа за пропускателна способност, както и асинхронна архитектура не може да бъде защитена само с проверки на концепти. Изискват се и двата вида доказателства.

8.2. Каталог на модулните и интеграционните доказателства

За да не остане стратегията за верификация само на равнище обща рамка, тя трябва да бъде разгърната и като конкретен каталог от артефакти в хранилището. Това е особено важно за настоящата работа, защото архитектурните претенции са разпределени между слоя на същностите, междинното представяне на заявките, договора на конектора, асинхронната инфраструктура и проверките на целевите системи на живо. Следващите таблици правят именно това: показват къде в хранилището са разположени основните доказателства и каква роля играе всяка тестова група.

Таблица 16. Основни модулни тестове за общите, основните и асинхронните подсистеми

Файл	Подсистема	Какво валидира
test_types.cpp	базови псевдоними на типове	коректност на публичните псевдоними и базови помощни зависимости
test_property.cpp	слой за свойства на същностите	модел по време на компилация на скаларните полета и поведение на характеристиките на типовете
test_relationship.cpp	слой за връзки на същностите	логика за извеждане (извеждане) на връзките и поведение на стратегията за съхранение
test_PqPqCSPŸPqPq._cpp	междинно представяне на заявките и изразителен програмен интерфейс (изразителен API)	коректност на select, insert, update, delete и производните структури на заявки
test_schema_migration.cpp	миграционен слой	генериране на DDL, логика за разлики и поведение на автомат с крайни състояния
test_P,,PqCİPqPq_safety.cpp	помощници за конкурентност	поведение на пула от връзки, достъп, локален за нишката, и механизма на предпазителя на транзакцията
test_task.cpp	абстракция за корутинна задача	жизнен цикъл на Task<T> и базова семантика на изчакване/корутини
test_P,,PqCİPqPq_pool.cpp	планиране на работни нишки	публикуване, планиране и координация на ресурсите на работните нишки
test_io_context.cpp	асинхронна оркестрация	цикъл за събития и свързване на продълженията за асинхронния слой за изпълнение
test_PqCİPqPq,,CİCİPqPq,,PqCİPqPq_PqPqPq,,PqPqCİPqPqCİPq._cpp	фасада на асинхронния конектор	свързваща (bridge) логика между async_db<DB> и куки на конектора, съобразени с корутини
test_wire_protocol.cpp	експериментален слой за изпълнение	ниско ниво на механизми по време на компилация / ориентирани към мрежовия протокол
test_cpp26	бъдеща езикова	ранен път за автоматизация, базирана на

Таблица 16 е важна, защото показва, че модулният слой не се изчерпва с проверки, подобни на синтактичен анализатор на заявки. Той покрива едновременно логическия модел, механиката на изпълнение, миграциите и инфраструктурата, съобразена с корутини, върху която стъпва асинхронната архитектура.

Таблица 17. Модулни тестове, ориентирани към целевите системи, и техният фокус върху договора

Файл	Целева система	Тип доказателство
test_postgresql_ _РәР«Р,,РхРәСЪР«СТ'. cpp	PostgreSQLDB	съответствие (compliance) <code>с i s _ c o n n e c t o r</code> , утвърждавания на възможностите и рендиране на SQL параметри
test_mysql _РәР«Р,,РхРәСЪР«СТ'. cpp	MySQLDB	договор за съединявания/транзакции, диалектно специфично рендиране и очаквания за свързване (свързване)
test_mongodb _РәР«Р,,РхРәСЪР«СТ'. cpp	MongoDB	негативно валидиране на възможности и рендиране на филтри, подобни на BSON
test_redis _РәР«Р,,РхРәСЪР«СТ'. cpp	RedisDB	отсъствие на съединявания/транзакции/агрегация и материализация на команди ключ-стойност
test_cassandra _РәР«Р,,РхРәСЪР«СТ'. cpp	CassandraDB	ограничения на възможностите и корелати за рендиране на CQL
test_neo4j _РәР«Р,,РхРәСЪР«СТ'. cpp	Neo4jDB	възможност за транзакции и материализация на заявки, ориентирани към графи

Тези модулни тестове, ориентирани към целевите системи, са критични, защото доказват не само позитивни възможности, но и честно декларирани ограничения. При библиотека с множество свързващи компоненти отрицателната верификация — че дадена система *не* поддържа дадена операция — е също толкова важна, колкото и позитивната.

Таблица 18. Каталог на интеграционните тестове и тестовете на живо

Файл	Режим на изпълнение	Какво демонстрира
test_mockdb.cpp	винаги налична интеграционна базова линия	композиция на заявки, свързване на параметри, подготвени заявки и CRUD сценарии без външна целева система
test_sqlite.cpp	локален интеграционен тест	реален релационен път на изпълнение с утвърждавания на възможностите, съединявания, транзакции и попълване на резултати
test_postgresql_live.cpp	условен тест на живо	реална връзка, жизнен цикъл на таблици и изпълнение на CRUD върху PostgreSQL
test_mysql_live.cpp	условен тест на живо	реален път на MySQL драйвера, вмъкване/обновяване/изтриване и филтриране при избор
test_mongodb_live.cpp	условен тест на живо	изпълнение на сценарии, ориентирани към документи, срещу жива целева система
test_redis_live.cpp	условен тест на живо	материализация на ключ-стойност и хеш команди върху Redis среда
test_cassandra_live.cpp	условен тест на живо	изпълнение на широки колони и платформено-специфични CQL сценарии
test_neo4j_live.cpp	условен тест на живо	изпълнение на графови заявки през контролирана Docker среда

Таблица 18 показва, че доказателствата, базирани на хранилището, са разслоени. Не всички целеви системи имат еднаква експлоатационна цена за верификация на живо, но хранилището съдържа ясна стратегия как и кога подобни тестове се активират. Това е особено важно за тезата, защото избягва смесването между коректност на имитациите и реално поведение, подкрепено от драйвер.

8.3. Карта на зрелостта според доказателствената опора

Самият профил на възможностите не е достатъчен, ако липсват доказателства за реално поведение. Затова е полезно целевите системи да се разгледат и като стълбица на зрелост, определена не само от поддържаните операции, но и от наличната тестова опора. Тук зрелостта означава комбинация от коректност на договора, доказателства за изпълнение и експлоатационен реализъм.

Таблица 19. Свързващ компонент maturity карта според наличната доказателствена опора

Свързващ компонент	Ниво на зрелост	Основна доказателствена опора	Интерпретация
MockDB	ниво 3	test_mockdb.cpp	свързващ компонент-independent изпълнение базова линия за заявка договор-а и prepared заявка механиката
SQLite	ниво 3 към 4	test_sqlite.cpp	локален реален изпълнение path с богата заявка evidence и възможност asserts
PostgreSQL	ниво 3	test_postgresql_live.cpp	силен договор плюс условна live свързващ компонент проверка
MySQL	ниво 3	test_mysql_live.cpp	реален експлоатационни path с релационен профил и dialect-specific свързване
MongoDB	ниво 3	test_mongodb.cpp	document-oriented изпълнение evidence без SQL-style възможности
Redis	ниво 3	test_redis_live.cpp	key-value изпълнение evidence с ограничен, но честен договор профил
Neo4j	ниво 3	test_neo4j_live.cpp	graph изпълнение path с Docker-gated live verification
Cassandra	ниво 3	test_cassandra_live.cpp	широки колонни path с фокус върху формална пригодност и live coverage

Картата в Таблица 19 е важна, защото предотвратява прекалено бинарно четене на свързващ компонент статуса. Конекторът не е просто наличен или липсващ; той може да бъде коректен по договор, валидиран при изпълнение, частично верифициран на живо или съобразен със схемата богат. Именно този по-нюансиран прочит прави верификационната глава по-научно убедителна и по-близка до реалния статус на хранилището.

8.4. Методология на сравнителен тест оценката

Емпиричната оценка е реализирана чрез Google Сравнителен тест [20] в `src/test/scala/com/google/benchmark/BenchmarkRunner.scala`. Тя не цели да симулира пълния жизнен цикъл на конкретна база данни, а да изолира архитектурния въпрос дали базираното на корутини изпълнение освобождава работните нишки по-ефективно от синхронното блокиране, когато между две кратки CPU фази има интервал на изчакване.

Сравнителен тест наборът е разделен на две групи. Първата група измерва пропускателна способност на цели партии от задачи. Синхронният базова линия изпълнява CPU работа, след което работна нишката блокира за предварително зададен интервал. Асинхронният вариант използва `SimulatedAsyncIO` изчакваем обект, който спира корутината, планира отложено възобновяване чрез опашка от таймери и връща продължаването обратно към `thread_pool`. Втората група измерва изолирани режимни разходи: създаване и изчакване на `Task<int>`, `thread_pool::post()` двупосочен път и вложени `co_await` вериги.

Методологично е важно, че сравнителните тестове за пропускателна способност използват реално измерване, а не чисто CPU време. Причината е изрично отбелязана и в самия код на сравнителния тест: при корутинни сценарии с фаза на изчакване калибрирането по CPU време на Mac OS може да даде подвеждащо поведение, защото спряната задача почти не консумира CPU. Затова съпоставката се извършва чрез `UseRealTime()` и фиксиран брой итерации.

В настоящата глава се използват режим на издание резултатите от локалния `build-rel` дърво за изграждане. Те не трябва да се интерпретират като абсолютни производствени числа за всички възможни машини и драйвери. Тяхната роля е сравнителна: всички варианти се изпълняват върху една и съща машина, с една и съща параметрична мрежа, така че да се изведе относителният ефект от различния изпълнение

МОДЕЛ.

Таблица 20. Семейства сравнителни тестове и тяхната цел

Семейство сравнителни тестове	Какво измерва	Основни параметри
BM_SyncThroughput	пропускателна способност на блокираща базова линия, при която работната нишка остава заета по време на фазата на изчакване	брой нишки, брой задачи, латентност на изчакване, CPU итерации
BM_AsyncThroughput	пропускателна способност на корутинен вариант със спиране/възобновяване и отложено възобновяване чрез опашка от таймери	брой нишки, брой задачи, латентност на изчакване, CPU итерации
BM_AsyncYieldThroughput	цена на редуване на корутини (корутина <code>interleave</code>) без изкуствена латентност на изчакване	брой нишки, брой задачи, CPU итерации
BM_TaskCreateAndWait	минимална цена на създаване и завършване на тривиална задача	единична корутина без външно планиране
BM_ThreadPoolPost	цена на <code>post()</code> и двупосочен път (двупосочен път) на обещание/бъдеща стойност към работна нишка	размер на пула от нишки
BM_NestedCoAwait	режийни разходи на вложено <code>co_await</code> продължаване при различна дълбочина	дълбочина на корутинната верига

8.5. Основни резултати за пропускателна способност

Таблица 21 обобщава представителните резултати в режим на издание за най-информативните конфигурации. Показани са двойки, за които има директно сравнение между синхронния и асинхронния вариант при едни и същи параметри.

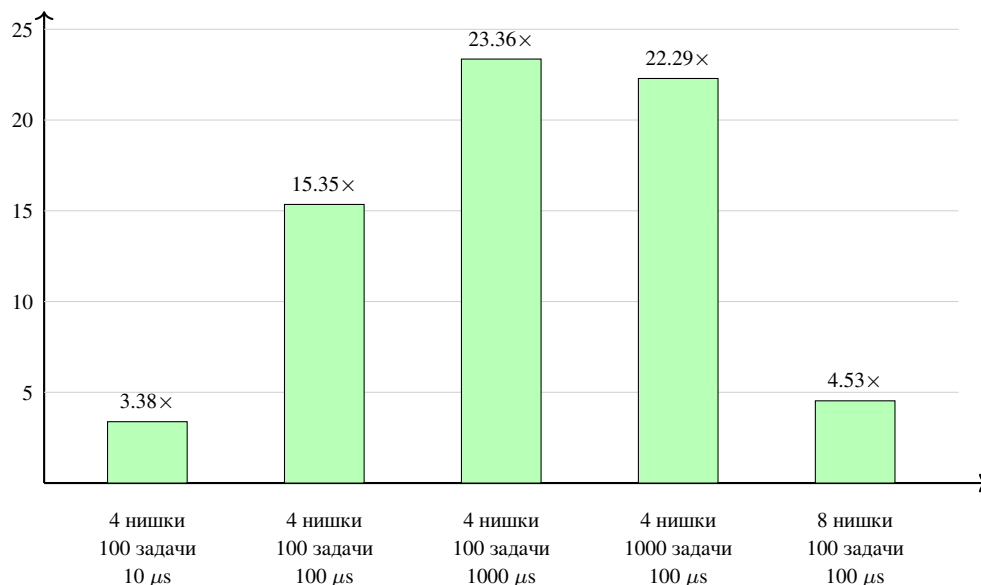
Таблица 21. Представителни резултати за пропускателната способност в режим на издание (режим на издание) за синхронно и асинхронно изпълнение

Нишки	Задачи	Латентност на изчакване	Синхронна	Асинхронна	Ускорение
			пропускателна способност	пропускателна способност	
4	100	10 μs	194.6k ops/s	658.5k ops/s	3.38×
4	100	100 μs	29.8k ops/s	458.2k ops/s	15.35×
4	100	1000 μs	3.22k ops/s	75.28k ops/s	23.36×
4	1000	100 μs	30.08k ops/s	670.5k ops/s	22.29×
8	100	100 μs	57.13k ops/s	258.5k ops/s	4.53×

Най-важният извод е, че предимството на корутинния модел нараства не когато локалната CPU работа нарасне, а когато фазата на изчакване започне да доминира над нея. При 4 работни нишки и 100 задачи разликата между 10 и 1000 микросекунди (μs) латентност на изчакване увеличава относителното предимство на асинхронния вариант от 3.38× до 23.36×. Това е именно очакваното поведение: при синхронната базова линия нишките се изчерпват от блокиране, докато при корутинния модел те се освобождават за друга работа.

Вторият важен извод е, че ефектът не изчезва при по-големи партии. При 4 нишки, 1000 задачи и 100 μs латентност на изчакване асинхронният вариант достига 670.5k операции в секунда срещу 30.08k при синхронната базова линия. Това показва, че архитектурното предимство не е ограничено до „демо“ случай с малък брой задачи, а се запазва и когато конкуренцията за ресурси на работните нишки нарасне.

ускорение асинхронен/синхронен



Фигура 15. Относително ускорение на асинхронната пропускателна способност спрямо синхронната базова линия при представителни конфигурации, ограничени от вход-изход (I/O-bound)

Фигура 15 визуализира същата тенденция в относителна форма. Дори когато броят на работните нишки се удвои до 8, асинхронният вариант остава приблизително 4.5 $\times$ по-бърз при 100 μ s латентност на изчакване. Това е важен практически приложим резултат: повече нишки намаляват част от натиска върху синхронната базова линия, но не елиминират структурното предимство на модела на изпълнение със спиране/възобновяване.

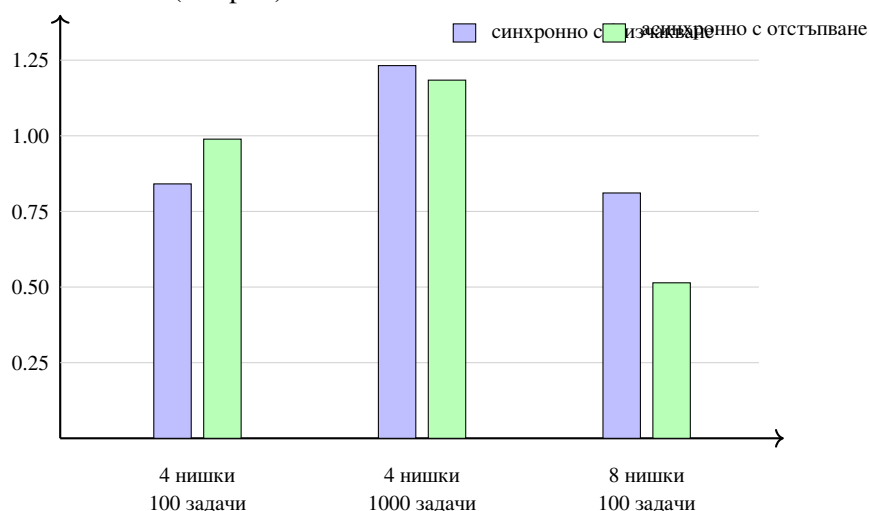
8.6. Нулева латентност на изчакване (0ns) и режимни разходи за планиране на корутините

При нулева изкуствена латентност на изчакване сравнението престава да измерва освобождаването на работните нишки от изчакване на вход-изход и започва да показва почти чистата цена на механиката за спиране/възобновяване. Поради това най-подходящата съпоставка, подкрепена от хранилището, комбинира `BM_SyncThroughput` с нулева стойност за изчакване и `BM_AsyncYieldThroughput`, който заменя базираното на таймер изчакване с еднократно прехвърляне (отстъпване) `PoolYield` обратно към `thread_pool`.

Таблица 22. Представителни резултати в режим на издание (режим на издание) при нулева изкуствена латентност на изчакване

Нишки	Задачи	Синхронна пропускателна способност	Пропускателна способност с отстъпване (асинхронен отстъпване)	Отношение асинхронен/синхронен
4	100	841.0k ops/s	989.4k ops/s	1.18×
4	1000	1.232M ops/s	1.184M ops/s	0.96×
8	100	810.9k ops/s	514.4k ops/s	0.63×

пропускателна способност (M ops/s)



Фигура 16. Съпоставка на пропускателната способност при нулева изкуствена латентност на изчакване

Таблица 22 и Фигура 16 показват, че при липса на фаза на изчакване корутинният слой вече не получава структурния бонус, наблюдаван в режима, ограничен от вход-изход (I/O-bound). При 4 нишки и 100 задачи вариантът само с отстъпване (само с отстъпване) все още е приблизително 1.18× по-бърз, но при 4 нишки и 1000 задачи той е практически паритетен със синхронната базова линия (0.96×), а при 8 нишки и 100 задачи пада до 0.63×. Това е полезен коректив към силните резултати при ограничение от вход-изход: без външно изчакване корутинният механизъм не „създава“ пропускателна способност, а добавя малка, но измерима цена за планиране, която може да бъде компенсирана само частично.

8.7. Интерпретация на микротестовите (microсравнителен tests) за режимни разходи

Изолираните измервания на режимните разходи (режимни разходи) показват, че корутинната инфраструктура сама по себе си не е доминиращият разход в сценариите, ограничени от вход-изход. При изграждане в режим на издание (издание) `BM_TaskCreateAndWait` измерва приблизително 71.6 ns за създаване и завършване на тривиална задача. `BM_ThreadPoolPost` остава около 5.0 μ s независимо дали пулт има 1, 2, 4 или 8 работни нишки. `BM_NestedCoAwait` достига около 324 ns при дълбочина 10 и около 2.375 μ s дори при дълбочина 100.

Тези числа не трябва да се четат като пряка обещана цена на всяка реална заявка, но са важни с друго. Те показват, че режимните разходи на корутинния механизъм са порядъци по-малки от фазите на изчакване от 100–1000 μ s, при които асинхронната пропускателна способност печели най-много. С други думи, наблюдаваното ускорение не е артефакт на измервателния инструмент, а следствие от това, че работните нишки престават да бъдат заключени в блокиращо изчакване.

8.8. Ограничения на емпиричната оценка

Емпиричната оценка съзнателно използва симулирана фаза на изчакване, а не пълно (от край до край) изпълнение срещу конкретен мрежов сървър за база данни. Това е ограничение, ако целта е да се предскаже абсолютна пропускателна способност за реална инсталация на PostgreSQL или Cassandra. Но то е и методологично предимство, ако целта е да се изолира моделът на изпълнение от влиянието на програмата за планиране на SQL (програма за планиране на SQL), мрежовите колебания, кеширането в драйвера и спецификите на ядрото за съхранение (ядро за съхранение).

Следователно настоящите резултати от сравнителните тестове не доказват, че „асинхронното е винаги по-бързо“. Те доказват по-тясна, но по-научно защитима теза: когато архитектурата работи в режим, ограничен от вход-изход (I/O-bound), и фазата на изчакване доминира над локалната изчислителна (CPU) работа, моделът, базиран на корутини, позволява съществено по-добро използване на ресурсите на работните нишки спрямо синхронната базова линия. Това е точно твърдението, което асинхронният слой на библиотеката трябва да обоснове.

В комбинация с тестовата верификация този резултат затваря двата основни въпроса на дипломната работа. От една страна библиотеката показва формална и тестово подкрепена дисциплина на коректност. От друга страна асинхронният модел на изпълнение показва измерима практически приложима полза именно в сценария, за който е проектиран. Така емпиричната оценка не стои встрани от архитектурата, а потвърждава нейния централен инженерен и научен смисъл.

IX. НАДГРАЖДАЕМОСТ ЧРЕЗ НОВИ КОНЕКТОРИ

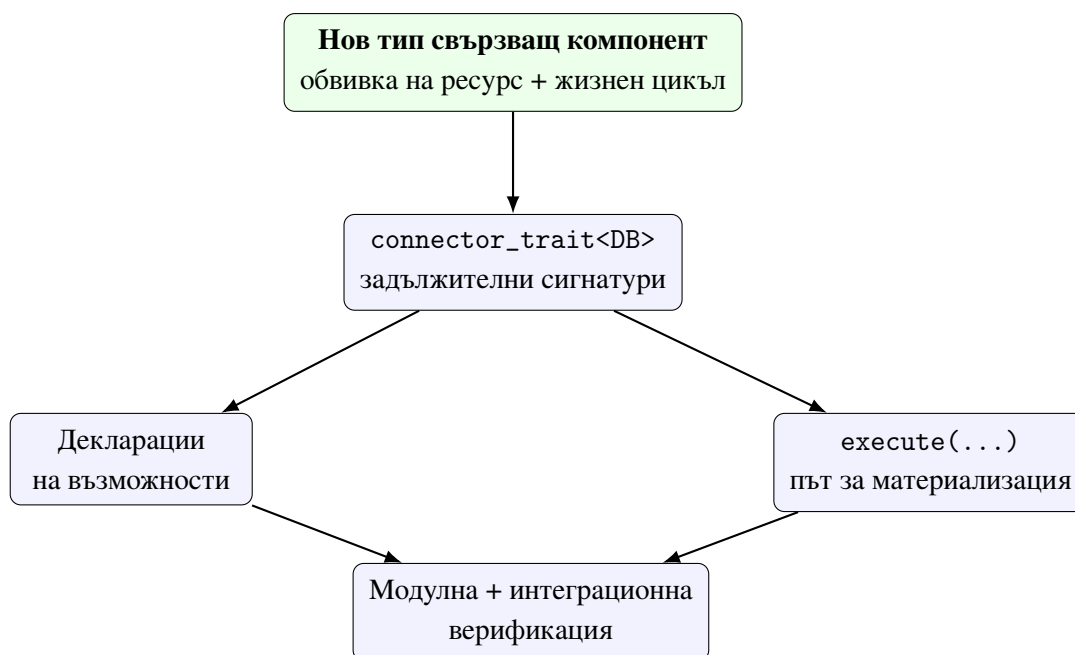
Една от централните претенции на разработката е, че библиотеката е надграждаема без компромис с нейните гаранции по време на компилация. Това изисква не просто наличие на тестове, а ясно дефиниран договор на конектора (конектор договор), който позволява нов свързващ компонент да бъде добавен без промяна в програмния интерфейс за заявки (заявка API) и без разпадане на типовата дисциплина, ограниченията на възможностите и архитектурните инварианти.

В предходните глави беше показано, че библиотеката вече работи с релационни и нерелационни компоненти, както и че асинхронният слой може да бъде реализиран по различен начин според платформено-специфичните възможности на конкретния драйвер. Настоящата глава формализира как това е възможно и как трябва да се добавя следващ свързващ компонент. С други думи, тук надграждаемостта престава да бъде общо качество на дизайна и се превръща в методология с конкретни изисквания, сигнатури (сигнатури), декларации на възможности, асинхронни задължения (асинхронни задължения) и критерии за завършеност.

9.1. Формален договор на конекторния слой

Конекторът се описва чрез тип `DB` и специализация `connector_trait<DB>`. Това е структурен, а не наследствен договор. Няма общ виртуален интерфейс `Connector`. Причината е принципна: различните компоненти имат различни възможности и ограничения, които трябва да бъдат видими по време на компилация. Виртуалната йерархия би изравнила твърде много различия и би прехвърлила част от проверките към времето за изпълнение.

Минималният работещ конектор трябва да осигури `wire_type`, `cursor_type` и подходящи входни точки (входни точки) за `execute(...)`. Допълнителните способности се означават чрез `label`и за възможности. Така договорът остава едновременно строг и разширяем. В тази конструкция няма “магически” регистрационен слой: компилаторът вижда специализацията директно и може да провери дали тя удовлетворява концепта (концепт) `is_connector`.



Фигура 17. Архитектурен работен процес (workflow) за добавяне на нов конектор

Фигура 17 показва, че добавянето на компонент не е едноетапно действие. То започва с ясно дефинирана обвивка на ресурс (обвивка на ресурс), продължава с договорена характеристика (договор на характеристика), профил на възможности и материализация на изпълнението, и завършва едва след тестова верификация. Това е съществено за дипломната работа, защото поставя надграждаемостта в дисциплиниран инженерен процес.

9.2. Задължителни елементи на минимален конектор

На практика един свързващ компонент се счита за минимално интегриран, ако:

1. съществува тип, който представлява обвивка на връзката на компонента;
2. съществува специализация `connector_trait<DB>` за този тип;
3. специализацията дефинира `template<typenameT>structwire_type;`
4. специализацията дефинира `cursor_type;`
5. специализацията може да изпълнява поне базовите OPC форми на заявки, които твърди, че поддържа.

Това означава, че “конектор” не е просто рендерираща функция, а пълно описание на начина, по който C++ типовете и междинното представяне преминават към конкретната целева система. Ако липсва дори един от тези елементи, добавеният компонент не е архитектурно пълноценен, независимо дали част от заявките могат да бъдат изкуствено изпълнени.

Таблица 23. Задължителни и опционални елементи на конектора

Елемент	Роля
Тип DB	обвивка над манипулатор на компонента, сесия или имитираща връзка
<code>connector_trait<DB></code>	централна специализация на договора
<code>template<typename T>\new linestruct\wire_type</code>	преобразува C++ тип към мрежово (wire) представяне
<code>cursor_type</code>	описва начина на обхождане на резултатите или техния жизнен цикъл
Претоварвания на <code>execute(. ..)</code>	материализират междинното представяне към поведение на компонента
Етикети за възможности	ограничават допустимите операции и политиките за жизнен цикъл
<code>begin/commit/rollback</code>	необходими при поддръжка на транзакции
<code>ddl_for(...)</code>	необходимо при интеграция, съобразена със схемата (съобразен със схемата), и <code>migrate<DB></code>
<code>render_constexpr(...)</code>	опционална кука (hook) за материализация по време на компилация за специализирани сценарии

9.3. Типизиране, сигнатури и именуване

Договорът на конектора е строг и по отношение на типовете. `wire_type<T>::ty` ре трябва да е стабилна функция по време на компилация на T. `cursor_type` трябва да моделира реалистично итериране на резултатите или поне ясно да обозначава имитиращо поведение. Претоварванията на `execute(...)` трябва да приемат DB& като първи аргумент

и да връщат `orm::result<...>` за SELECT форми или `result<std::tuple<>>` за операции за запис.

Именуването на конекторите в хранилището показва последователна конвенция. “Локални” или удобни за модулно тестване (удобни за модулно тестване) конектори използват имена като SQLiteDB, PostgreSQLDB, MySQLDB, MongoDB, RedisDB, Neo4jDB и CassandraDB. Реалните варианти на живо се разграничават експлицитно чрез суфикса `LiveDB`. Това не е дребен стилев избор, а част от яснотата на договора: още от името е ясно дали даден тип е адаптер за имитация (адаптер за имитация), локална/в паметта обвивка или реална целева система, подкрепена от драйвер.

Липсата на наследяване също трябва да се разглежда като формално правило, а не като детайл от реализацията. Нов конектор не трябва да наследява “обща OPC база”, защото това би смесило специфичната за компонента логика с полиморфизъм по време на изпълнение. Правилният механизъм за разширение е специализацията на `connector_trait<DB>`.

9.4. Механизъм на възможностите и ограничения по време на компилация

Етикетите за възможности са основният начин библиотеката да различава компонентите по допустими операции. Ако `connector_trait<DB>` не декларира `supports_joins`, тогава опит за заявка, базирана на съединяване, е архитектурно неправилен и следва да бъде отхвърлен още при компилация. Същото важи за транзакции, агрегация и конкурентно изпълнение.

От гледна точка на верификацията това е значимо, защото коректността вече не зависи само от тестовете. Част от договора е проверим директно от компилатора. Тестовете тогава не доказват съществуването на договора, а че конкретните специализации го прилагат последователно. Именно по тази логика `tests/PäP«PŸČĤPŦP,,Pč/test_mongodb_PəP«P,,PŧPəČĤP«CĬ'.cpp` и `tests/PäP«PŸČĤPŦP,,Pč/test_redis_PəP«P,,PŧPəČĤP«CĬ'.cpp` валидират отсъствието на `supports_joins`, `supports_transactions` и `supports_aggregation` при съответните конектори.

Тази негативна верификация е също толкова важна, колкото и позитивната. Да докажеш, че компонент *не* поддържа дадена операция, е толкова съществено, колкото да докажеш, че поддържа друга.

9.5. Минимален и експлоатационен скелет на конектора

В хранилището MockDB и MockMigrateDB са особено ценни, защото играят ролята на архетипове на конектори. Първият показва експлоатационния договор за изпълнение на заявки, а вторият — как същият договор може да бъде разширен към миграционни сценарии, съобразени със схемата.

```
1  template <>
2  struct конектор_trait<MockDB>
3  {
4      using supports_joins          = void;
5      using supports_transactions    = void;
6      using supports_aggregation     = void;
7      using supports_upsert          = void;
8      using supports_bulk_insert     = void;
9      using supports_concurrent_execute = void;
10
11     template <typename T>
12     struct wire_type { using type = T; };
13
14     struct cursor_type
15     {
16         [[nodiscard]] bool has_next() const noexcept { return false; }
17     };
18
19     template <typename Response, typename Joins, typename Wheres,
20              typename Limits, typename Groups, typename Orders>
21     static auto execute(MockDB& db,
22                         select_заявка<Response, Joins, Wheres, Limits, Groups, Orders> q)
23         -> result<projected_type<Response>, Response>;
24 };
```

Listing 11: Извадка от connector_trait<MockDB>

Listing 11 е показателен, защото събира в кратка форма почти всички задължителни елементи: профил на възможностите, wire_type, cursor_type и специфични за заявките сигнатури за execute(...). Освен това конекторът използва отделни претоварвания за select_query, insert_query, update_query и delete_query, което прави съпоставянето с поведението на целевата система експлицитно.

```
1  template <>
2  struct конектор_trait<MockMigrateDB>
3  {
4      using supports_joins          = void;
5      using supports_transactions    = void;
6      using supports_aggregation     = void;
```

```

7      using supports_concurrent_execute = void;
8
9      template <typename T>
10     struct wire_type { using type = T; };
11
12     template <typename... Args>
13     static auto execute(MockMigrateDB& db, Args&&... args)
14     {
15         return конектор_trait<MockDB>::execute(
16             static_cast<MockDB&>(db), std::forward<Args>(args)...);
17     }
18
19     [[nodiscard]] static std::string ddl_for(const create_table_op& op);
20     [[nodiscard]] static std::string ddl_for(const add_column_op& op);
21 };

```

Listing 12: Извадка от разширението MockMigrateDB, съобразено със схемата

Listing 12 показва как експлоатационният скелет на конектора може да бъде надграден. Тук логиката за изпълнение на заявките се делегира към MockDB, но се добавят DDL куки и куки за транзакции. Това е добър пример за модел на прогресивна зрялост на конектора.

9.6. Транзакции, нишкова безопасност и подготвени заявки

Пълноценният договор на целевата система не се изчерпва с единично изпълнение на SELECT. Ако конекторът декларира `supports_transactions`, от него се очаква да поддържа `begin`, `commit` и `rollback`, така че `transaction_guard<DB>` да бъде валиден. Ако декларира `supports_concurrent_execute`, той трябва да може да участва безопасно в `connection_pool<DB,N>`.

Слоят за подготвени заявки също поставя изискване за стабилност на жизнения цикъл на DB& и за консистентно поведение при свързване при многократно изпълнение. Следователно зрелият конектор трябва да бъде оценяван не само по това дали рендира правилен синтаксис, а и по експлоатационния договор около ресурси, транзакции и повторна употреба. Това е особено важно за компоненти, които имат по-сложен жизнен цикъл на драйвера от MockDB.

9.7. Миграции и интеграция, съобразена със схемата

Конектор, който участва в `migrate<DB>`, трябва да предложи и точки за разширение на DDL. В хранилището това е демонстрирано чрез `MockMigrateDB`. Този пример е особено полезен, защото показва как върху вече съществуваща целева система за изпълнение може да се надгради поведение, съобразено със схемата. По този начин библиотеката формализира различни нива на завършеност на конектора.

Тук е важно да се отбележи, че поддръжката, съобразена със схемата, не е задължителна за всяка целева система. Някои конектори могат да бъдат напълно адекватни за анализ на изпълнението на заявките, без да поддържат автоматизирани миграции. Но ако конекторът претендира за по-пълна OPC интеграция, наличието на `ddl_for(...)` куки и миграционни тестове вече става задължително.



Фигура 18. Стълба на зрелостта за нов конектор

Фигура 18 предлага практичен модел за оценка. Тя е полезна и за самото бъдещо развитие на библиотеката, защото позволява новите компоненти да бъдат позиционирани не бинарно като “има/няма конектор”, а според нивото на тяхната интеграция.

9.8. Методика за добавяне на нов свързващ компонент

Добавянето на нов компонент следва ясна последователност. Първо се дефинира типът на целевата система и неговата обвивка на ресурс. След това се реализира `connector_trait<DB>` със задължителните елементи. Следват декларациите на възможности и правилата за рендиране или изпълнение на междинното представяне. Накрая се добавят модулни тестове за договора и, при възможност, интеграционни тестове на живо за реалния

драйвер.

Архитектурно правилният ред е важен: не се започва от заобиколни решения по време на изпълнение, а от формализиране на границите по време на компилация и на логиката на съпоставяне. Ако този ред бъде нарушен, резултатът обикновено е конектор, който “работи” за единичен идеален сценарий (идеален сценарий), но не е съвместим с общата архитектура.

Таблица 24. Препоръчителна матрица за верификация за нов конектор

Пласт на верификация	Какво трябва да се докаже	Примерен корелат в хранилището
Съответствие с концепт	is_connector<DB> е удовлетворен	tests/PëP«PŸCñPñP,,Pç/test_mongodb_PəP«P,,PçPəCñP«CŸ'.cpp
Честност на възможностите	декларираните и недеклаираните labelи за възможности са правилни	tests/PëP«PŸCñPñP,,Pç/test_red_is_PəP«P,,PçPəCñP«CŸ'.cpp, tests/PçP,,CñPçPŸCŸPəCçPçP«P,,P,,Pç/test_sqlite.cpp
Коректност на изпълнението	междинното представяне се материализира коректно	tests/PçP,,CñPçPŸCŸPəCçPçP«P,,P,,Pç/test_moc_kdb.cpp и тестовете на живо
Коректност на ресурси/жизнен цикъл	куките за транзакции и конкурентност се държат коректно	tests/PëP«PŸCñPñP,,Pç/test_P,,PçCŸPəPə_safe_ty.cpp
Коректност, съобразена със схемата	DDL куките и миграционните разлики работят коректно	tests/PëP«PŸCñPñP,,Pç/test_schema_migration.cpp

9.9. Критерии за „напълно имплементиран и функциониращ“ конектор

На базата на кода и тестовете в хранилището могат да се формулират следните критерии:

1. конекторът удовлетворява договора `is_connector`;
2. декларациите на възможности съответстват на реалното поведение;
3. конвенциите за именуване и типизиране са съвместими с останалите конектори;
4. ресурсният жизнен цикъл е коректен и проверим;
5. поддържаните OPC операции са покрити от модулни тестове;
6. при наличен драйвер на живо има интеграционни тестове срещу реална база данни;
7. при заявена поддръжка, съобразена със схемата, са налични DDL куки и миграционни тестове.

Тези критерии са по-строги от обикновено “има работещ адаптер”, защото изискват консистентност между архитектурно намерение, договорен програмен интерфейс и верифицирано поведение. Именно това различава непланираната (непланирана) интеграция от конектор, който наистина принадлежи към библиотеката.

9.10. Верификация чрез тестове

Тестовият слой в хранилището следва структурата на архитектурата. Модулните тестове за отделните конектори валидират съответствието с `is_connector`, декларациите на възможности и основното рендиране. `tests/PèP«PÿCтРъP,,Pç/test_P,,PçCİPəPə_safety.cpp` проверява `connection_pool`, `thread_local_db` и `transaction_guard`. `tests/PèP«PÿCтРъP,,Pç/test_schema_migration.cpp` валидира логиката за разлики в миграциите. `tests/PçP,,CтРхPÿCİPəCЗPçP«P,,Pç/test_mockdb.cpp` и интеграционните тестове на живо за отделни системи доказват, че абстракцията работи и при реално изпълнение.

Следователно верификацията не е концентрирана само на едно място. Тя е разпределена така, че всяко архитектурно твърдение да има поне един пряк тестов

корелат. Това превръща разширяемостта на конекторите от декларативна цел в проверяема инженерна практика.

9.11. Оценка на наличните конектори

SQLiteDB може да се разглежда като референтна зряла реализация за релационна целева система. PostgreSQLDB, MySQLDB, MongoDB, RedisDB, Neo4jDB и CassandraDB вече демонстрират важни части от договора и имат различна степен на покритие на живо. MockDB и MockMigrateDB играят особена роля като верификационни конектори: те не са само тестови двойници, а средство за формализиране на правилата на архитектурата.

Оттук следва и по-общият извод на настоящата глава: надграждаемостта на библиотеката не е добавена впоследствие, а е заложена в самия начин, по който е дефиниран конекторният слой. Именно това позволява разработката да бъде продължавана с нови свързващи компоненти, без да се нарушава формалният характер на OPC архитектурата по време на компилация.

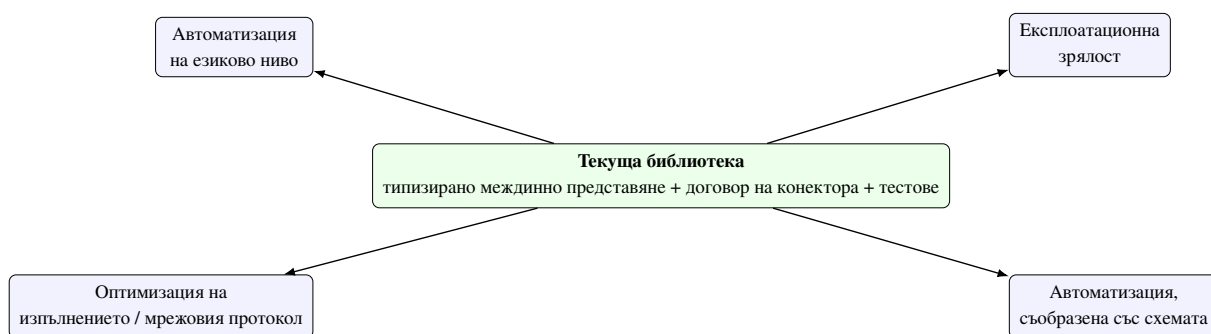
Х. ОГРАНИЧЕНИЯ И БЪДЕЩО РАЗВИТИЕ

Текущата версия на библиотеката вече демонстрира работеща архитектура за библиотека за обектно-релационно съпоставяне по време на компилация върху релационни и нерелационни свързващи компоненти, но научната коректност изисква ясно разграничение между вече постигнатото и следващите стъпки. Библиотеката съдържа реален слой за междинно представяне на заявките, базиран на възможности конекторен договор, миграционен прототип, помощници (помощници) за нишкова безопасност и набор от модулни и интеграционни тестове. Въпреки това част от подсистемите все още трябва да бъдат разширени или валидирани по-задълбочено в условия, близки до производствените (условия, близки до производствените).

Бъдещото развитие не е произволен списък от идеи. То следва непосредствено от архитектурните решения, анализирани в предходните глави. Ако C++ моделът е първичен носител на логическата схема, следваща естествена стъпка е по-пълна автоматизация на инструментариума, съобразен със схемата (инструментариум, съобразен със схемата). Ако конекторният слой е централна точка на разширяемост (разширяемост), тогава бъдещата работа трябва да уточни по-фини класове от възможности. Ако междинното представяне е реално споделян слой между различни свързващи компоненти, тогава то трябва да бъде изпитано при по-богати експлоатационни сценарии и ограничения на производителността.

10.1. Стратегически направления за развитие

От гледна точка на настоящата дипломна работа бъдещото развитие може да бъде групирено в четири стратегически направления. Първото е *по-голяма експлоатационна зрялост* на вече наличните целеви системи. Второто е *по-пълна автоматизация на схемата и метаданните*. Третото е *намаляване на режимните разходи по време на изпълнение (по време на изпълнение режимни разходи)* чрез по-ниско ниво на изпълнение и по-добър контрол върху слоя на мрежовия протокол (мрежов протокол). Четвъртото е *намаляване на шаблонния код (boilerplate)* чрез по-дълбока интеграция с новите езикови механизми на C++.



Фигура 19. Четири стратегически направления за бъдещо развитие

Фигура 19 е полезна, защото показва, че бъдещата работа не е еднопластова. Част от задачите са насочени към експлоатационна зрялост, други — към по-добра автоматизация по време на компилация, а трети — към инженеринг на производителността и времето за изпълнение. Това разграничение е важно, защото различните направления изискват различни критерии за успех.

10.2. Разширяване на покритието на целеви системи на живо

Макар хранилището вече да съдържа интеграционни тестове на живо за няколко системи, зрелостта на отделните реализации не е еднаква. SQLite остава най-естественият референтен релационен път, защото съчетава най-плътно свързани рендиране на заявки, логика за свързване (свързване) и попълване на резултати (попълване на резултати). При PostgreSQL, MySQL, MongoDB, Redis, Neo4j и Cassandra следващата стъпка е по-широко покриване на експлоатационни сценарии: по-богати типове, диагностични пътища при грешка, политики за жизнен цикъл на връзките и оценка на поведението при по-сложни заявки.

Това направление е важно не само като инженерно подобрене, а и като научна проверка на границите на общото междинно представяне. Само при по-широко емпирично покритие може да се оцени доколко един и същ логически слой остава адекватен за различни типове системи. Например логика, базирана на съединявания, която е естествена за PostgreSQL, има съвсем различен експлоатационен смисъл в Neo4j или Cassandra. Следователно покритието на живо не е просто тестова дисциплина, а средство за изследване на архитектурната приложимост.

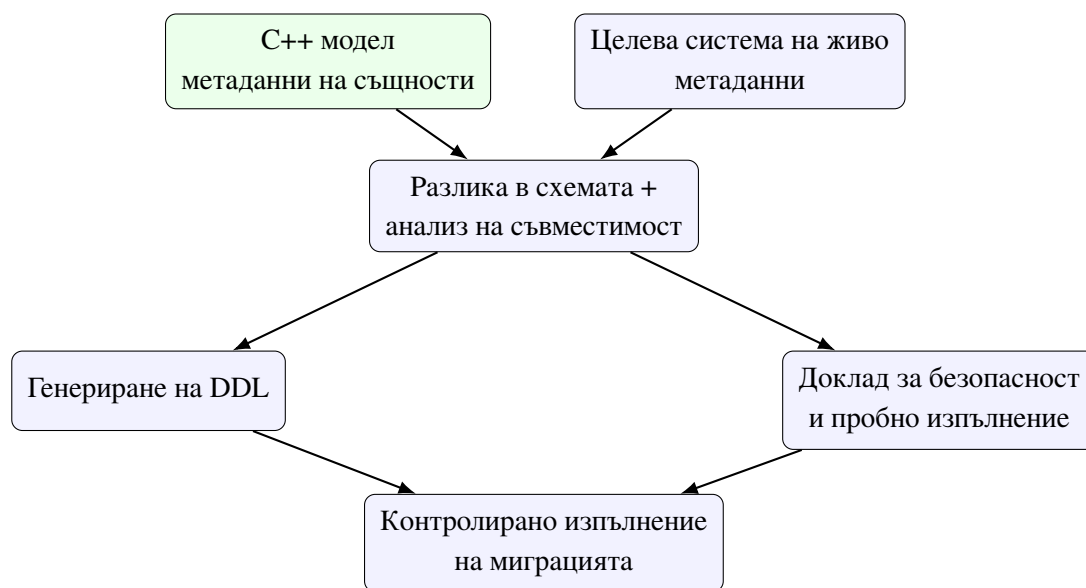
Допълнително, покритието на живо трябва да обхване и негативните пътища: грешки при установяване на връзка, частично невалидни параметри, специфични за

драйвера сценарии за изтичане на времето (изтичане на времето) и поведение при прекъсване на връзката. Тези въпроси почти винаги са подценени в ранните OPC библиотеки, но в контекст с множество свързващи компоненти те са решаващи за реалната употреба.

10.3. Пълна интеграция, съобразена със схемата, и интроспекция по време на изпълнение

Прототипният миграционен слой е достатъчен, за да покаже как C++ моделът на същността може да участва в описване на еволюцията на схемата. Следващото естествено направление е интроспекция на живо на реалната схема, извличане на метаданни от драйверите и безопасно изпълнение на DDL операции. Това е особено важно за релационните целеви системи, но не е без значение и за документни или широки колонни системи, където понятието за схема може да бъде по-гъвкаво или частично.

По-нататъшната работа в тази посока трябва да изясни и границата между гаранции по време на компилация и факти по време на изпълнение за вече разположена система. Компиляторът може да валидира структурата на заявката, но не и реалното състояние на производствената схема без допълнителна информация по време на изпълнение. Следователно бъдещият миграционен слой трябва да комбинира два източника на истина: C++ описанието на същността и извлечената картина от метаданни на реалната база.



Фигура 20. Целева посока за инструментариум, съобразен със схемата, отвъд текущия миграционен прототип

Фигура 20 показва, че бъдещият миграционен конвейер трябва да бъде двупосочен: едновременно да чете от описанието по време на компилация и от живата система. Това е естествена следваща стъпка, ако библиотеката трябва да бъде използвана не само експериментално, но и като практичен инструмент за разработка.

10.4. Нишкова безопасност и жизнен цикъл на връзките

Съществуващите помощници (помощници) за пул от връзки, достъп, локален за нишката, и предпазител на транзакция показват, че библиотеката вече съдържа архитектурна основа за многонишкова употреба. Бъдещото развитие трябва да отговори на по-конкретни въпроси: как се дефинира `supports_concurrent_execute` за различните драйвери; кога една връзка е безопасна за споделяне; какви са правилата за повторно свързване (повторно свързване), затваряне и отложено освобождаване на ресурси.

Тук има и важен изследователски аспект: различните целеви системи имат различна семантика на конкурентността, а следователно обозначаването на възможности по време на компилация трябва да бъде достатъчно прецизно, за да не създава подвеждащи обещания. Напълно възможно е бъдещи версии на библиотеката да разделят общото `supports_concurrent_execute` на по-фини класове от възможности като безопасно паралелно четене, паралелно изпълнение с отделни манипулатори на връзки или строга политика за сериализация.

Практически това означава, че помощният слой за нишкова безопасност не трябва да остане изолиран модул. Напротив, той трябва да се превърне в архитектурно продължение на договора на конектора и да участва в пълната оценка на зрелостта на дадена целева система.

10.5. Ниско ниво на изпълнение и оптимизации на мрежовия протокол

Експерименталният слой в `WireProtocol/wire_protocol.hpp` показва възможни бъдещи посоки като изчакваеми обекти за `io_uring` / IOCP, `zero_copy_result`, `batch_insert` и `make_constexpr_sql`. Тези елементи все още не са готов за производство комуникационен стек, но очертават възможност библиотеката да се разшири отвъд класическия синхронен драйверен модел.

Особено перспективни са нулево-копийната обработка на резултати и генерирането на SQL по време на компилация за конектори, които го позволяват. Те биха приближили библиотеката до още по-ниски режимни разходи по време на изпълнение, без да се жертва типизираната архитектура. Това е важно, защото една честа критика към богато типизираните слоеве на абстракция е, че скриват прекомерен разход по време на изпълнение. Бъдещата работа трябва да покаже, че изразителността по време на компилация и ниските режимни разходи не са задължително противоположни цели.

Освен това темата за производителността не трябва да се разглежда само през латентност или пропускателна способност. Тя включва и разположението в паметта на резултатите, броя временни заделения на памет (заделения на памет), повторната употреба на артефакти на подготвени заявки и качеството на стратегиите за масово обработване (масово обработване) на ниво драйвер.

10.6. Интеграция с отражение (отражение) в C++26 и допълнителна автоматизация

Поддръжката на отражение в C++26 е вече концептуално подготвена чрез логика за откриване (откриване) и помощници, а `tests/ReflectionTest.cpp` и `test_cpp26_ReflectionTest.cpp` показва наличието на ранен път за верификация. Публичният програмен интерфейс обаче все още използва експлицитни низови буквални имена за `property<T, Name>`. Бъдещото развитие трябва да превърне пътя с отражение в стабилен публичен механизъм, който позволява автоматично извличане на имената на полетата, без това да нарушава обратната съвместимост.

Подобно развитие би намалило шаблонния код и би направило още по-видима идеята, че C++ моделът е първичният източник на логическата схема. От изследователска гледна точка то би било интересно и защото би доближило библиотеката до стил, ориентиран към домейна (ориентиран към модела), при който голяма част от информацията за метаданните се извлича автоматично, а не се описва повторно.

Тук обаче има и важна граница. Отражението не трябва да разрушава яснотата на договора по време на компилация. Ако автоматизацията намали прозрачността на типа или затрудни съобщенията за грешка, тя може да отслаби една от основните ценности на библиотеката. Следователно тази посока трябва да се развива внимателно.

10.7. Разширяване на договора на конектора и таксономията на възможностите

С добавянето на нови целеви системи ще стане необходимо по-прецизно разграничаване между различни класове възможности. Възможно е например бъдещи версии на библиотеката да разграничават отделно възможност за обхождане на графи, поддръжка на частично вграждане на документи, поддръжка на интроспекция на схемата, поддръжка на поточни резултати или изрична поддръжка на масово обработване (масово обработване). Тази посока следва естествено от текущата архитектура, базирана на характеристики.

По-нататъшната работа трябва да пази едно важно правило: новите възможности не бива да бъдат декоративни. Те трябва да носят реално значение по време на компилация и да ограничават недопустимите операции. Ако label за възможност няма архитектурно последствие, той не добавя стойност. Точно обратното — прави договора по-шумен и по-малко точен.

Таблица 25. Приоритетни направления за бъдещо развитие

Направление	Засегнати подсистеми	Очакван ефект	Основно предизвикателство
Зрялост на компонентите на живо	конектори, интеграционни тестове, пътища за грешки	по-висока практическа надеждност	различна експлоатационна семантика между компонентите
Автоматизация, съобразена със схемата	migrate<DB>, DDL куки, извличане на метаданни	по-близка до производствената ОРС интеграция	съчетаване на истина по време на компилация и по време на изпълнение
Оптимизация на изпълнението	слой на мрежовия протокол, подготвени заявки, масово обработване	по-ниски режимни разходи по време на изпълнение	запазване на типизираната абстракция
Моделиране, управлявано от отражение	слой на същностите, извличане на метаданни, ергономика на програмния интерфейс	по-малко шаблонен код и по-автоматичен подход, ориентиран към модела	баланс между автоматизация и прозрачност
Прецизиране на възможностите	connector_trait t<DB> и статични проверки	по-точен договор за нови свързващи компоненти	избягване на декоративни labels за възможности

10.8. Необходимост от по-строга емпирична оценка

Една бъдеща версия на работата следва да включва и по-систематичен слой за сравнителен анализ (сравнителен testing). Той трябва да сравнява не само отделни целеви системи, но и различни режими на изпълнение вътре в библиотеката — директно изпълнение, prepare() + повторна употреба, сценарии с масово вмъкване (масово

вмъкване) и различни форми на материализация на резултатите. Такъв сравнителен анализ би бил ценен както инженерно, така и научно, защото би показал реалната цена на различните архитектурни избори.

Наред с това, полезно би било да се добави и формализирана оценка на диагностиката на грешките. Една от силните страни на дизайна за ОРС по време на компилация е именно ранното локализиране на грешки. Следователно качеството на съобщенията за грешка и яснотата на пътищата за отказ по време на компилация също са валиден обект на изследване.

10.9. Обобщение

Бъдещото развитие на библиотеката не започва от празно място. Напротив, то стъпва върху вече работеща архитектура, която е достатъчно обща, за да поеме нови свързващи компоненти, и достатъчно строга, за да наложи формални ограничения. Именно това прави разработката подходяща както за продължаване като инженерна система, така и за по-нататъшни академични изследвания в областта на моделирането на достъп до данни по време на компилация.

Най-важното заключение от настоящата глава е, че бъдещата работа не трябва да се разбира като поправка на несъстоятелен прототип. Тя е естествено разгръщане на вече защитима архитектурна основа. Това е съществено различие: библиотеката е достигнала ниво, на което разширението е въпрос на дисциплинирано развитие, а не на концептуално преосмисляне.

XI. ЗАКЛЮЧЕНИЕ

Настоящата дипломна работа представи проектирането, реализацията и анализа на библиотека за обектно-релационно съпоставяне по време на компилация за C++, ориентирана към работа както с релационни, така и с нерелационни бази данни. В центъра на разработката стои идеята, че структурата на заявката, логическият модел на данните и съществена част от ограниченията върху употребата на целевите системи могат да бъдат описани и валидирани чрез типовата система на езика.

С това работата адресира един конкретен проблем от съвременната практика: разминаването между силно типизиран приложен код и хетерогенните модели за съхранение, които този код трябва да използва. Класическите ОРС решения са най-силни в рамките на релационния свят, а подходите, ориентирани изцяло към драйвера (ориентирани към драйвера), често прехвърлят твърде голяма част от отговорността за коректността към времето за изпълнение. Настоящата разработка показва, че между тези два полюса съществува жизнеспособна архитектурна позиция — ОРС слой по време на компилация, който е едновременно практичен, формално структуриран и достатъчно честен по отношение на различията между свързващите компоненти.

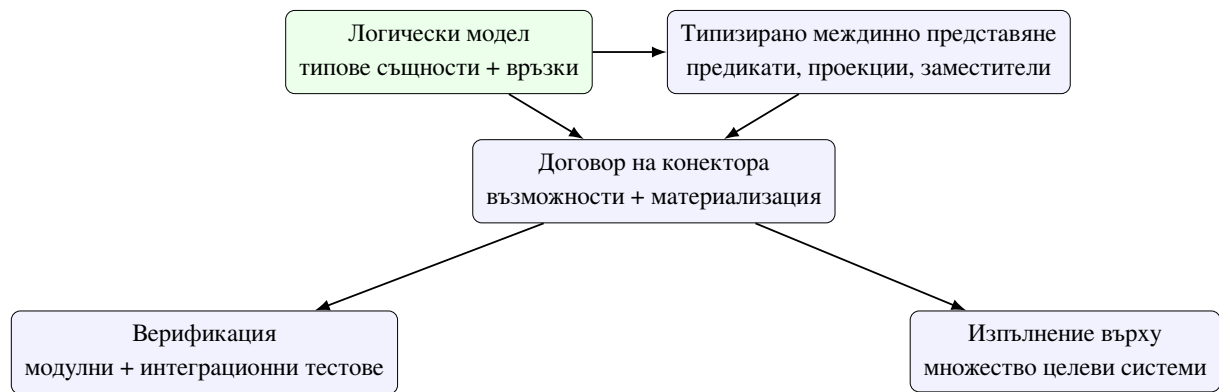
11.1. Обобщение на решената задача

Поставената задача не беше просто да се изгради още един удобен конструктор на заявки (заявка конструктор). Целта беше да се проектира библиотека, в която:

1. логическият модел на данните се описва чрез C++ типове;
2. заявките се представят като типизирано междинно представяне, а не като низове;
3. целевите системи се интегрират чрез формален договор на конектора;
4. различията между моделите за съхранение се управляват чрез механизми за възможности и ограничения;
5. коректността се подкрепя едновременно от проверки по време на компилация и от тестова инфраструктура.

В рамките на дипломната работа всяка от тези подцели беше адресирана с конкретни архитектурни решения и решения по реализацията. Сложат на същностите

въведе типизирани свойства и отношения, политика за съхранение чрез `store_as` и характеристики за именуване на логическите контейнери. Конструкторът на заявки беше организиран около типизирано междинно представяне. Конекторният слой беше формализиран чрез `connector_trait<DB>`, а стратегията за верификация бе подкрепена от модулни и интеграционни тестове за множество свързващи компоненти.



Фигура 21. Синтез на основната конструкция на разработената библиотека

Фигура 21 обобщава цялостната логика на дипломната работа. Тя показва, че библиотеката не е съвкупност от отделни техники, а последователна система, в която логическият модел, междинното представяне на заявките, договърът на конектора, верификацията и изпълнението върху множество компоненти се поддържат взаимно.

11.2. Обобщение на постигнатите резултати

Разработката показва, че C++ предоставя достатъчно силни механизми — шаблони, `constexpr`, концепти (`concepts`) и специализации на характеристики — за изграждане на ОРС слой, в който заявките не се описват като низове. Вместо това те се представят чрез типизирано междинно представяне, което може да бъде анализирано по време на компилация и материализирано по различен начин според целевата система.

На равнище модел на данните библиотеката демонстрира как `property`, `relationship`, `store_as` и `table_name_trait` могат да играят ролята на единен логически слой. Този слой не е ограничен до релационната интерпретация, а служи като общо описание на данните и връзките между тях. Именно това позволява една и съща структура на приложно ниво да бъде пренасяна към таблици, документи, ключове и стойности, графови възли или широки колонни редове.

На равнище архитектура характеристиката на конектора (`connector_trait`

t) е формализирана като централен механизъм за разширяемост. Именно чрез нея библиотеката адаптира едно и също междинно представяне към SQL, BSON-подобни филтри, Redis команди, Cypher и CQL. Механизмът на възможностите (възможност) допълва тази архитектура, като прави недопустимите операции видими още при компилация.

На равнище верификация разработката се опира на систематичен набор от модулни и интеграционни тестове, които покриват слоя на същностите, конструктора на заявки, механизма за подготвени заявки, миграциите, нишката безопасност и конкретните конектори. По този начин архитектурните твърдения са подкрепени с реални доказателства от кода и тестовата инфраструктура.

11.3. Научни и приложни приноси

Първият принос на работата е демонстрацията, че практически полезна библиотека за обектно-релационно съпоставяне по време на компилация може да бъде реализирана в стандартен C++. Това се постига без външна кодогенерация, без виртуален интерфейс на конектора и без принуждаване на потребителя да напуска идиоматичния C++ модел.

Вторият принос е в изясняването на връзката между C++ описанието на същността и физическата организация на данните. Показано е, че C++ моделът може да бъде третиран като първичен носител на логическата схема, докато конкретната целева система определя начина на материализация. Тази постановка има стойност не само за OPC библиотеките, а и по-общо за системите, които се опитват да комбинират силна статична типизация с хетерогенни източници на данни.

Третият принос е формализирането на договор на конектора, който обединява разширяемост и строга дисциплина по време на компилация. Този договор прави възможно надграждането на библиотеката с нови системи за съхранение, без да се разрушават вече съществуващите гаранции в програмния интерфейс за заявки. Точно тук работата се различава от подходи, които предлагат общ интерфейс, но скриват критичните разлики между целевите системи.

Четвъртият принос е показването, че верификацията на подобна библиотека трябва да бъде многостепенна. Не е достатъчно само “да има тестове” или само “да има проверки по време на компилация”. Необходима е комбинация от двете: ограничения на ниво концепти, утвърждаване на възможности, тестове за рендиране на заявки, интеграционни

сценарии на живо и проверки на миграциите и нишката безопасност.

Таблица 26. Връзка между целите на дипломната работа и реализираните резултати

Цел	Реализиран механизъм	Доказателствена опора
Типизиран модел на данните	property, relationship, table_name_trait	заглавни файлове на същности и модулни тестове за свойства/връзки
Типизиран слой за заявки	конструктори на заявки и типизирано междинно представяне	заглавни файлове на заявки, tests / PëP«PÿCñPъP,,Pç/test_PçPөCSPÿPөPө .cpp, tests/PçP,,CñPхPÿCГPөCЗPçP«P,,Pç/test_mockdb.cpp
Разширяем слой за множество целеви системи	договор на конектора и labelи за възможности	заглавни файлове на конектори и платформено-специфични модулни тестове / тестове на живо
Посока, съобразена със схемата	migrate<DB> и DDL куки	tests/PëP«PÿCñPъP,,Pç/test_schema_migration.cpp
Безопасна експлоатационна употреба	помощници за нишкова безопасност и транзакции	tests/PëP«PÿCñPъP,,Pç/test_P,,PçCïPөPө_safe ty.cpp

Таблица 26 показва, че дипломната работа не остава на равнището на декларативен архитектурен документ (архитектурен документ). За всяка основна цел има конкретен реализиран механизъм и поне един пряк технически корелат в хранилището.

11.4. Основни ограничения и граници на обобщението

Работата също така ясно показва, че ОРС абстракцията за множество целеви системи има граници. Не всеки модел за съхранение предлага едни и същи операции, а следователно общото междинно представяне не може да бъде интерпретирано еднакво

навсякъде. Именно затова механизмите за възможности и ограничения са толкова важни. Те не отслабват библиотеката, а я правят коректна спрямо различните системи.

Допълнително, макар архитектурната основа за миграции, нишкова безопасност и по-ниско ниво на изпълнение вече да съществува, тези подсистеми остават естествено поле за по-нататъшно развитие и емпирично разширяване. Това означава, че работата трябва да бъде оценявана като завършена архитектурна демонстрация и демонстрация на реализацията, но не и като последна дума по темата за достъп до данни по време на компилация.

От академична гледна точка е важно да се подчертае и още една граница. Настоящата разработка не доказва, че ОРС по време на компилация е универсално най-добрият подход за всички приложения. Тя доказва нещо по-точно: че за определен клас системи — системи с нужда от ясна статична структура, висока типова безопасност (типова безопасност) и контролирана разширяемост към множество целеви системи — този подход е жизнеспособен и инженерно защитим.

11.5. Значение за практиката и за бъдещите изследвания

Практическата стойност на разработката се състои в това, че библиотеката вече осигурява стабилна основа за по-нататъшно инженерно развитие. Описанието на същностите, междинното представяне на заявките, договорът на конектора и стратегията за верификация са достатъчно ясно формулирани, за да позволят дисциплинирано добавяне на нови целеви системи и експлоатационни възможности.

Изследователската стойност е свързана с по-общия въпрос за ролята на типовата система в моделирането на достъпа до данни. Работата показва, че типовата система на C++ може да служи не само за описание на данни и алгоритми, но и за формализиране на архитектурни граници, допустимост на операции и преход между логически и физически слоеве. Това поставя библиотеката в по-широк контекст, който надхвърля ОРС темата в тесния смисъл.

11.6. Заключение бележки

Направеният анализ потвърждава, че типовата система на C++ е достатъчно мощна, за да служи не само за описване на данни и алгоритми, но и за формализиране на достъпа до персистентни системи. Комбинацията от статичен модел на същностите, междинно

представяне, конекторен слой, базиран на характеристики, и систематична верификация показва устойчив път към библиотеки, които са едновременно изразителни, безопасни и надграждаеми.

По този начин дипломната работа постига както инженерна, така и изследователска цел: изгражда работеща библиотека и едновременно формулира принципи, които могат да бъдат използвани при бъдещи разработки на слоеве за достъп до данни по време на компилация. Най-същественият извод е, че подходът за обектно-релационно съпоставяне по време на компилация не е просто теоретично упражнение, а реална инженерна алтернатива за системи, които изискват строгост, яснота и разширяемост при работа с множество модели за съхранение.

ЛІТЕРАТУРА

- [1] ISO/IEC JTC1/SC22/WG21, “ISO/IEC 14882:2024 – programming language C++,” international standard, International Organization for Standardization, 2024.
- [2] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [3] D. R. Hipp, “SQLite C/C++ interface – an introduction.” <https://www.sqlite.org/cintro.html>, 2000. Online documentation.
- [4] The PostgreSQL Global Development Group, “libpq — C library for PostgreSQL.” <https://www.postgresql.org/docs/current/libpq.html>, 1996. Online documentation.
- [5] Oracle Corporation, “MySQL connector/C – C client library for MySQL.” <https://dev.mysql.com/doc/c-api/en/>, 2000. Online documentation.
- [6] D. R. Hipp, “SQLite: A self-contained, serverless, zero-configuration, transactional SQL database engine.” <https://www.sqlite.org/>, 2000. Online documentation.
- [7] The PostgreSQL Global Development Group, “PostgreSQL documentation: Asynchronous command processing in libpq.” <https://www.postgresql.org/docs/current/libpq-async.html>, 2026. Online documentation, accessed 2026-04-06.
- [8] Oracle Corporation, “MySQL C api developer guide: C api asynchronous interface.” <https://dev.mysql.com/doc/c-api/8.4/en/c-api-asynchronous-interface.html>, 2026. Online documentation, accessed 2026-04-06.
- [9] Redis Ltd. and hiredis contributors, “hiredis — minimalistic C client library for Redis.” <https://github.com/redis/hiredis>, 2026. GitHub repository and documentation, accessed 2026-04-06.
- [10] DataStax, “DataStax C/C++ driver: Futures.” <https://datastax.github.io/cpp-driver/2.6/topics/basics/futures/>, 2026. Online documentation, accessed 2026-04-06.
- [11] MongoDB, Inc., “libmongoc: mongoc_client_pool_t.” https://mongoc.org/libmongoc/current/mongoc_client_pool_t.html, 2026. Online documentation, accessed 2026-04-06.
- [12] C. Leishman, “libneo4j-client — C client library for Neo4j.” <https://neo4j-client.net/>, 2026. Online documentation, accessed 2026-04-06.

- [13] M. Hryniuk and M. Bao, “SOI: The c++ database access library.” <https://soci.sourceforge.net/>, 2004. Online documentation.
- [14] R. Witt, “sqlpp11: A type safe embedded domain specific language for SQL queries and results in C++.” <https://github.com/rbock/sqlpp11>, 2014. GitHub repository.
- [15] Code Synthesis Tools CC, “ODB: C++ object persistence framework.” <https://www.codesynthesis.com/products/odb/>, 2010. Online documentation.
- [16] ISO/IEC JTC1/SC22/WG21, “ISO/IEC 14882:2020 – programming language C++,” international standard, International Organization for Standardization, 2020.
- [17] ISO/IEC JTC1/SC22/WG21, “P0912R5: Merge coroutines TS into C++20 working draft.” <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0912r5.html>, 2019. ISO/IEC JTC1/SC22/WG21 proposal.
- [18] ISO/IEC JTC1/SC22/WG21, “P1063R0: Core coroutines.” <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1063r0.pdf>, 2018. ISO/IEC JTC1/SC22/WG21 proposal.
- [19] Apple Inc., “kqueue(2) — kernel event notification mechanism.” https://developer.apple.com/library/archive/documentation/System/Conceptual/ManPages_iPhoneOS/man2/kqueue.2.html, 2009. Manual page.
- [20] Google, “Google Benchmark user guide.” https://google.github.io/benchmark/user_guide.html, 2026. Online documentation, accessed 2026-04-06.